A | 程序与工具

一个支持工单应用最终要接收问题、查找数据、作出判断，并交给人或其他程序继续处理。在接入网络、数据库和 AI 以前，先要有一层能看清、能修改、出了错能找回原因的程序基础。

这一册从“到底运行了哪份文件”开始，逐步进入 Python 的值、条件、循环和函数，再让程序调用现成代码、接受测试、保存文件。后半册把运行环境、进程和版本接上，最后用一个本地工单整理器汇合这些关系。

学完这里的目标，不是背下一套命令，而是遇到一个小程序时，能解释它怎样得到输入、在哪一步改变值、把结果交给谁；修改以后能验证，换个干净环境也知道怎样重新运行。

第一次只看这里

直接进入 A01。第一次只追三件事：电脑实际运行了谁、值怎样在程序里变化、结果用什么证据确认。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

A01 | 这条命令会运行哪个文件？ 3

A02 | 输入怎样变成可继续使用的结果？ 9

A03 | 程序为什么走这一条路？ 19

A04 | 下一次从哪里来，又在哪里停？ 30

A05 | 出错以后，程序怎样继续？ 42

A06 | 别处的代码，怎样为当前程序工作？ 63

A07 | 怎样知道这次修改真的修好了？ 75

A08 | 程序结束后，数据怎样留下来？ 85

A09 | 怎样确认运行条件没有找错？ 98

A10 | 程序运行后，怎样观察和停止？ 107

A11 | 怎样保存和找回一份确定的代码？ 121

A12 | 离开当前环境还能重新运行吗？ 132

A13 | 把输入整理成一份可靠报告 145

知识关系总览 | 把程序与工具放回同一张图 167

文件操作查用 | 先核对目标，再复制、移动或清理 174

本册总览（需要时再回看） 178

本册知识结构总览 179

这册怎样展开 179

A01 | 这条命令会运行哪个文件?

写好 Python 代码后，首先要让电脑运行到正确的那份文件。如果修改的是一份，运行的却是另一份，就容易误以为代码没有改对。

这一章要弄清楚：**一条运行命令，到底怎样选中一个文件?** 我们会从两个同名文件开始，看看当前位置怎样影响文件选择，再比较两种指定路径的方法，最后判断“找不到文件”时应该先检查哪里。

弄清这件事，后面运行更复杂的程序时，就能先确认运行的是谁，再判断它内部的代码。这里建立的位置关系，也会在后续读取文件和启动测试时继续用到。

> **第一次先做一个判断：**编辑器里看到的文件、命令给出的路径、PowerShell 当前所在的目录，可能是三个不同的位置。先只追这三者怎样合成“实际打开的文件”；`Get-Location`、`Set-Location` 和 `Resolve-Path` 是随后用来观察的工具，不需要先背下它们。

先按下面给出的文件和结果推演；实际跟着操作前，先完成后面“在电脑上试一遍”中的准备。假设电脑里有两份名称相同的 `hello.py`：

```
D:\FDE-Lab\A01
├── 前台
│   └── hello.py
└── 后台
    └── hello.py
```

前台文件的内容是：

```python
print("FRONT FILE")
```

后台文件的内容是：

```python
print("BACK FILE")
```

这两份 `hello.py` 都是保存着 Python 代码的文本文件，也叫脚本（script）。这里的 `print` 把括号里的文字显示出来，两份文件因此有不同的输出。“前台”和“后台”只是文件夹的名字；修改其中一份文件，不会改变另一份。

本章的真实对象不是命令本身，而是这两份会输出不同标记的文件。每次看到 `FRONT FILE` 或 `BACK FILE`，都在回答同一个问题：**这次 Python 实际打开了谁?**

现在，编辑器里打开的是前台文件，PowerShell 中输入的命令却显示了 `BACK FILE`：

```
POWERSHELL
python .\hello.py
```

为什么选中了后台文件？答案藏在 `.` 从哪里开始。

这条路径从哪里开始找？

`python` 表示启动 Python。Python 是读取并执行这些程序文件的软件；命令后面的 `.\hello.py` 告诉它要打开哪份文件。

路径可以理解为文件的地址。这里的 `.` 表示当前目录，`\` 分隔路径中的不同部分，所以 `.\hello.py` 可以读成“当前目录里的 hello.py”。但当前目录到底是哪一个？

PowerShell 会记住自己当前所在的文件夹。这个位置叫当前工作目录（current working directory），它可以和编辑器打开的文件所在目录不同。`Get-Location` 用来查看这个位置：

```
POWERSHELL
Get-Location
```

假设结果中的路径是：

```
D:\FDE-Lab\A01\后台
```

在本例这样直接启动 Python 时，Python 会沿用这个出发目录。于是目标可以具体展开：

```
出发目录  D:\FDE-Lab\A01\后台
命令给出  .\hello.py
实际目标  D:\FDE-Lab\A01\后台\hello.py
```

Python 打开的是后台文件，因此显示 `BACK FILE`。编辑器正在展示哪份文件，没有改变这条命令的目标。

`.\hello.py` 这种需要结合出发目录的写法，叫相对路径（relative path）。**相对路径必须和当前工作目录放在一起，才能确定实际目标。**

命令不变，只换当前位置会怎样？

要改变 PowerShell 的当前位置，可以使用 `Set-Location`。它只改变这个会话接下来从哪里工作，不会移动或修改文件。第一次只需看清：它把“从哪里出发”改成了前台目录。`-LiteralPath`、引号和含空格路径的写法属于查用细节，命令需要照抄时再回来看旁边的说明。

先把当前位置设到前台，再运行刚才那条命令：

```powershell
Set-Location -LiteralPath 'D:\FDE-Lab\A01\前台'
python .\hello.py
```

这里真正要观察的只有一件事：当前位置改变后，同一条相对路径指向了另一份文件。完整选项的含义不改变这条因果关系。

此时 `.\hello.py` 指向前台里的文件，输出：

```
FRONT FILE
```

再把当前位置设回后台，命令仍然相同：

```powershell
Set-Location -LiteralPath 'D:\FDE-Lab\A01\后台'
python .\hello.py
```

这次输出 `BACK FILE`。两次都使用 `.\hello.py`，改变的是出发目录，所以打开了不同的文件。这也解释了开头的困惑：改对了文件，还需要让运行命令指向它。

不改变当前位置，也能指定前台文件吗？

可以。在这个例子里，前台文件的完整地址是：

```
D:\FDE-Lab\A01\前台\hello.py
```

这样从盘符一直写到文件名，已经把目标写完整，叫绝对路径（absolute path）。即使 PowerShell 仍在后台目录，下面的命令也会打开前台文件，显示 `FRONT FILE`：

```powershell
python 'D:\FDE-Lab\A01\前台\hello.py'
```

它不需要用当前位置补全地址，所以改变当前位置不会改变这次选中的脚本。不过，文件换到另一台电脑的其他位置后，这条绝对路径也要随之改变。相对路径则可以随整个项目一起移动，只要仍从合适的位置出发。

> **第一次学到这里先停。**如果你能用“出发目录 + 路径 → 实际目标文件”解释为什么会出现 `FRONT FILE` 或 `BACK FILE`，A01 的核心关系已经建立。后面的错误路径、`Resolve-Path` 和准备命令，是遇到定位问题时的查用证据。

两种路径都能正确指定文件。选择时要看：这次希望直接写清完整地址，还是利用项目内部已有的位置关系。

这次，运行的出发目录和脚本所在目录不同：

> PowerShell 当前目录　D:\FDE-Lab\A01\后台
> Python 沿用的目录　D:\FDE-Lab\A01\后台
> Python 打开的文件　D:\FDE-Lab\A01\前台\hello.py

PowerShell 负责找到并启动 Python，Python 取得文件路径与工作目录，再打开文件执行代码。终端（terminal）则是输入命令和查看结果的界面。界面上显示了前台文件的输出，并不表示命令的出发目录也变了。

运行过前台文件，下一条命令就从前台出发吗？

这里很容易混淆“刚运行的文件在哪里”和“PowerShell 当前在哪里”。先试着用已有的路径关系判断一次。

仍使用开头的目录结构：只有前台和后台各有一份 `hello.py`，A01 根目录没有第三份。现在 PowerShell 处于 `D:\FDE-Lab\A01`，依次运行：

```powershell
python .\前台\hello.py
python .\hello.py
```

第二条会显示哪份文件的文字，还是无法打开文件？可以先在脑中展开它指向的完整地址，再往下读。

第一条以 A01 为出发点，沿 `前台\hello.py` 定位到 `D:\FDE-Lab\A01\前台\hello.py`，显示 `FRONT FILE`。这里是在确定文件地址，没有改变 PowerShell 的当前位置。

这份脚本只打印文字。它结束以后，PowerShell 仍在 `D:\FDE-Lab\A01`，并没有执行过 `Set-Location`。所以第二条从同一个出发点查找：

> 出发目录　D:\FDE-Lab\A01
> 命令给出　.\hello.py
> 实际目标　D:\FDE-Lab\A01\hello.py

这份文件不存在，因此第二条无法打开脚本。**运行某个目录中的脚本，不会自动把 PowerShell 的当前位置改到那个目录。**

如果刚才认为第二条也会显示 `FRONT FILE`，需要修正的是这个区别：脚本所在目录和命令的出发目录可以不同。在第一条之后查看 `Get-Location`，就能看到 PowerShell 仍然在 A01。

文件没打开时，错误信息会告诉什么？

上面第二条命令可能显示这样的错误片段：

> can't open file 'D:\FDE-Lab\A01\hello.py': [Errno 2] No such file or directory

错误开头的 Python 安装路径可能因电脑而异。这里先看 `can't open file`（无法打开文件），以及紧跟着的完整目标：它指向 A01 根目录，而不是前台文件夹。

Python 已经启动，才会尝试打开这个目标；但文件没能打开，里面的 `print` 就没有执行。此时修改前台文件的代码，不会让这个不存在的路径变正确。把命令恢复成 `python .\前台\hello.py`，或使用正确的绝对路径，才解决了当前问题。

Python 已经启动，不等于目标脚本已经开始执行。 看到这类错误，先核对其中写出的目标路径，不要把它当成文件内部代码运行出错。

路径更长时，怎样核对自己的判断？

前面的路径很短，可以直接展开。项目目录变深以后，`Resolve-Path` 能帮助核对一个已有文件的完整位置。它只是查看工具，不会运行 Python，也不改变文件。

例如先把当前位置设到 A01，再核对前台文件：

```powershell
Set-Location -LiteralPath 'D:\FDE-Lab\A01'
Resolve-Path -LiteralPath '.\前台\hello.py'
```

结果中的路径应当是 `D:\FDE-Lab\A01\前台\hello.py`。它和自己推导的目标是否一致？如果不一致，先检查出发目录和写下的路径；如果目标不存在，这个命令也会报错。

用工具核对之后，真正运行时仍是 Python 接收路径并打开文件。`Resolve-Path` 并非每次启动都要执行的步骤。

在电脑上试一遍

前面的例子已经给出了理解所需的文件内容和结果。实际操作时，先用文件资源管理器建立一个独立练习目录，按开头的树形结构放好两份文件，再从 Windows 开始菜单搜索并打开 PowerShell。不要覆盖已有同名文件；如果示例目录已被使用，可以另取一个名字，并相应替换命令中的路径。

两份文件都用纯文本编辑器保存，文件名真正以 `.py` 结束，避免变成 `hello.py.txt`；保存编码选择 UTF-8。没有 D 盘时，使用自己的练习目录即可。编辑器保存文件后，再运行命令核对输出。

可以直接重现“运行过前台文件，下一条命令就从前台出发吗”那一段。两条运行命令之前，先用 `Set-Location` 设到自己的 A01 练习目录，再查看 `Get-Location`；第一条运行结束后再查看一次，就能把纸面判断与实际位置对应起来。

如果 PowerShell 提示找不到 `python`，先检查 Python 的启动入口：

```powershell
Get-Command python
python --version
```

第一条查看 `python` 会找到什么命令，第二条尝试显示 Python 的版本。找不到命令、跳出意外界面或没有显示版本，都说明入口还需要处理，尚不能用这些结果判断 `hello.py` 的内容。保留完整提示；安装和入口问题可以参照 Python 官方 Windows 使用说明中的安装或 Troubleshooting（排错）部分，处理后再回到文件定位。

以后遇到同样的问题，应该想起什么？

遇到“改了代码，输出却像另一份文件”，先想到命令的目标：用当前位置与路径推导打开了谁，再用本例这类独有输出核对执行到了哪里。这个判断换一组文件夹名字后也应该成立，不需要记住 前台、后台 或教材原句。

`.` 表示当前目录，这个短入口值得直接认得。查看位置、改变位置和核对完整目标这几个动作，会在后续任务中逐渐用顺；命令忘了可以查，但应当知道自己为什么需要它，以及返回结果说明什么。完整选项和示例盘符不需要默写。

本章追到了 Python 打开文件的位置。下一章从正确文件已经开始执行的位置继续：输入的文字怎样变成值，又怎样交给后面的计算。

A02 | 输入怎样变成可继续使用的结果?

程序运行到了正确的文件，接下来要看文件里的代码做了什么。比如活动有 12 人报名，每人准备 2 瓶水：人很容易算出 24，程序却得先取得输入、认出它是什么，再完成计算。

这一章沿着一个物资估算器，弄清楚**输入的文字怎样变成能计算、能交给下一步使用的结果**。先让一个小程序从输入走到输出，再处理文字和小数的实际需要，最后把计算交给自己编写的函数，看看结果怎样返回。

这条关系会贯穿后面的程序：屏幕上显示什么，与代码手里拿到了什么，需要分开看。先追清值的来去，之后才容易判断条件为什么成立、函数为什么算错，或者哪一步没有交回结果。

先让程序问一句，再把输入显示出来

物资估算器先做一件小事：询问报名人数，并原样显示收到的内容。下面两行就是一个完整程序：

```python
raw = input("报名人数：")
print(raw)
```

运行后，在提示后输入 `12`，按回车，会看到：

```
报名人数：12
12
```

第一行的 `12` 是键盘输入，第二行的 `12` 是程序显示的结果。两行代码不是一起完成的：

1. `input("报名人数：")` 先显示括号里的提示，然后等待输入；按回车后，把读到的文字交回来。
2. `raw = ...` 中的 `=` 把右边得到的结果交给左边的名字 `raw`。以后写 `raw`，就能取到这次保存的输入。
3. `print(raw)` 取出这个值，交给 `print` 显示。

`input` 和 `print` 都是 Python 已经提供的函数（function）：可以通过名字调用的一段工作。名字后面的括号表示调用，括号里放这次要交给它的内容。`input` 接收提示文字，`print` 接收要显示的值。

代码里的引号表示一段文字，所以 `"报名人数："` 是提示文字；`raw` 没有引号，表示取这个名字当前对应的值。如果写成 `print("raw")`，显示的就只是三个字母 `raw`。

`raw` 叫变量（variable）。这个名字不是 Python 的特殊命令；这里用它表示原始输入。变量的意义在于：给已经得到的值一个名字，后面才能继续使用。

赋值先求右边的结果，再交给左边的名字。 第一行等待输入时，不能先跳到第二行；也不是先把 `raw` 交给 `input` 填写。

输入的是 12，为什么乘 2 得到 1212?

接下来按每人 2 瓶水计算。Python 中的 * 表示乘法；但它对文字还有另一种作用。先看这个看起来合理、结果却不对的版本：

```
PYTHON
raw = input("报名人数：")
total = raw * 2
print(total)
```

输入 `12`，输出是 `1212`，不是 `24`。

原因是 `input` 交回来的是文字，也叫字符串（string），在 Python 中记作 `str`。输入看起来像数字，不会让它自动变成数值。

> "12" 是文字：两个字符 1 和 2
> 12 是整数：可以表示十二个人

这里用引号表示值的种类；实际输入时只键入 `12`，不用键入引号。`print` 显示这两种值时都可能只显示 `12`，所以只看屏幕上的字形，不足以判断程序拿到的是什么。

字符串乘整数的含义是重复文字。`"12" * 2` 得到 `"1212"`，Python 按文字规则完成了工作，只是这不是人数计算需要的结果。

要把这段数字文字作为人数使用，可以调用 `int`。`int(raw)` 尝试根据 `raw` 产生一个整数（integer），这个整数的类型也叫 `int`：

```
PYTHON
raw = input("报名人数：")
people = int(raw)
total = people * 2
print(total)
```

现在输入 `12`，输出就是 `24`。沿着同一份输入追一遍：

> input 交回 "12"　　→ raw 对应文字 "12"
> int(raw) 得到 12　　→ people 对应整数 12
> people * 2 得到 24　→ total 对应整数 24
> print(total)　　→ 屏幕显示 24

`int(raw)` 产生了一个整数，但没有回头把 `raw` 改成整数。此时 `raw` 仍是文字 `"12"`，`people` 才是数值 `12`。同样，`people * 2` 产生新结果，不会把人数改成 24。

像 `int(raw)`、`people * 2` 这样能够求得一个值的代码，叫表达式（expression）。赋值语句先把右边的表达式求完，再让左边的名字对应这个结果。

第一次读到这里先停。 现在只确认一条主线：`input` 交回文字，`int` 产生整数，计算使用整数，`print` 显示结果。可以先用一个自己的输入预测 `total`，或运行这几行看输出；暂时不用继续背下面的格式、清理和小数写法。后面的内容仍属于本章，但会在需要对应问题时再用。

现在这个版本处理的是能转换的整数文字。`cat` 不能转换成人数，转换会报错；怎样处理这种失败，后面的异常章节会展开。另外，`-3` 可以转换成整数，但并不因此成为合理的报名人数：**转换决定得到什么类型的值，业务规则决定这个值能不能用。**

把文字和数字相加，为什么也不行？

有时还要多准备两瓶水。下面是一个独立的错误示例：

```python
raw = "12"
total = raw + 2
print(total)
```

失败发生在 `raw + 2`：左边是字符串，右边是整数，Python 不会猜应该相加还是拼接。错误属于 `TypeError`，这里先把它理解为“当前操作遇到了不相配的值类型”。因为右边没能得到结果，这个独立程序没有完成对 `total` 的赋值，最后的 `print` 也不会执行。

如果目标是十二加二，写 `int(raw) + 2`，得到整数 `14`。如果目标确实是拼接文字，`raw + "2"` 得到字符串 `"122"`；`str(2)` 也能把整数 `2` 转成文字 `"2"`，于是可以写 `raw + str(2)`。

选择取决于下一步要做什么。例如编号 `"0012"` 虽然全是数字，但需要保留前导零，就适合继续用文字；转换成整数后只剩 `12`。

怎样显示完整说明，又保留原来的数值？

只显示 `24`，人还得猜单位。可以把结果放进一句说明里：

```python
people = 12
total = people * 2
label = f"需准备 {total} 瓶水"
print(label)
```

输出：

> 需准备 24 瓶水

字符串前的 `f` 表示：遇到花括号，就把里面表达式的结果放进文字。这里 `{total}` 取到整数 `24`，用它组成整句显示文字。这种写法叫 f-string（格式化字符串）。没有前面的 `f`，`"需准备 {total} 瓶水"` 就只是一段普通文字，花括号也会原样显示。

此时两个名字承担不同工作：`total` 仍对应整数 `24`，可以继续计算；`label` 对应字符串 `"需准备 24 瓶水"`，交给人阅读。显示说明没有把原来的 `total` 改成文字。

输入两端有空格时，清理结果去了哪里？

人输入的文字可能带空格。假设原始输入是左右各有一个空格的 `" 12 "`，要显示清理后的内容，可以用字符串自带的 `strip()`：

```python
raw = " 12 "
cleaned = raw.strip()
print(f"原输入：[{raw}]")
print(f"清理后：[{cleaned}]")
```

输出中的方括号只是帮助看清空格的位置：

> 原输入：[12]
> 清理后：[12]

`raw.strip()` 的点号表示使用这个字符串上的操作；带括号的 `strip()` 才是实际调用。这样的操作叫方法（method）。这里没有在括号中额外指定规则，所以它按默认方式去掉两端的空白，不删除中间的字符。

字符串清理交回的是一个结果，不会原地改掉原字符串。 这里 `cleaned` 接住 `"12"`，`raw` 仍对应 `" 12 "`。如果只写 `raw.strip()` 而不使用返回结果，随后显示 `raw`，仍会看到原来的空格。

如果不再需要保留原输入，也可以重新赋给同一个名字：

```python
raw = " 12 "
raw = raw.strip()
print(f"[{raw}]")
```

这次输出 `[12]`。仍然是先计算右边，得到 `"12"`，再让 `raw` 对应这个结果；不是字符串自己被改写了。

对于这里的整数转换，`int(" 12 ")` 本来就能处理两端空白，所以物资程序不必先加 `strip()` 才能运行。清理文字在姓名、标签等需要整齐显示的地方有自己的用途，不是所有转换前都必须做的一步。

计算结果有小数，和显示几位小数是一回事吗？

物资估算不总是整瓶计数。例如把 7 升饮用水平均分给 3 人，每人的量就带小数。Python 的 `/` 表示除法：

```python
average = 7 / 3
print(average)
```

输出：

```
2.3333333333333335
```

这里得到的是浮点数（floating-point number），类型叫 `float`，用于表示这类小数计算。很多小数在电脑中只能近似保存，所以结果不是无限精确的数学小数。若输入文字本身是 `"7.5"`，可以用 `float("7.5")` 得到数值 `7.5`；`int("7.5")` 则不能直接把这样的文字转换成整数。

报告里可能只想显示两位小数。可以比较两种做法：

```python
average = 7 / 3
rounded = round(average, 2)
label = f"{average:.2f}"

print(rounded)
print(label)
```

两行都显示 `2.33`，但程序中的值不同：

写法	得到什么	接下来适合做什么
`round(average, 2)`	舍入后的数值 `2.33`	继续进行数值计算
`f"{average:.2f}"`	显示文字 `"2.33"`	组成报告或界面文字

`round` 的第二个值 `2` 指定舍入到小数点后两位。f-string 中，冒号后面的 `.2f` 指定按固定小数形式显示两位；它调整的是生成的文字。两种写法都没有改掉原来的 `average`。

例如 `rounded + 1` 能继续做数值加法，`label + 1` 则不能把字符串和整数直接相加。这里值得分清的是计算用值与显示文字的关系；格式说明的具体拼法可以在需要时查，不必现在背一份格式大全。

计算能不能交给一个自己定义的函数？

回到整瓶水。上午活动有 12 人，每人 2 瓶；下午活动有 5 人，每人 3 瓶。两次都要做“人数乘每人用量”，只是交进去的数不同。

直接分别写 `12 * 2` 和 `5 * 3` 完全可以。若希望把这个计算作为一个有名字、可重复调用的工作，可以自己定义函数。先看一个独立、完整的例子：

PYTHON

```
def calculate_bottles(people, per_person):
    bottles = people * per_person
    return bottles


morning = calculate_bottles(12, 2)
afternoon = calculate_bottles(5, 3)
total = morning + afternoon
print(total)
```

输出是 `39`：上午需要 24 瓶，下午需要 15 瓶，总共 39 瓶。

`def` 用来定义函数。`calculate_bottles` 是给这段计算起的名字，`people` 和 `per_person` 是调用时用来接收数值的名字，逗号分开两个位置；冒号后面缩进的两行属于函数体。这里每行缩进四个空格，不与最外层代码对齐。

读到 `def` 时，Python 建立这个函数的定义，**不会立刻运行缩进里的乘法；调用发生时，才进入函数体。** 所以执行并不是从第一行一路穿过所有缩进行。

追踪第一处调用：

```
morning = calculate_bottles(12, 2)
        └── 把 12、2 交给函数
          people = 12，per_person = 2
          bottles = 12 * 2，得到 24
          return bottles，把 24 交回
        └── 这次调用的结果是 24
morning 接住 24
```

`return` 结束当前这次函数执行，并把后面的值交回调用位置。此处调用在赋值的右边，所以调用结束以后，才轮到 `morning` 接住 `24`；并不是直接跳到文件末尾。

接下来第二次调用用 5 和 3 再做同一段计算，交回 15，由 afternoon 接住。之后 morning + afternoon 得到 39，最后才显示。

调用时真正传入的值叫实参（argument），函数定义中接值的名字叫形参（parameter）。这两个名称是在描述刚才已经看见的交接：第一个值交给 people，第二个值交给 per_person。名字不必和调用者的名字一样。

函数里面的名字，会自动出现在外面吗？

不会。在这个程序里，bottles 是函数内部计算时使用的局部名字，外部代码没有自动得到一个同名变量。return bottles 交回的是它对应的数值 24，外部用 morning 接住它。

下一次调用又有自己的一组参数和局部名字，算出 15，不会把前一次已经交给 morning 的 24 改掉。

因此，调用者不需要依靠函数内部叫什么名字才能取到结果。哪怕把内部的 bottles 统一改成 amount，只要仍返回同一个计算结果，外面的调用方式和结果都可以不变。

这个短例子用函数，是为了把交接看清楚，不是说每个乘法都必须包成函数。只出现一次的简单表达式可以直接写；一段计算有自己的用途、要重复调用，或者以后需要单独检查时，命名成函数才更有帮助。

函数里已经打印了 24，为什么外面还拿不到它？

把函数的 return 换成 print，运行这个独立对照程序：

PYTHON
```
def calculate_bottles(people, per_person):
    print(people * per_person)


needed = calculate_bottles(12, 2)
print(needed)
print(needed + 1)
```

前面先显示：

```
24
None
```

然后在 needed + 1 处报 TypeError。屏幕曾出现 24，但 needed 接住的不是它。

第一行输出来自函数内部的 print。函数随后走到末尾，没有用 return 交回瓶数，这次调用便交回 None——Python 中表示空缺的特殊值，不是整数 0，也不是文字 "None"。所以外部执行的是 needed = None。

`print(needed)` 把这个值显示成 `None`；再计算 `None + 1`，就无法得到瓶数。错误发生在加法，最后一次 `print` 还没来得及显示结果。

`print` 把内容显示给人，`return` 把值交回调用代码。屏幕上的输出不会自动成为调用结果。 `print()` 自身的返回值也是 `None`，不会把它显示的那段文字交回来。

计算函数如果要让后面的代码继续使用瓶数，就应返回计算结果；外部代码再决定要不要显示、怎样显示。函数也可以既显示又返回，但那是两个独立动作，不能互相代替。

两个地方都叫 total，它们会一起改变吗？

用一个小判断检查“名字在什么位置、结果交到哪里”这条关系。下面是完整程序，可以先判断最后两行各显示什么，再读解释：

```python
total = 100


def calculate_bottles(people, per_person):
    total = people * per_person
    return total


result = calculate_bottles(3, 2)
print(total)
print(result)
```

输出分别是 `100` 和 `6`。

最外层的 `total` 一开始对应 `100`。调用函数后，函数内部的局部 `total` 对应 `3 * 2` 的结果 `6`；`return` 把 `6` 交回调用位置，左边的 `result` 才接住它。这个程序没有给外层的 `total` 重新赋值。

如果判断成两行都是 `6`，混淆的是同名不等于同一个位置：函数内部给局部名字赋值，不会因此改掉外部同名变量。如果判断 `result` 是 `None`，就回到上一节的区别：这里实际执行了 `return total`，已经交回数值；并不需要先在函数里打印。

把输入、计算和结果使用连起来

现在可以把输入人数交给计算函数，并为这一场活动额外留一瓶备用。先用自然语言把任务串起来：取得人数文字，转换成人数，计算基本瓶数，加一瓶备用，显示采购数量。这样的动作草稿叫伪代码（pseudocode），它帮助安排工作，但不是 Python 会直接执行的代码。

下面是对应的完整程序，可以保存为 materials.py：

```python
def calculate_bottles(people, per_person):
    bottles = people * per_person
    return bottles


raw = input("报名人数：")
people = int(raw)
needed = calculate_bottles(people, 2)
# 每场固定多备一瓶，留作临时补充
order = needed + 1
print(f"需采购 {order} 瓶水，其中 1 瓶备用")
```

后面是注释，供人阅读，不会作为指令执行。这里的注释解释额外加一瓶的原因；真正完成加法的仍是下一行代码，注释不能代替它。

输入 12，会显示：

```
需采购 25 瓶水，其中 1 瓶备用
```

这次 return 交回的 24 有了明确的下一步用途：先由 needed 接住，再参加 needed + 1，生成采购数 25。最后一行才负责把采购数排成给人看的文字。

输入、计算和显示现在各自做着可见的工作。以后把键盘输入换成从文件取得人数，计算函数仍可以接收人数和每人用量；改变提示文字，也不必改乘法。能否这样分别修改，取决于计算函数仍然接收数值并返回数值，而没有把取得输入或显示文字的工作混进去。这就是这里分开这些工作的实际收益。

在电脑上核对

在自选的独立练习目录中，用纯文本编辑器把完整程序保存为 UTF-8 的 materials.py，确认不是 materials.py.txt，也不要覆盖已有文件。将 PowerShell 切到文件所在目录后运行：

```powershell
python .\materials.py
```

每次运行输入一个人数。输入 3、12、25 时，采购数应分别是 7、25、51，每次都包含一瓶备用。文件找不到时，先检查当前位置和命令路径；已经出现“报名人数”提示，则说明执行到了 input。

需要实际修改一次时，可以把完整程序中的 `calculate_bottles(people, 2)` 改成 `calculate_bottles(people, 3)`。输入 12，基本用量变成 36，加备用瓶后采购数是 37。如果仍显示 25，先核对保存和运行的文件，再检查调用位置是否确实传入了 3；不要仅凭变量名字猜它的值。

下一步会用到这里的什么？

这里最值得留下的不是一份物资程序原文，而是判断值的办法：当前拿到的是文字还是数字，右边产生了什么，哪个名字接住它，下一行又怎样使用它。函数调用还要多追一段：值从调用处交进去，函数体运行，结果通过 `return` 交回来。

`input(...)`、`int(...)`、赋值、函数调用和 `return` 会反复出现，需要在实际程序里逐渐用顺；变量名字和提示文字可以按任务改变，格式说明与低频选项可以查。忘记某个拼法时，仍应能指出自己要取输入、转换、计算、返回还是显示，才知道该查哪一个。

本章还没有处理“人数为负数”“超过场地容量”等规则。转换得到整数，只说明现在有了一个可以参与判断的值。下一章就从这里继续：同样是整数人数，为什么有的能继续，有的应该走另一条路线？

进一步查阅：CS50P Week 0 讲义讲解函数与变量；Python 内置函数说明可查 `input`、`int`、`print`、`round` 的具体行为；Python 函数教程说明定义、参数与返回关系。

A03 | 程序为什么走这一条路？

取得一个整数之后，程序往往还不能直接使用它。比如活动只接受 12 到 70 岁的人报名：`9`、`16`、`70` 都是整数，但应该得到不同的处理。把输入转换成功，与符合活动规则，是两件事。

这一章要看清**已有的值怎样经过判断，决定接下来运行哪一段代码**。先从一条年龄限制开始，再加入其他去向，比较“只选一条路线”与“分别检查几件事”；最后让一个报名函数返回明确结果。票号分流和票种名称会用来说明：条件需要的事实，有时来自计算，有时来自某个固定选项。

以后遇到“为什么输出了两次”“为什么符合条件却没走进去”，就可以沿实际检查顺序找原因，而不是只看某个条件单独是不是成立。

条件成立时执行，不成立时去哪？

先只检查最低年龄，不考虑名额或其他限制。下面是完整的小程序：

PYTHON
```
age = 16

if age >= 12:
    print("达到最低年龄")

print("本次检查结束")
```

它显示：

```
达到最低年龄
本次检查结束
```

`age = 16` 先让 `age` 对应整数 `16`。`>=` 表示“大于或等于”，所以 `age >= 12` 问的是“16 是否至少为 12”。比较产生 `True`，表示这个条件成立。

`if` 根据条件决定是否执行下面的缩进块。冒号表示后面接这段代码，缩进说明哪些行属于它；这里缩进四个空格的 `print` 才受年龄条件控制。最后一个 `print` 回到了最外层，不属于这个块。

如果把第一行改为 `age = 9`，比较产生 `False`，表示不成立。程序跳过缩进中的输出，仍然执行最后一行，于是只显示：

```
本次检查结束
```

`True` 和 `False` 是布尔值（Boolean），在 Python 中的类型叫 `bool`。它们不是带引号的提示文字。比较本身可以单独计算，例如：

```python
age = 9
allowed = age >= 12
print(allowed)
```

输出 `False`。比较得到一个值，赋值把它交给 `allowed`；有了这个结果，`if` 才能据此选择去向。

条件不成立，是跳过它控制的代码块，不是让整个程序停止。 条件成立、块内执行完以后，也会接着运行块外的下一行，除非代码里另有结束或改道的指令。

如果每个人都要得到一个明确结果呢？

现在补全这场活动的年龄规则：小于 12 或大于 70 岁拒绝；其余人中，12 到 17 岁走青少年路线，18 到 70 岁走普通路线。边界年龄都按刚才这句话处理，不让程序猜。

```python
age = 16

if age < 12:
    result = "拒绝：年龄不足 12 岁"
elif age > 70:
    result = "拒绝：年龄超过 70 岁"
elif age <= 17:
    result = "接受：青少年路线"
else:
    result = "接受：普通路线"

print(result)
print("本次检查结束")
```

`<` 是小于，`>` 是大于，`<=` 是小于或等于。`if`、`elif`、`else` 在这里对齐，组成一条分支链。它们下面缩进的赋值，各自属于对应分支。

`elif` 可以理解为“前面的条件没选中，再检查这里”。`else` 表示“前面都没选中，使用这条剩余路线”，所以不再写一个新条件。

年龄为 16 时，执行过程是：

```
16 < 12 → False，跳过第一段赋值
16 > 70 → False，再向下检查
16 <= 17 → True，把 result 设为“接受：青少年路线”
跳过这条链剩下的 else
来到链后的 print(result)，显示这一个结果
继续显示“本次检查结束”
```

年龄为 9 时，第一个条件就成立，后面的两个 `elif` 不会再检查。年龄为 70 时，三个比较依次得到 `False`，于是由 `else` 设置普通路线。

一条 `if/elif/else` 链从上往下找，选中一段后，就跳过这条链的其余分支。 有最后的 `else`，这里每个人都会得到结果；没有 `else` 时，也可能所有条件都不成立，整条链一段都不执行。

第三个条件为什么只写 age <= 17？

单看 `age <= 17`，9 岁也符合。但能走到这里的人，已经没有被前面的年龄下限和上限拦下，所以此处只需继续区分青少年与普通路线。

如果把 `age <= 17` 放到最前面，9 岁就可能先被接受；后面即使还有拒绝条件，也不会再检查。**检查顺序承载着规则的优先级，Python 不会自动把“拒绝条件”排到前面。**

这也说明 `else` 不是“输入一定合法”的保证。在这个程序中，前面的分支排除了哪些情况，决定了剩余路线能作出什么判断。

把 elif 换成另一个 if，结果为什么变了？

下面是一个独立的错误版本。为便于看清覆盖过程，只保留最低年龄与青少年分类：

PYTHON

```python
age = 9

if age < 12:
    result = "拒绝：年龄不足 12 岁"

if age <= 17:
    result = "接受：青少年路线"
else:
    result = "接受：普通路线"

print(result)
```

它竟然显示 接受：青少年路线。

第一个 `if` 没有失效：`9 < 12` 为真，`result` 确实先得到拒绝说明。但赋值不会结束程序。接下来遇到另一个独立 `if`，Python重新检查 `9 <= 17`，又把 `result` 改成了接受说明；最后显示的是后一次赋值。

这里的 `else` 只与第二个 `if` 配对。空一行不会把两个独立判断变成一条链；真正表达“前面没选中才检查这里”的，是 `elif`。

如果目标是给一个人选择一个最终年龄结果，就应使用前面的分支链。若目标是分别发出多个提醒，例如“年龄未满 18 岁需要监护人信息”和“没有名额需要候补说明”，多个独立 `if` 可以有各自的作用。要先分清业务需要，再选结构，不能把所有独立 `if` 都当成错误。

“任意一个不合格”和“全部都符合”怎样表达？

前面把两种年龄拒绝分成两段，是为了看清检查顺序。如果它们使用同一条拒绝说明，就可以把条件合起来。

任意一边成立：or

```python
age = 9

if age < 12 or age > 70:
    print("年龄不在允许范围内")
else:
    print("年龄符合要求")
```

`or` 表示这里的两个条件只要有一个成立，就足以选择拒绝路线。两个条件要分别写完整；代码检查的是 `age < 12` 和 `age > 70`。

它从左边开始判断。年龄为 `9` 时，`age < 12` 已经是 `True`，整个 `or` 的真假已经确定，右边的 `age > 70` 不再执行。年龄为 `71` 时，左边为 `False`，才继续检查右边并得到 `True`。年龄为 `16` 时，两边都不成立，走 `else`。

这种左边已经能决定结果、就不再计算右边的行为，叫短路（short-circuit）。读执行路线时不能把“右边也可以算出真假”写成“右边实际已经算过”。

两边都要成立：and

反过来，年龄符合要求，需要同时达到下限、不超过上限：

```python
age = 16
allowed = age >= 12 and age <= 70
print(allowed)
```

输出 True。and 在这里要求两边比较都成立。它同样先看左边：9 岁已经不能达到下限，便不必再计算右边；16 岁通过左边，才检查上限。

不要把这里的 and 换成 or。例如9岁虽然没达到下限，却满足“不超过70”，用 or 就会错误地认为整个年龄条件成立。

对于同一个值的连续范围，还能写得更直接：

```python
age = 16
allowed = 12 <= age <= 70
print(allowed)
```

这叫链式比较，读成“12 不大于年龄，年龄不大于 70”。这里与前面的两个比较用 and 相连意思相同；9 岁会在第一个比较处停下。

本节组合的是比较产生的布尔值，因此组合结果也是 True 或 False。以后遇到 and/or 两边直接放字符串等其他值时，还会涉及取值规则；不能把本节的范围例子外推成它们永远返回布尔值。

检查相等，与给名字赋值，不是同一个动作

票种也是活动规则的一部分。这里规定只能输入小写的 standard（普通票）或 support（支持票），其他文字不算有效票种。比较文字可以用 ==：

```python
ticket_type = "support"
allowed = ticket_type == "standard" or ticket_type == "support"
print(allowed)
```

第一行的单个 = 给名字赋值；第二行的 == 检查两边的值是否相等。"support" 不等于 "standard"，但等于 "support"，所以输出 True。

!= 则检查“不相等”。要表示“既不是普通票，也不是支持票”，可以写 ticket_type != "standard" and ticket_type != "support"。这一条件成立时，才说明两种允许票都不匹配。

如果已经有了布尔结果，还能用 not 反转它。例如 allowed 是 True，not allowed 就是 False；allowed 是 False，not allowed 就是 True。后面的报名函数会用它表示“检查不通过就退出”。

票号的奇偶不是现成事实，怎样先算出来？

已经获准入场的人还要分到两个入口。假设活动使用正整数票号，希望偶数票号走 A 入口，奇数票号走 B 入口。

普通除法告诉我们商，`%` 则取得除法的余数。`104 % 2` 得到 `0`，因为 104 能被 2 整除；`105 % 2` 得到 `1`。有了余数，才有可以比较的值：

```python
ticket_no = 105
remainder = ticket_no % 2

if remainder == 0:
    print("请走 A 入口")
else:
    print("请走 B 入口")
```

执行先得到余数 `1`，再比较 `1 == 0`，结果为 `False`，所以显示 `请走 B 入口`。把票号换成 `104`，得到余数 `0`，才选择 A 入口。

`%` 本身没有替程序选入口，它只算出余数。后面的比较产生布尔值，`if` 再决定显示哪条说明。这个算值再判断的过程，也可以用于轮转或其他周期关系；年龄是否在允许范围内，直接比较上下限即可。

多条不合格规则，能不能发现一条就返回？

现在让报名程序给后面的代码返回一个结果，而不仅仅在分支里打印。规则扩展为：

- 年龄必须在 12 到 70 岁之间，含边界。
- 票种只能是 `standard` 或 `support`；本例要求按提示输入这两个小写值。
- 年龄或票种不合格，先拒绝；不能因为没名额，就让不合格者进入候补。
- 合格者在剩余名额为零或负数时进入候补；负数在此例按没有可用名额处理。
- 有名额时，12 到 17 岁走青少年路线；其中支持票单独说明，其余合格者走普通路线。

年龄是否合格是一个可以单独回答的问题。下面这个完整小程序把比较结果返回，再显示它：

```python
def age_is_allowed(age):
    return 12 <= age <= 70


allowed = age_is_allowed(16)
print(allowed)
```

输出 `True`。`def` 建立函数，调用时 `16` 交给参数 `age`；范围比较得到布尔值，`return` 把它交回，外面的 `allowed` 接住。返回值不一定是瓶数，也可以是“是否符合要求”这个判断结果。

在下面的完整报名程序里，age_is_allowed 仍只回答年龄是否符合范围；
registration_result 按顺序检查整套规则，并返回最终说明。
if not age_is_allowed(age) 的意思是：先调用年龄函数，如果返回 False，not 把它变成 True，才进入拒绝代码。

程序末尾的 input 取得文字，int 把年龄与名额文字转换为整数。嵌套写法 int(input(...)) 仍然先做括号内部的输入，再转换结果；票种直接保留为文字。

PYTHON
```
def age_is_allowed(age):
    return 12 <= age <= 70


def registration_result(age, seats_left, ticket_type):
    if not age_is_allowed(age):
        return "拒绝：年龄不符合规则"

    if ticket_type != "standard" and ticket_type != "support":
        return "拒绝：票种不符合规则"

    if seats_left <= 0:
        return "候补：当前没有剩余名额"

    if age <= 17 and ticket_type == "support":
        return "接受：青少年支持票路线"
    elif age <= 17:
        return "接受：青少年路线"
    else:
        return "接受：普通路线"


age = int(input("年龄："))
seats_left = int(input("剩余名额："))
ticket_type = input("票种（standard/support）：")
result = registration_result(age, seats_left, ticket_type)
print(result)
```

输入 16、3、support，沿途是：

> age_is_allowed(16) → True
> not True → False，跳过年龄拒绝
> "support" != "standard" → True，需要继续检查 and 右边
> "support" != "support" → False，跳过票种拒绝
> 3 <= 0 → False，跳过候补
> 16 <= 17 → True，继续检查票种是否 support
> 票种相等 → True，进入青少年支持票路线
> return 把说明交回 result
> 最后的 print 显示这一条说明

这里没有自动减少名额，也没有保存报名记录；它只根据这次传入的值给出判断。真正占用名额和保存结果，是后续会增加的工作，不能因为出现“接受”两个字就当成已经完成。

前面说多个 if 会继续检查，为什么这里又可以？

区别在于分支里执行的动作。前面的错误程序只是给 result 赋值，做完后当然继续往下；这里发现年龄不合格，就执行 return，整次 registration_result 调用已经结束，后面的独立 if 没机会再执行。

例如年龄 9、名额 0、票种 support：年龄函数返回 False，not False 为真，立即返回年龄拒绝说明。候补条件虽然单独看也会成立，却根本没有被检查。

elif 控制同一条分支链还检查谁；return 结束的是当前整个函数调用。 它们有时都能让后面的代码不执行，但改变的范围不同。

先排除不合格情况以后，后面的代码能在“年龄和票种有效”这个前提上继续。最后用 age <= 17 分类，不会重新接受9岁，因为9岁早已返回。这种写法让后续判断少重复前提；如果一条简单分支链已经清楚，也不必为了使用提前返回而改造它。

有支持票、年龄也符合，为什么仍然候补？

仍使用上面的完整函数。给它 age = 17、seats_left = 0、ticket_type = "support"，先判断它会返回什么，再读下面解释。

结果是 候补：当前没有剩余名额。

年龄符合要求，票种也有效，前两处拒绝都跳过。随后 0 <= 0 成立，执行候补分支中的 return；函数到这里就结束了，不再检查青少年与支持票条件。外部接住候补说明，再显示出来。

如果判断为青少年支持票，最早漏掉的是“名额检查发生在通道分类之前”。不是后面条件写错了，而是它没有被执行。如果觉得两个结果都应该出现，就回到上一节的范围区别：本例的 return 已经结束函数；它不是打印一条消息后继续往下。

这段顺序也表达了设计选择：先决定能否安排入场，再决定入场通道。若活动要求改变，就要先明确新优先级，再相应改检查顺序，不能只把某个条件挪到最前面试运气。

只是把票种换成名称，需要整套报名判断吗？

不需要。比如只想把 `standard` 显示成“普通票”，把 `support` 显示成“支持票”，当前问题是一个值与几个明确选项的对应。

使用熟悉的分支链，可以写成这个独立程序：

```python
ticket_type = "support"

if ticket_type == "standard":
    label = "普通票"
elif ticket_type == "support":
    label = "支持票"
else:
    label = "未知票种"

print(label)
```

Python 3.10 及以上也可以用 `match` 表达这组具体文字的匹配：

```python
ticket_type = "support"

match ticket_type:
    case "standard":
        label = "普通票"
    case "support":
        label = "支持票"
    case _:
        label = "未知票种"

print(label)
```

`match ticket_type` 指定这次看哪个值；`case "support"` 表示匹配这个确切字符串；`case _` 接住前面都没匹配的情况。这里的下划线不是名为“_”的票种，而是剩余情况的匹配符。

`support` 会得到“支持票”，`standard` 得到“普通票”，`guest` 得到“未知票种”。选中一个 `case` 执行后，不会继续落入后面的 `case`。这两个独立程序在给定的票种映射上结果相同。

`match` 还可以匹配更复杂的结构，本节只使用文字选项，不需要提前展开那些用法。年龄范围或多个不同要求一起判断，前面的比较与条件组合已经足够；`match` 不替整套报名规则规定优先级。

若解释器不支持这个语法，可以在 PowerShell 查看 `python --version`，并先使用相邻的 `if/elif/else` 写法。语法不支持与输入了未知票种，是两种不同情况。

在电脑上核对这些边界

把前面的完整报名程序保存到自己的独立练习目录，文件名为 `registration.py`，使用 UTF-8 纯文本，不覆盖已有文件。从这个目录运行：

```powershell
python .\registration.py
```

每次运行依次输入年龄、剩余名额和票种。程序目前要求年龄、名额是能转换的整数文字；`cat` 会在转换时失败，不会进入报名函数。它与“年龄9岁被正常拒绝”不同，输入失败的恢复放在异常章节处理。

核对时可以使用下面这些边界，而不是只试一个普通年龄：

输入：年龄 / 名额 / 票种	应得到的结果
11 / 3 / standard	年龄拒绝
12 / 3 / standard	青少年路线
17 / 0 / support	候补
17 / 3 / support	青少年支持票路线
18 / 3 / support	普通路线
70 / 2 / standard	普通路线
71 / 2 / standard	年龄拒绝
20 / 3 / guest	票种拒绝

这些输入分别检验下限、上限、通道切换和检查优先级。先保持代码不变，只换输入，才能知道差别来自同一套规则怎样处理不同值。

从这里接到多条记录

现在值与执行路线已经连上：输入和转换提供值，比较形成条件，分支选择代码；函数通过 `return` 交回结果，调用处继续显示或处理。想修改某个年龄界限，应找规则里的比较；只改显示名称，则不需要改变输入转换。

读条件时，重要的是能用具体值指出“当前到底检查到了哪里，为什么下一段执行或没执行”。`=` 与 `==`、`if/elif/else` 的范围，以及布尔条件的组合需要在实际修改中逐渐用顺；具体年龄界限、票种名称和本例输出文字不是要背的内容。忘记 `match` 的完整写法时可以查，也可以用已经理解的分支链表达同一映射。

这份程序每次处理一条报名。真实活动还会不断收到下一条记录：下一章要解决下一条从哪里来、什么时候停止，以及怎样保留已经得到的结果。

进一步查阅：CS50P Week 1 讲义介绍条件判断、取余与匹配；Python 条件语句说明和布尔运算说明可用于核对执行规则。

A04 | 下一次从哪里来，又在哪里停？

一条报名可以得到一个判断，真实活动却会不断收到下一条。复制同一段代码十次，只能处理提前写好的十次；人数变化，或者工作人员还没决定何时结束，程序就需要自己回到下一轮。

这一章先看重复怎样发生、什么时候停止，再让多个人的姓名和状态留在程序里。我们会从一个小倒数开始，走到按顺序处理名单、按键查状态，最后组合成一份可录入、可汇总的名单程序。

循环负责继续处理，数据结构负责保留和组织结果。 看清这两种工作怎样配合，才能判断程序为什么停不下来、为什么漏了人，或者为什么处理完以后只剩最后一个结果。

同一段代码，为什么会再执行一次？

先用三个待处理名额演示倒数，每次显示当前剩余量，再减一。这个例子只演示计数，不代表已经完成真实报名或扣减数据库名额：

PYTHON
```
remaining = 3

while remaining > 0:
    print(remaining)
    remaining = remaining - 1

print("结束")
```

输出依次是 `3`、`2`、`1`、结束，各占一行。

`while` 根据条件重复执行下面的缩进块。它和 `if` 都会检查条件，区别在块执行完以后：`if` 接着向下走，`while` 要回到自己的条件重新检查。

> remaining 是 3 → 3 > 0 成立 → 显示 3 → 减到 2
> 回到条件 → 2 > 0 成立 → 显示 2 → 减到 1
> 回到条件 → 1 > 0 成立 → 显示 1 → 减到 0
> 回到条件 → 0 > 0 不成立 → 跳出循环，显示“结束”

`remaining = remaining - 1` 仍然先用右边的旧值计算，再把新值交给左边的名字。条件只读取当前值，不会自己把它减一。每一次进入并执行循环体，叫一轮迭代（iteration）；这里总共进入了三轮。

最后一轮把值改成 `0`，不会立刻中断正在执行的循环体。如果减一后还有其他语句，它们仍会继续；到下一次回到 `while`，才发现条件不成立。循环里的 `if` 也只在每次执行到它时判断一次，不会代替外面的 `while` 重复整段工作。

如果初值就是 `0`，第一次检查已经不成立，循环体一轮也不会执行，但最后的 `print` 仍会运行。

如果忘了减一，为什么不会自己停?

不需要实际运行无限版本，就能推演问题：

> 第 1 次检查：remaining 是 3，条件成立；显示 3，值仍是 3
> 第 2 次检查：remaining 还是 3，条件成立；仍显示 3
> 后面每次相同，没有任何一步让条件变成 False

`while` **会重新检查条件，但不会替代码推进条件依赖的值。** 排查重复不止时，应先找“下一轮看到的值究竟变没变”，不是猜程序还差几次才会停。若误运行了不断输出的版本，可以在终端按 Ctrl+C 中断，再修正代码；外部中断不等于循环设计已经正确结束。

已经有一份名单，下一位由谁提供?

如果内容已经在这里，程序可以直接逐项取出，不必自己维护倒数。

下面的方括号把两个姓名放进一份列表（list）；逗号分开其中的项目。列表按位置保存内容，可以保留重复值，也不要求所有值都必须是同一种类型；这里专门保存姓名，方便统一处理。

PYTHON
```
names = ["林青", "周岚"]

for name in names:
    print(name)

print("名单处理完毕")
```

输出：

> 林青
> 周岚
> 名单处理完毕

`for name in names` 可以读成“从 `names` 依次取出一项，交给 `name`”。第一轮 `name` 对应 `"林青"`，执行缩进中的输出；接着 Python 取得下一项 `"周岚"`，再执行一次。没有下一项时，才走到循环后的说明。

`name` 是接收本轮项目的普通变量名，不是特殊命令，也不是整份列表。第二轮会让它对应另一个值，但原来的列表仍保存两个人；没有因为一个人被取出显示，就自动把他从名单里删除。

能够依次提供项目的对象，称为可迭代对象（iterable）。列表是其中一种。`for` 的关键是向它取得下一项，不是一定要预先知道一个固定次数，也不是只能处理已经全部存好的列表。

只想按编号重复三次，也要手写三个项目吗？

不必。`range(3)` 提供从 `0` 开始、不包含 `3` 的整数序列，供循环依次取用：

```python
for number in range(3):
    print(number)
```

输出是 `0`、`1`、`2`。`range` 本身不执行打印；它提供值，`for` 每次取得一个值后才执行循环体。这也解释了为什么“for 只能处理列表”不准确。

还不知道会录入几个人，怎样取得下一条输入？

工作人员希望不断输入姓名，直到输入小写的 `end`。这时下一项来自未来的键盘输入，不能先把所有姓名写进列表。

先建立空列表 `[]`。每得到一个正常姓名，就用 `names.append(name)` 把它加到列表末尾；`append` 是列表的添加方法。遇到结束指令，用 `break` 立即离开当前循环：

```python
names = []

while True:
    name = input("姓名（输入 end 结束）：")

    if name == "end":
        break

    names.append(name)

print(names)
```

`while True` 的条件始终为真，所以不会靠条件变假结束；停止由循环体里的 `break` 负责。这里按提示输入准确的 `end`，它不作为姓名保存。

依次输入 林青、周岚、end，循环中的变化是：

> 最初 names = []
> 读到“林青” → 不等于 end → append 后保存 ["林青"]
> 回到 while → 读到“周岚” → append 后保存 ["林青", "周岚"]
> 回到 while → 读到“end” → break，跳过后面的 append
> 来到循环外的 print → 显示整份名单

实际最后会显示 `['林青', '周岚']`。这是 Python 显示列表的方式，列表外面的方括号与字符串的引号不是额外录入的人。

`input` 放在循环里面，所以每轮才有机会取得新输入。如果只在循环外读一次，然后反复检查那个没有更新的名字，就无法收到后来输入的结束指令。

这份小程序不校验空姓名，也不自动清理空格；它专门展示录入和停止的关系。后面真的需要姓名校验时，再在加入列表前判断，不能把“已经 append”说成“输入一定有效”。

append 与字符串清理的结果为什么不同？

`names.append(name)` 会改变这份列表本身；它的返回值是 `None`，不是更新后的列表。所以这里直接调用它，不能写成 `names = names.append(name)`，否则添加虽然发生了，`names` 随后却被重新赋成 `None`。

对比字符串的 `strip()`：它返回清理结果，原字符串不变，需要使用或接住返回值。**调用一个方法时，要看它改变了原对象，还是交回了一个结果，不能只凭点号和括号猜。**

此时 `while` 和 `for` 的选择也更具体了：录入阶段反复等下一次输入，并检查结束事件；拿到列表后，报告阶段可以用 `for` 依次处理其中的项目。它们可以在同一程序中承担不同工作，不需要把所有重复都改成同一种写法。

需要第几位时，位置和人数有什么区别？

列表可以按位置取值。位置也叫索引（index），从 `0` 开始。两个人的名单有两个项目，但有效位置是 `0`、`1`：

PYTHON
```
names = ["林青", "周岚"]
print(names[0])
print(names[1])
print(len(names))
```

三行输出分别是 林青、周岚、2。`len(names)` 取得项目数量，不是最后一个有效位置。只需每个人的名字时，直接 `for name in names` 更清楚；只有需要原始位置时，才需要多做索引这一步。

例如给报告中的姓名标出原始顺序：

```python
names = ["林青", "周岚"]

for index in range(len(names)):
    name = names[index]
    print(f"第 {index + 1} 位：{name}")
```

`len(names)` 先得到 `2`，`range(2)` 提供位置 `0`、`1`。每轮的 `names[index]` 再根据这个位置取姓名。显示时加一，是为了给人看“第1位、第2位”，并没有改变列表内部的索引规则。

`range(len(names))` **只提供位置，**`names[index]` **才取得相应的值。** 字符串也可以按位置取字符；例如 `word = "cat"` 时，`range(len(word))` 提供 `0`、`1`、`2`，`word[1]` 才得到 `"a"`。

若写 `names[len(names)]`，这里就是访问不存在的 `names[2]`，会发生 `IndexError`，意思是索引越过了可用范围。循环不会为错误索引自动补一个项目。

知道报名编号，不知道它排第几位，怎样查状态？

按位置找人，与按编号查状态，是不同的取回方式。下面使用字典（dictionary，类型名 `dict`）保存编号和状态的对应关系：

```python
status_by_id = {
    "R101": "accepted",
    "R102": "waitlist",
}

print(status_by_id["R102"])
```

输出 `waitlist`。`accepted` 表示已接受，`waitlist` 表示候补；`R101`、`R102` 只是这份独立示例的报名编号。

花括号表示字典，每个冒号左边是用于查找的键（key），右边是对应的值（value）；逗号分开各组对应。`status_by_id["R102"]` 不是“取第 R102 位”，而是“用这个键找到它对应的值”。

如果只想改 R102 的状态，可以赋值：

```python
status_by_id = {"R102": "waitlist"}
status_by_id["R102"] = "accepted"
print(status_by_id["R102"])
```

输出 accepted。同一个键再次赋值，会替换它的旧值，不会在字典里新增第二个同名键。因此，想保留两条不同记录，不能把它们都塞到同一个键下面后期待自动留住历史。

不存在的键，会被自动忽略吗？

不会。直接读取不存在的 status_by_id["R999"] 会发生 KeyError，表示找不到这个键。若“编号可能查不到”是正常情况，应由程序明确处理：

```python
status_by_id = {"R101": "accepted", "R102": "waitlist"}
search_id = "R999"

if search_id in status_by_id:
    print(status_by_id[search_id])
else:
    print("没有这个报名编号")
```

这里 in 判断键是否存在，不是在字典的值里找相同文字。结果为 False，所以没有执行方括号查值，转而显示未找到。它和 for name in names 中连接本轮变量与数据来源的 in 所在结构不同：一个是成员判断，一个是循环取项。

也可以直接用 for 遍历字典，但得到的是键，再用键取对应的值：

```python
status_by_id = {"R101": "accepted", "R102": "waitlist"}

for registration_id in status_by_id:
    print(registration_id, status_by_id[registration_id])
```

输出 R101 accepted、R102 waitlist，各占一行。print 中的逗号让它接收两个值，默认用一个空格分开显示。循环不需要猜编号，也没有把字典误当成姓名列表。

一个人有多个字段，多个人又该怎样放？

一条记录可以用字典把姓名、状态放在一起，多条这样的记录再按顺序放进列表：

```python
records = [
    {"name": "林青", "status": "waitlist"},
    {"name": "周岚", "status": "accepted"},
]

for record in records:
    print(record["name"], record["status"])
```

输出是 林青 waitlist、周岚 accepted，各占一行。

外面的方括号保存多条记录；里面每组花括号保存一条记录的字段。第一轮，record 取得第一整条字典，再用 "name" 和 "status" 这两个键分别取值。下一轮取得第二条字典，不会让上一条从列表里消失。

同样是字典，上一节用报名编号作键，这里用字段名作键。两者都遵守键到值的对应规则，只是当前要取回的东西不同。records[0]["status"] 就是先按位置取得第一条字典，再按字段键得到 "waitlist"；它是两次取值，不是一个特殊字段名。

这种“列表里放字典”的结构适合多条字段相似的记录，但不会自动保证每条都有 "status"。缺少时直接读取仍会报 KeyError；正式接收外部记录时，需要先验证必需字段。

怎样在循环结束以后，仍记得出现过候补？

报告不只逐条显示，还想在最后提醒“名单中是否有人候补”。只看循环最后一条不够，因为候补可能出现在前面。

PYTHON

```
records = [
    {"name": "林青", "status": "waitlist"},
    {"name": "周岚", "status": "accepted"},
]
has_waitlist = False

for record in records:
    print(record["name"], record["status"])

    if record["status"] == "waitlist":
        has_waitlist = True

if has_waitlist:
    print("汇总：有人候补")
else:
    print("汇总：无人候补")
```

最后显示 汇总：有人候补。has_waitlist 最初为 False，含义是“这次扫描到目前为止，还没有见到候补”。林青是候补，就把它设为 True；周岚不是候补，只跳过本轮内部的赋值，不会把已有的 True 改回去。

这类布尔标记（Boolean flag）保存的是扫描过程中积累的一件事实。**想记住整个过程，就把初值设在循环前；不能每轮都重新清空。** 当然，若某个值确实只描述当前这一轮，则可以每轮重新算，取决于它负责记住什么。

可以先判断一个改动：如果把 has_waitlist = False 移到 for 里面、放在每轮开头，最后会显示什么？

会错误地显示无人候补。第二轮一开始就抹掉了第一轮留下的 `True`，而周岚是 accepted，没有再把它设回 `True`。最早的断点不是比较没认出林青，而是下一轮初始化的位置覆盖了历史结果。恢复到循环外初始化，才保留整次扫描的事实。

并不是所有循环都要加一个标记。只逐条显示姓名时，不需要记住候补是否出现；多一个可变值，也多一处可能被错误初始化或覆盖的地方。

找到第一位候补就够了，怎样提前结束？

逐项报告需要读完整份名单，查第一位候补却可以找到就停。这时 `for` 还没耗尽，也可以使用 `break`：

PYTHON

```
records = [
    {"name": "林青", "status": "accepted"},
    {"name": "周岚", "status": "waitlist"},
    {"name": "陈禾", "status": "waitlist"},
]

for record in records:
    if record["status"] == "waitlist":
        print(record["name"])
        break

print("查找循环结束")
```

输出 `周岚`，然后仍显示 `查找循环结束`。陈禾虽还在列表中，这次任务已经不再扫描他。若没有人候补，`for` 会因为取不到下一项而自然结束，最后一行同样会执行；到达相同位置，不代表结束原因相同。

如果另一段代码需要接住姓名，则可以让函数直接返回结果：

PYTHON
```
def first_waitlisted(records):
    for record in records:
        if record["status"] == "waitlist":
            return record["name"]

    return "无人候补"


records = [
    {"name": "林青", "status": "accepted"},
    {"name": "周岚", "status": "waitlist"},
]
result = first_waitlisted(records)
print(result)
```

return 把 "周岚" 交给外面的 result，结束的是整个 first_waitlisted 调用，不会再到函数末尾返回无人候补。只有名单耗尽仍没找到时，才走到最后一行 return。空列表也会走到那里，因为 for 一轮都没进入。

这几种结束方式可以按实际去向区分：while 在条件不成立时离开，for 在没有下一项时离开；break 提前离开最内层正在运行的循环，继续循环后面的代码；return 则离开整个当前函数，把结果交回调用处。

每行还有几个座位，内层循环会重新开始吗？

假设要逐个打印两行座位标签：

PYTHON
```
seats = [
    ["A1", "A2", "A3"],
    ["B1", "B2", "B3"],
]

for row in seats:
    print("开始新的一行")

    for seat in row:
        print(seat)

print("座位处理结束")
```

外层第一轮把整行 ["A1", "A2", "A3"] 交给 row。进入内层后，依次把 A1、A2、A3 交给 seat；内层没下一项了，外层才去取第二行。

外层第二轮来到 `for seat in row` 时，这次要遍历的是 `["B1", "B2", "B3"]`，从 B1 重新开始，不是延续上一次已经耗尽的 A 行。最后外层也没下一行，才显示座位处理结束。

外层每次运行到内层循环，都会重新启动这次内层遍历。 如果在内层的 `print(seat)` 后面加一个同层缩进的 `break`，每行只打印第一个座位：A1 后离开内层，外层仍继续到 B 行，再打印 B1。不能把内层的 `break` 当成结束两层循环。

本例的数据本来有行与座位两层，所以适合嵌套。嵌套循环也可以用于两组项目逐项配对，不要求数据必须事先存成两层；关键是外层每项是否真的需要再完成一轮内层工作。只显示一份普通姓名列表时，就不需要第二层。

把录入与汇总接成一个小程序

现在可以在取得姓名后再取得状态，把它们组成一条字典，加入 `records` 列表。录入结束后再扫描这份列表，显示每个人，并汇总是否有人候补。

这份程序由工作人员按已知结果输入状态，只使用小写 `accepted` 或 `waitlist`；它不重新实现年龄、票种和名额判断，也不会把手工录入的文字当成已经验证过的业务事实。姓名允许重复，每次录入都保留为单独一条，不拿姓名当唯一键。

PYTHON

```
records = []

while True:
    name = input("姓名（输入 end 结束）：")
    if name == "end":
        break

    status = input("状态（accepted/waitlist）：")
    record = {"name": name, "status": status}
    records.append(record)

has_waitlist = False

for record in records:
    print(record["name"], record["status"])
    if record["status"] == "waitlist":
        has_waitlist = True

if has_waitlist:
    print("汇总：有人候补")
else:
    print("汇总：无人候补")

print(f"共 {len(records)} 条记录")
```

输入林青、waitlist、周岚、accepted、end，录入结束后的输出是：

> 林青 waitlist
> 周岚 accepted
> 汇总：有人候补
> 共 2 条记录

每轮的 `record = {...}` 重新组成一条记录，再由 `append` 放进同一份列表；`records` 在循环前只初始化一次，所以不会每轮丢掉前面录入的人。遇到 `end`，先 `break`，这一轮既不询问状态，也不会添加一条 end 记录。

录入与汇总不必交织在一起：前一段负责向列表添加，后一段读取收集完成的记录。列表承担了两段之间的交接。将来把录入来源换成文件，只要得到同样字段的记录，后面的扫描可以继续使用；但这份列表现在只存在于运行中的程序里，关闭后不会自动保存到文件。

在电脑上核对

把完整程序保存到自己独立练习目录中的 `roster.py`，使用 UTF-8 纯文本，不覆盖已有文件；从该目录运行：

```powershell
python .\roster.py
```

可以分别试上面的两条记录，以及一开始就输入 `end`。后者应显示无人候补、共0条记录，因为没有进入录入的添加部分，汇总的 `for` 也没有任何项目可取。再录入同名的两条正常记录，数量应为2，不应被姓名相同覆盖。

本例要求按提示输入状态。如果写成别的文字，程序仍会把它保存下来，却不会把它识别为 `waitlist`；这就是缺少输入校验的实际边界，不是字典或循环自动完成了校验。需要扩展时，应在追加前判断允许的状态，再决定重新询问还是拒绝这条记录。

后面的程序还会用到这些关系

现在可以沿一个具体人追完整过程：本轮 input 得到文字，字典把相关字段放在一起，append 把记录留进列表，for 逐条取回，条件更新需要跨轮保留的事实。出错时能进一步判断，是没有取得下一项、状态被重新清空、键或位置不对，还是提前结束的范围不对。

`while`、`for`、`append`、按位置或键取值，需要在实际任务中逐渐用顺。这里的姓名、编号、循环变量名不需要背；忘记具体方法时可以查，但要先能分清自己想继续读取、保存一条，还是取回已有内容。

这份名单程序只读文字。下一章会换到需要整数的输入任务：如果某一轮输入不能转换，程序为什么中断，又怎样处理失败并取得下一次输入？循环提供再次执行的能力，但不会自动解决循环体里的失败。

进一步查阅：CS50P Week 2 讲义介绍循环与容器；Python 循环与控制流教程说明取项、break 与循环关系；Python 数据结构教程说明列表和字典操作。

A05 | 出错以后，程序怎样继续？

活动开始前，需要根据预计到场人数准备饮用水：每人两瓶。输入 12 很顺利；但输入 cat 时，程序会在转换处停下。仅有循环，并不能保证出错后还能继续询问。

这一章要解决的不是“如何让错误信息消失”，而是**某一步做不成以后，哪些工作不能再做，程序应该去哪里继续。** 先看成功与失败在哪里分开，再让失败后的输入能够重试；最后把“取得整数”和“判断人数是否合规则”分给不同部分。

代码有没有真正开始运行

这一段先建立成功路线，再区分两个失败阶段。当前只看一次输入，不处理重试。

先让输入 12 完整走到结果

```python
people_text = input("预计到场人数：")
people = int(people_text)
bottles = people * 2
print("需要准备", bottles, "瓶水")
```

输入：

```
12
```

执行路线：

```
input() 返回文字 "12"
↓
people_text 保存 "12"
↓
int("12") 产生整数 12
↓
people 保存 12
↓
12 * 2 产生 24
↓
bottles 保存 24
↓
屏幕显示：需要准备 24 瓶水
```

这里已经有一条完整的数据流：外部文字进入程序，转换成为整数，普通计算产生新值，`print()` 把结果交给人。

少一个引号：普通程序还没有开始

把第一份程序换成只有这一行的独立文件：

```python
print("活动开始)
```

右边的结束引号缺失。Python 无法先理解这份源代码的结构，因此普通程序体没有开始执行，`print()` 也没有真正被调用。

这个阶段的问题叫 `SyntaxError`：syntax 是代码必须遵守的结构规则。修复方法是回到源代码补上缺失结构。

> Python 读取源代码
> ↓
> 字符串没有结束
> ↓
> 源代码结构无法解析
> ↓
> 普通程序没有开始

运行中的 `try/except` 不能替尚未开始的程序修好这种语法错误。

代码能开始，某项操作仍可能在运行中失败

恢复第一份正确代码，只把输入改成：

```
cat
```

Python 能理解源代码，`input()` 也成功返回了文字。程序一直运行到：

```python
people = int(people_text)
```

此时右边实际执行的是 `int("cat")`。失败发生在程序已经运行以后，所以它属于运行时失败，而不是 `SyntaxError`。

最短区别是：

> SyntaxError：源代码还不能开始执行
> 运行时失败：代码已经开始，某项具体操作做不成

下一段会放大 `int("cat")` 失败的瞬间：它不仅产生一条错误信息，还会阻止本次赋值和后续代码发生。

文字不能变成整数时，赋值发生了什么

等号右边先产生结果，左边才接得到

成功路线中：

```python
people = int("12")
```

会依次发生：

```
int("12") 产生整数 12
↓
people 接到 12
```

失败路线中：

```python
people = int("cat")
```

会变成：

```
int("cat") 无法产生整数
↓
等号右边没有结果可交付
↓
people 的这次赋值没有完成
↓
当前普通路线被中断
```

这里不是 `people = 0`，也不是 `people = "cat"`，更不是 `people = None`。左边没有收到任何替代值。

本例从新的运行开始，没有旧的 `people`。若某个名字之前已有值，失败的这次赋值也不会自动清除旧值；关键始终是“这次没有赋成”，不是“异常会删除变量”。

这种在程序运行中发出的失败信号叫 exception（异常：某项运行中操作无法按预期完成时发出的失败信号）。这一次的具体类型是 `ValueError`：`int()` 可以接收字符串，但字符串内容不能表示整数。

先为这一次 ValueError 安排清楚去向

现在已经能指出最早失败的操作，也知道失败时希望显示一条可理解的提示。先把转换和本次成功工作放进一个 try：

PYTHON
```
people_text = input("预计到场人数：")

try:
    people = int(people_text)
    bottles = people * 2
    print("需要准备", bottles, "瓶水")
except ValueError:
    print("请输入整数，例如 12。")

print("本次登记处理结束。")
```

输入 12：

```
进入 try
↓
int("12") 成功，people 保存 12
↓
bottles 保存 24
↓
显示饮用水数量
↓
没有 ValueError，跳过 except
↓
显示：本次登记处理结束
```

输入 `cat`：

> 进入 try
> ↓
> int("cat") 发出 ValueError
> ↓
> people 的赋值没有完成
> ↓
> try 中剩余计算与显示被跳过
> ↓
> 执行匹配的 except ValueError
> ↓
> 显示：请输入整数，例如 12
> ↓
> 继续执行整个结构下面的代码

`try` 表示“实际尝试这些操作”，`except ValueError` 表示“如果其中发出这一种已知失败，就从这里继续”。它们不会把 `cat` 修成整数，也不会自动再次运行 `input()`。

只写 `except:` 虽然能捕获更多失败，却可能把拼写错误或程序 bug 一起隐藏。当前只知道怎样解释和恢复 `int()` 的 `ValueError`，所以明确写出这一种类型。

为什么输入 `-2` 不进入 `except`

> int("-2")
> ↓
> 成功产生整数 -2
> ↓
> 没有 ValueError

但活动人数不能为负数。这是已经得到整数后的普通业务判断，应该交给 `if`：

```python
people = -2

if people < 0:
    print("人数不能是负数。")
```

`try/except` 与 `if` 的最近区别是：

- `try/except` 处理一项真实操作在运行中做不成；
- `if` 根据已经存在的普通值选择业务路线。

能转换不等于符合活动规则。异常处理不能取代业务判断。

输入 `cat` 后，`people` 是错误整数、`None`，还是本次赋值没有完成？程序接着去哪里？

正确路线是：

> int("cat") 没有产生整数
> → people 的本次赋值没有完成
> → try 中剩余代码被跳过
> → 执行 except ValueError
> → 再执行 try/except 结构后面的代码

- 如果认为 `people` 得到错误值或 `None`：回看“等号右边先产生结果”。
- 如果认为 `except` 会自动重新输入：回看“先为这一次 ValueError 安排去向”。当前代码只有一次 `input()`。

当前版本还有一个设计问题：`try` 包住了转换、计算和显示。下一段会把它缩小到真正可能发出 `ValueError` 的转换，并处理缩小后暴露出的新路线。

为什么缩小 `try` 后会暴露 `NameError`

`except ValueError` 的含义很具体。为了能准确知道它保护哪项工作，`try` 应尽量只圈住我们预期可能发出这种异常的操作。

只圈住转换以后，成功代码不能随便放

先用一个全新的独立脚本观察失败，不沿用其他文件或交互环境中的旧名字：

PYTHON
```
try:
    value = int("cat")
except ValueError:
    print("转换失败。")

print(value)
```

逐行追踪：

> 进入 try
> ↓
> int("cat") 发出 ValueError
> ↓
> value 的赋值没有完成
> ↓
> except 显示：转换失败
> ↓
> 离开 try/except
> ↓
> print(value) 仍然运行
> ↓
> 当前脚本中从未建立 value
> ↓
> Python 发出 NameError

`NameError` 表示当前代码试图读取一个并不存在的名字。这里的原因不是 `try` 形成了特殊作用域；真正原因仍是右边转换先失败，赋值从未完成。

给原来的 `try` 再增加一个 `except NameError`，也接不到块外 `print(value)` 发出的异常。程序真正需要的是：只有转换成功、`value` 确实存在时，才运行依赖它的成功代码。

`else` 接住“尝试成功”的路线

PYTHON

```
text = input("预计到场人数：")

try:
    people = int(text)
except ValueError:
    print("请输入整数，例如 12。")
else:
    bottles = people * 2
    print("需要准备", bottles, "瓶水")

print("本次登记处理结束。")
```

输入 `12`：

> int("12") 成功
> → people 保存 12
> → 跳过 except
> → 执行 else
> → bottles 保存 24
> → 显示结果
> → 执行结构后面的 print

输入 `cat`：

> int("cat") 发出 ValueError
> → people 没有建立
> → 执行 except
> → 跳过 else
> → 执行结构后面的 print

这里的 `else` 属于 `try/except` 结构：在 `try` 没有异常、正常执行到末尾时运行。它和 `if/else` 使用同一个英文词，但判断依据不同：

- `if/else` 看一个普通条件是 `True` 还是 `False`；
- `try/except/else` 看尝试的操作是否发出了异常。

把依赖成功结果的工作放进 `else`，就能从控制路线本身保证 `people` 已经存在。`else` 中若另有操作失败，不会再被前面这个 `except` 接住；它只负责对应 `try` 内的失败。

回到完整路线：现在有分叉，还没有重试

> input() 返回文字
> ↓
> try 中 int() 尝试转换
> ├── ValueError → except 显示提示 → 本次处理结束
> └── 成功 → else 使用整数 → 本次处理结束

`else` 解决了“成功代码何时安全运行”，却没有让输入再次发生。下一段要把这条单次路线放入循环。

为什么处理一次不会自动重新询问

`try/except` 只决定本次尝试失败后去哪里。要再次运行 `input()`，控制必须回到包含它的循环开头。

while 才负责产生下一轮

```python
while True:
    text = input("预计到场人数：")

    try:
        people = int(text)
    except ValueError:
        print("请输入整数，例如 12。")
    else:
        break

print("得到整数", people)
```

输入顺序为 cat、12 时：

```
while 检查 True，进入第 1 轮
→ input() 返回 "cat"
→ int("cat") 发出 ValueError
→ except 显示提示
→ 到达循环体末尾
→ while 再检查 True，进入第 2 轮
→ input() 返回 "12"
→ int("12") 产生 12
→ else 执行 break
→ 离开 while
→ 显示：得到整数 12
```

重新询问不是 except 的隐藏功能。**失败路线没有离开循环，到达循环体末尾后，while 才开始下一轮。**

pass 可以安静处理，但不会制造重试

如果不想每次都显示错误提示，可以把上面循环中的失败分支替换成下面两行。它是替换片段，不是可单独运行的完整程序：

```python
except ValueError:
    pass
```

`pass` 的作用是“这里不执行额外动作”。之后控制自然到达循环体末尾，再由 `while` 开始下一轮。

> ValueError
> → except
> → pass 不做额外动作
> → 循环体末尾
> → while 决定下一轮

因此，`pass` 本身既不重试，也不结束循环。它只是允许这个代码块合法存在并继续走自然路线。在面向人的输入中，清楚提示通常比静默更友好；这里只把安静版本当作一种取舍。

`break` 与 `return` 结束的范围不同

现在把“取得转换成功的整数”装进函数。第一版先用 `break` 离开循环，再用 `return` 把值交回调用者。这里先展示函数定义；定义本身不会询问，调用 `get_int()` 才会进入：

PYTHON
```
def get_int():
    while True:
        try:
            value = int(input("请输入整数："))
        except ValueError:
            print("请输入整数。")
        else:
            break

    return value
```

成功时：

> break
> → 只离开 while
> → 继续执行函数中循环后的 return value
> → return 结束 get_int
> → 把整数交回调用位置

因为成功路线最终就是离开整个函数并交回整数，也可以直接写：

```python
def get_int():
    while True:
        try:
            value = int(input("请输入整数："))
        except ValueError:
            print("请输入整数。")
        else:
            return value
```

`return value` **结束当前函数，并把值交给调用它的代码。**它不只是“更强的 break”，因为它还携带结果并结束函数范围。

四种去向可以这样找回：

- 自然到循环体末尾：回到 `while` 条件；
- `pass`：不做额外动作，再走自然路线；
- `break`：离开当前循环，执行循环后的代码；
- `return`：结束当前函数，把结果交给调用位置。

`cat` 进入 `except` 后，是什么让 `input()` 再次运行：`except`、`pass`，还是 `while`？下一执行位置依次在哪里？

> except 只接住本轮 ValueError
> → 提示或 pass 完成本分支
> → 控制到达循环体末尾
> → while 再检查条件
> → 下一轮才重新执行 input()

- 如果选择 `except`：回看“while 才负责产生下一轮”。
- 如果选择 `pass`：回看“pass 可以安静处理”。删掉 `pass` 而保留一条合法的提示语句，循环仍会重试。

现在已有一个能交回转换成功的整数的函数。最后一段会让调用者提供自己的提示文字，并把转换规则与活动规则分给两个不同函数。

输入函数怎样把转换成功的整数交给 `main`

当前目标不只是让程序运行，而是让每个部分只承担一项清楚工作：

> get_int：取得能转换的整数
> main：判断人数是否符合活动规则，并计算结果

调用者把提示文字作为参数交进去

下面这行将放在完整程序的 `main` 中；参数版 `get_int` 的定义紧接着给出。它不是要单独运行的一行：

```python
people = get_int("预计到场人数：")
```

这行执行时，右边的函数调用要先完成：

> main 传入文字 "预计到场人数："
> ↓
> get_int 的 prompt 参数接住这段文字
> ↓
> input(prompt) 把它显示给使用者
> ↓
> 函数取得转换成功的整数后 return
> ↓
> 回到原来的调用位置
> ↓
> people 才接到返回的整数

`prompt` 只是参数名，表示显示给使用者的询问文字。它让同一个输入函数可以被不同调用者复用，而不把问题文字写死在函数内部。

完整程序：转换规则和活动规则分开

这次活动只接受大于 0 的人数：整数 `0` 和负数都需要重新登记。先由输入函数取得整数，再由 `main` 检查这条规则。

PYTHON

```
def get_int(prompt):
    while True:
        try:
            value = int(input(prompt))
        except ValueError:
            print("请输入整数，例如 12。")
        else:
            return value


def main():
    while True:
        people = get_int("预计到场人数：")

        if people > 0:
            break

        print("人数必须大于 0。")

    bottles = people * 2
    print("需要准备", bottles, "瓶水")


main()
```

这里有两个循环，但它们没有重复承担同一责任：

- `get_int` 内的循环处理“文字能否转换”；
- `main` 内的循环处理“整数是否符合活动人数规则”。

如果以后另一个任务允许 `0` 或负数，`get_int` 不需要改变；改变的是调用者自己的业务规则。

从 cat → -2 → 12 追完整条路线

1. 程序调用 main()。
2. main 调用 get_int("预计到场人数："），并等待右边返回结果。
3. prompt 接到询问文字，input(prompt) 第一次返回 "cat"。
4. int("cat") 发出 ValueError；value 没有赋值。
5. except 显示“请输入整数”；到循环末尾后，get_int 的 while 开始下一轮。
6. 第二次输入 "-2"；int("-2") 成功产生整数 -2。
7. else 执行 return -2；get_int 结束，结果回到 main 中等待的调用位置。
8. people = get_int(...) 的右边完成，people 接到 -2。
9. main 检查 people > 0，结果为 False；显示“人数必须大于 0”。
10. 到达 main 的循环体末尾，外层 while 开始下一轮，再次调用 get_int。
11. 第三次输入 "12"；转换成功，get_int return 12。
12. main 的 people 接到 12；people > 0 为 True，break 离开业务循环。
13. bottles = people * 2 产生 24。
14. 屏幕显示“需要准备 24 瓶水”。

这条路线有两个失败边界：cat 在输入转换边界失败；-2 没有转换失败，而是在活动业务规则处被拒绝。

输入 12 后，return 回到哪一行？谁检查 12 > 0？如果输入 -2，是哪一个循环产生下一次询问？

- return 12 回到 people = get_int(...) 右边的调用位置；右边完成后，people 才得到 12。
- main 中的 if people > 0 检查活动业务规则，get_int 不负责这项判断。
- 输入 -2 时，get_int 已经结束；是 main 的外层业务循环再次调用 get_int。

如果答案不同，先按最早断点回看：

- 不知道 return 交给哪里：回看“调用者把提示文字作为参数交进去”中的调用路线；
- 认为 get_int 应该检查 12 > 0：回看“完整程序：转换规则和活动规则分开”；
- 认为输入 -2 后仍是 get_int 自己重问：回看cat → -2 → 12 完整路线的第 6 ~ 10 步。

如果把业务检查也塞进 get_int，这个函数就不再只是“取得整数”，而会被某一个活动规则绑住。

现在再给这些普通关系命名

先看已经发生的事实：

> 外部文字由 get_int 接收和转换
> → 转换失败在 get_int 内改道
> → 转换成功的整数通过 return 交给 main
> → main 检查业务规则并计算
> → print 把结果交给人

现在这些词才有具体含义：

- 每个部分“只负责什么”，叫职责；
- 调用时传入 `prompt`、返回时交出整数，是两个部分的接口；
- `people`、`bottles` 等当前值，是运行中的状态；
- `ValueError` 在输入函数内处理，说明失败也有处理边界。

这就是最初的系统认知：不是先背复杂架构词，而是能指出数据从哪里来、谁改变它、失败在哪里改道、可靠结果交给谁。

有电脑时用运行核对这些路线

把“完整程序：转换规则和活动规则分开”中的代码保存到自己的独立练习目录，使用 UTF-8 纯文本，例如新建 `registration.py`，不要覆盖已有文件。从该目录运行：

```powershell
python .\registration.py
```

分别核对：

1. 只输入 `12`：应显示需要 24 瓶水。
2. 输入 `cat`、再输入 `12`：应先提示整数格式，再显示 24 瓶水。
3. 输入 `cat`、`-2`、`12`：应分别经过转换失败、业务规则失败和成功路线。
4. 单独运行缺结束引号的文件：核对它是 `SyntaxError`，且普通程序未开始。
5. 单独运行读取未建立 `value` 的脚本：核对它在提示后出现 `NameError`。

运行只是验证材料中追踪的路线。真正需要保留的是：看见错误时能指出最早做不成的操作、未发生的赋值和下一执行位置。

哪些关系会继续用到？

这一章真正改变的是失败后的控制路线：`int()` 做不成时，本次赋值没有完成；匹配的 `except` 接住这次失败，`else` 使用成功结果，循环决定是否重试，`return` 把整数交回调用处。普通业务判断仍由拿到整数的 `main` 负责。

`try/except` 和循环、返回之间的常用组合，需要在运行、修改和排错中逐渐用顺，不必逐字背下整份程序。看到 `ValueError`，应能沿错误位置找到哪项操作没有得到值，而不是背整行报错。`get_int`、`prompt` 是这里选择的函数名和参数名；错误消息的具体英文会随 Python 版本变化，忘记细节可以查。

以后若更换人数规则，修改调用者就够了；若更换输入提示，调用时传入另一段文字即可。这样，职责和交接已经落在具体代码上，不需要额外背一套架构定义。A06 会继续追问：标准库、第三方库和自己的另一个文件，怎样把现成能力带进当前程序？

核心解释到这里已经完整。后面的官方练习是可选迁移；不读也不会缺少理解本章主线的前提。

可选迁移一 | Fuel Gauge 还多了哪些关系

官方 Fuel Gauge 要求读取 `X/Y` 形式的油量分数，把合法结果取最近的整数百分比；不超过 1% 显示 `E`，不低于 99% 显示 `F`。`X` 应为非负整数，`Y` 应为正整数，而且 `X <= Y`；不符合这些要求时需要重新询问。

这一题把异常关系用到分数输入。下面先恢复额外需要的操作，再给测试方向；各片段不组成完整提交答案。

`split()` 先产生多个字符串，再由变量接住

```python
parts = "1/2".split("/")
print(parts)
```

结果是一个包含两段文字的列表：

```
['1', '2']
```

接着沿用刚才的 `parts`，执行下面的续接片段：

```python
x_text, y_text = parts
```

表示左边两个名字依次接住右边两项：

> parts 是 ["1", "2"]
> → x_text 接到 "1"
> → y_text 接到 "2"

数量不匹配时，拆包本身也会发出 `ValueError`：

> "1/2/3".split("/")
> → ["1", "2", "3"]
> → 三项无法交给两个名字
> → ValueError

因此，Fuel 中的 `ValueError` 不一定只来自 `int()`。

三条失败路线发生在不同操作

> 输入 cat/4
> → split 成 ["cat", "4"]
> → int("cat") 做不成
> → ValueError

> 输入 3/0
> → 拆分和 int 都成功
> → 真正计算 3 / 0 时做不成
> → ZeroDivisionError

> 输入 5/4
> → 拆分、转换和除法都能完成
> → 但 X > Y 不符合题目规则
> → 普通 if 路线，不是异常

当两种已知异常需要进入同一恢复路线时，可以分别写两个 `except`，也可以写：

PYTHON
```
try:
    numerator = int("3")
    denominator = int("0")
    ratio = numerator / denominator
except (ValueError, ZeroDivisionError):
    print("本次转换或除法没有完成")
```

这个隔离示例会走到 ZeroDivisionError；把 "3" 改成 "cat"，则会先走到 ValueError。括号里的意思是：任意一种类型出现，都进入同一个处理块。这里不需要展开学习 tuple；只要知道它压缩了两个相同去向。

continue 只在本轮确实还有代码要跳过时出现

Fuel 的循环中，某轮输入如果已经确定无效，后面的百分比计算就不应继续。此时控制路线可以是：

> 本轮解析失败或普通范围无效
> → continue
> → 跳过本轮剩余代码
> → 回到 while 的下一轮
> → 再次 input

continue 不会离开循环；它结束当前这一轮。若无效分支之后本来就没有其他代码，自然到循环体末尾也能开始下一轮，因此不要为了“异常必须配 continue”而机械添加。

先用两份确定能拆分、转换且分母非零的文字，单独观察“范围无效以后跳过计算”：

PYTHON
```
for fraction in ["5/4", "3/4"]:
    x_text, y_text = fraction.split("/")
    x = int(x_text)
    y = int(y_text)

    if x > y:
        print("超过整箱，跳过本轮计算")
        continue

    percent = round(x / y * 100)
    print(percent)
```

第一轮显示超过整箱，然后 continue 跳过 percent 的计算和显示；for 取得下一份 "3/4"。第二轮得到 3 / 4 = 0.75，乘以100成为百分数75，round(...) 取最近的整数，最后显示 75。这是定位跳转的隔离片段，没有处理未知键盘输入、转换异常或 E/F 输出规则。

建议在自己的实现中逐条验证：3/4 → 75%、4/4 → F、0/4 → E，以及 4/0、three/four、1.5/3、-3/4、5/4 都会重新询问。精确要求以官方 Fuel Gauge 页面为准。

可选迁移二 | Taqueria 怎样处理字典、未知菜名与输入结束

官方 Felipe’s Taqueria 要求逐行接收菜名；每输入一个有效菜名，就显示到目前为止的总价。大小写不敏感，未知菜名被忽略，输入来源结束时停止。

这一题连接字典、累计状态和 `EOFError`。下面恢复必要关系，但不把它们拼成可提交的完整答案。

字典没有这个 key 时，Python 不会自动忽略

先用一个缩小菜单恢复查值：

```python
menu = {
    "Taco": 3.00,
    "Burrito": 7.50,
}
```

```
"Taco" in menu
→ True

menu["Taco"]
→ 3.00
```

原始输入可以先规范成菜单使用的大小写：

```python
item = "tAcO".title()
```

结果是 `"Taco"`。接着沿用上面的 `menu` 和 `item`，先检查成员关系，再决定是否查值：

```python
if item in menu:
    price = menu[item]
```

未知菜名的完整路线是：

```
输入 Burger
→ title() 得到 "Burger"
→ "Burger" in menu 为 False
→ 不进入缩进代码
→ 不执行 menu[item]
→ 总价保持不变
→ 循环自然进入下一轮
```

如果没有成员检查而直接执行 `menu["Burger"]`，Python 会发出 `KeyError`。它绝不会自动忽略缺少的 key。官方题允许“避免或捕获 KeyError”；当前支架选择先用 `if item in menu`，因为未知菜名是可直接判断的正常业务路线。

总价是跨轮保存并更新的状态

```python
total = 0.0
price = 3.00
total = total + price
```

本轮开始 total 是 0.0
→ 查到 Taco 的 price 是 3.00
→ 加法产生 3.00
→ total 更新为 3.00
→ 下一轮从 3.00 继续

再次输入 Taco 时，新一轮把 `3.00 + 3.00` 更新为 `6.00`。完整录入程序应把 `total = 0.0` 放在循环前，只初始化一次；每轮查到有效价格后才更新它。若每轮都设回零，就只剩这一单的金额。

需要显示美元符号和两位小数时，可以在刚才得到 `total` 后接上：

```python
print(f"Total: ${total:.2f}")
```

这里显示 `Total: $3.00`。`$` 是普通显示字符，花括号取出 `total`，`.2f` 把显示保留为两位小数；它没有替程序累计金额。异常处理也不负责替累计变量保存价格。

`EOFError` 表示输入来源结束，不是文字格式错误

`input()` 平时返回一行文字；当使用者明确结束输入来源时，它无法再返回下一行，于是发出 `EOFError`。

最小控制形状可以用一个独立的“逐行显示”程序观察：

```python
while True:
    try:
        raw = input("Item: ")
    except EOFError:
        print()
        break

    print("收到", raw)
```

```
还有一行输入
→ input() 返回字符串
→ print 显示“收到”以及这行文字
→ 循环继续询问

输入来源结束
→ input() 发出 EOFError
→ except 打印换行
→ break 离开输入循环
```

这个独立程序只显示收到的文字。回到菜名任务时，再把显示后的工作换成规范化、成员判断和有效价格累计。这里的 `try` 只圈住真正可能发出 `EOFError` 的 `input()`。`EOFError` 不表示菜名拼错；未知菜名仍由字典成员判断处理。

在 cs50.dev / Unix 环境中，官方说明使用 Ctrl-D 结束输入；在 Windows PowerShell 中，通常使用 Ctrl-Z 后按 Enter。实际按键属于环境差异，异常关系不变。

当前官方菜单与精确输出格式以Felipe's Taqueria 页面为准。测试时至少追踪有效菜名、大小写变化、未知菜名、重复菜名和 EOF 五条路线。

其他官方练习怎样选择

- Grocery List 主要巩固 `EOFError`、字典计数、按 key 排序和格式化输出；当这些关系还需要放进综合任务再练一次时再做。
- Outdated 同时协调两种日期输入、拆分、月份查找、范围检查、重试和规范输出；适合需要更强综合迁移时使用。

如果目标包含课程官方完成状态，应按 Problem Set 3完成四题，并遵守当前 Academic Honesty要求。这里的讲解与运行不能替代自己的提交。

来源与进一步查阅

课程主线参照 CS50P Week 3 讲义，运行语义可查 Python 错误与异常教程。本文的活动主例和中文解释为自学场景编写；练习的完整规格以各自官方页面为准。

所参考的 CS50P 材料采用 CC BY-NC-SA 4.0；本文保留署名、非商业与相同方式共享条件，不表示 Harvard 或 CS50 背书。官方题目、菜单、提交方式和政策可能更新，实际提交前再核对。这里链接的是实时官方入口，不是固定版本快照。

A06 | 别处的代码，怎样为当前程序工作?

支持人员想从几条工单中抽一条检查，也想知道几次处理平均花了多久。这些计算不一定都要自己从头写：Python 已经提供了一些现成能力，项目里的另一个文件也可以提供可复用的函数。

这一章先让一个现成函数接收数据、交回结果，再看名字从哪里来、命令怎样传入数据，以及自己的文件怎样被复用。最后再看需要另外安装的库。重点是**把“能力来自哪里”和“调用时发生什么”连接起来**，不是背下一份库名称大全。

从三条工单中抽一条，要自己写随机算法吗?

这里的编号只是独立示例：`T-104`、`T-105`、`T-106`。任务允许抽到其中任意一条，使用 `random` 模块里的 `choice` 就能完成：

PYTHON
```
import random

ticket_ids = ["T-104", "T-105", "T-106"]
sample_id = random.choice(ticket_ids)
print(sample_id)
```

一次可能显示 `T-105`，另一次可能显示 `T-104`；也允许连续显示同一个编号。不能把“随机”理解成“每次必须不同”。

`random` 是随 Python 提供的模块。模块（module）把相关名字组织在一起，这里其中一个名字就是函数 `choice`。先执行 `import random`，当前文件才有 `random` 这个名字可用；随后 `random.choice` 从这个模块取得函数，括号把名单交给它。

```
ticket_ids 保存三条编号
→ random.choice(ticket_ids) 接到这份非空名单
→ 返回其中一个编号，例如 "T-105"
→ sample_id 接到这个字符串
→ print 显示它
```

这仍是熟悉的函数交接：调用先得到结果，左边的赋值才完成。不同的是函数由标准库提供，当前文件没有自己写它的 `def`。

`choice` 不会把抽中的编号从原列表中删除，也不会自动查询真实工单。这里的任务只是从已有内容里选一个。名单为空时没有可选项目，调用会发生 `IndexError`，不是返回一个空编号；后面接收命令行输入时，会先检查有没有工单。

换一种 import，调用名字为什么也变了？

下面是可以独立运行的另一种写法：

```python
from random import choice

ticket_ids = ["T-104", "T-105", "T-106"]
sample_id = choice(ticket_ids)
print(sample_id)
```

两份程序选取的规则相同，但当前文件得到的名字不同：

写法	当前文件得到的名字	调用方式
`import random`	`random`，对应模块	`random.choice(...)`
`from random import choice`	`choice`，对应那个函数	`choice(...)`

第二种不是“跳过模块，只复制一小段函数文字”。Python 仍需要加载来源模块，再把指定名字放进当前文件。只写第二种 import 时，不会顺便在当前文件中建立 `random` 这个名字。

来源值得保留、名字容易冲突时，`random.choice` 更容易读。只用少量明确名字时，直接写 `choice` 也合理。不要因为短就使用 `from random import *`：它会让一批名字进入当前文件，后来很难看清哪个名字来自哪里。

计算平均时长，交给现成函数的是什么？

三次处理分别用了 12、18、9 分钟。下面用另一个标准库模块处理这组普通数字：

```python
import statistics

durations = [12, 18, 9]
average = statistics.mean(durations)
print(average)
```

输出 `13`。`mean` 计算平均值，这里相当于 `(12 + 18 + 9) / 3`；它把结果交回来，没有替程序决定结果显示在哪里。将来需要把平均值写到文件，仍由调用者使用这个结果。

同一组数字的平均值应该相同；同一份工单名单的随机抽查项则允许变化。**选用现成函数之前，要先知道任务要的是确定计算，还是允许变化的选择。** 不要用随机选择去生成要求唯一的工单编号；也不要因为两次抽到同一项就判断函数失效。

空数据没有这里所说的平均值。`statistics.mean([])` 会发出 `StatisticsError`，而不是自动返回零。标准库负责平均值的计算，不会代替业务决定“没有记录时是否显示暂无数据”。

`random`、`statistics`、稍后使用的 `sys` 都属于标准库，正常 Python 安装已经提供。现在不需要为它们运行安装命令。

不改源代码，怎样换一批工单？

把名单写在文件里，每次换工单都要编辑代码。也可以在启动命令后面给出这次的数据。先不抽查，只观察它们怎样进入 Python。

在自己的一个新练习目录中，将下面的代码以 UTF-8 纯文本保存为 `show_args.py`；不要覆盖已有文件：

```python
import sys

print(sys.argv)
```

从该目录运行：

```powershell
python .\show_args.py T-104 T-105
```

这次 Python 启动以后，`sys.argv` 是一个字符串列表。可以按下面的形状理解显示结果：

```
['.\\show_args.py', 'T-104', 'T-105']
```

列表显示里的 `\\` 表示字符串中的一个反斜杠，不是文件名多了一个斜杠。脚本项的具体拼写可能随启动方式变化，关键是位置：第0项标识脚本，后面才是本次给脚本的参数；启动解释器的 `python` 不在这个列表里。

`sys.argv` 不是函数调用，后面没有 `()`。`argv` 是 `sys` 模块提供的列表，程序读取它，就能取得启动时交进来的文字。它不会像 `input()` 一样再次停下来等键盘输入。

只想要工单，不要脚本名，怎样取？

下面用同样形状的列表观察切片（slice）：

```
PYTHON
arguments = ["show_args.py", "T-104", "T-105"]
ticket_ids = arguments[1:]

print(ticket_ids)
print(arguments)
```

输出：

```
['T-104', 'T-105']
['show_args.py', 'T-104', 'T-105']
```

`[1:]` 的含义是从位置1开始，一直取到末尾。它取得一个新列表，没有从原列表删除脚本名。实际程序就把上面的 `arguments[1:]` 换成 `sys.argv[1:]`。

若命令是 `python .\show_args.py 12`，参数也是字符串 `"12"`，不是整数 `12`。只有代码显式执行 `int(...)`，才发生数字转换。命令行只是另一种输入来源，不会替程序理解每个参数的业务含义。

没给工单时，怎样在抽查之前停止？

先把“有没有参数”和“抽哪条”放进一个小程序，保存为新文件 `pick_ticket.py`：

```
PYTHON
import random
import sys

ticket_ids = sys.argv[1:]

if len(ticket_ids) == 0:
    sys.exit("请在命令后提供至少一个工单编号。")

sample_id = random.choice(ticket_ids)
print(sample_id)
```

运行：

```
POWERSHELL
python .\pick_ticket.py T-104 T-105
```

得到其中一个编号。若只运行 `python .\pick_ticket.py`，`ticket_ids` 是空列表，程序先显示用法说明并结束，不会到更后面的 `choice` 再偶然报索引错误。

`sys.exit("说明")` 在这里发出 `SystemExit`；没有被外层捕获时，Python 用这段说明作为错误输出，并以非零退出状态结束。终端通常也会显示这段文字，但它走的是错误输出通道，和普通 `print` 的输出在程序层面可以区分。退出状态和输出通道在 A10 再完整展开。

这与 `return` 不同：`return` 结束当前函数，把结果交给调用者；`sys.exit` 表达的是本次命令应当结束，不是在返回一份正常工单结果。它也不是不经过 Python 的强制杀进程命令。这里不捕获 `SystemExit`，所以后面的抽查不会继续。

自己写的显示规则，也能放在另一个文件里吗？

假设几个工具都想把一个编号显示成 `本轮人工抽查：T-104`。可以把这项显示规则写一次，再让各个调用者使用它，不必复制函数正文。

在同一个练习目录保存 `ticket_tools.py`：

```python
def format_sample(ticket_id):
    return f"本轮人工抽查：{ticket_id}"
```

再保存 `use_tools.py`：

```python
import ticket_tools

message = ticket_tools.format_sample("T-104")
print(message)
```

运行 `python .\use_tools.py`，输出 `本轮人工抽查：T-104`。

use_tools 导入 ticket_tools
→ 调用 ticket_tools.format_sample("T-104")
→ ticket_tools.py 中的 ticket_id 接到 "T-104"
→ 函数返回说明文字
→ use_tools.py 中的 message 接住结果
→ print 显示结果

`import ticket_tools` 不写 `.py`。在这种直接运行脚本的普通方式下，Python 会把脚本所在目录作为模块查找位置之一，因此这两个文件放在同一目录就能互相找到。本章只用这种简单布局，不要求配置额外查找路径。

如果找不到这个模块，通常会在 import 处出现 `ModuleNotFoundError`，还没进入后面的调用。先核对文件名和放置位置，不要先改函数内部的 f-string。自己的文件也不要命名为 `random.py`、`statistics.py` 或 `httpx.py`，以免挡住原本想导入的同名库。

为什么只导入一个文件，也可能出现额外输出？

导入并非仅登记一个文件名。通常第一次导入时，Python 会执行模块顶层代码，建立其中的名字；`def` 会建立函数，但不会因此执行函数体。

用两个新文件观察，保留上面的文件不动。`import_demo.py`：

```python
print("模块顶层正在运行")

def describe():
    print("函数体正在运行")
    return "一条说明"
```

`observe_import.py`：

```python
import import_demo

print("导入已经结束")
message = import_demo.describe()
print(message)
```

输出顺序是：

```
模块顶层正在运行
导入已经结束
函数体正在运行
一条说明
```

顶层 `print` 在导入期间执行，函数内的 `print` 要到显式调用 `describe()` 才执行。假如把 `input()` 放在模块顶层，导入者也会被它停下来询问。

同一 Python 进程里，普通的重复导入通常复用已经加载的模块，不会每次重新执行全部顶层代码。这不是永久保存：退出后重新启动程序，又是一次新的运行。

怎样让演示只在直接运行这个文件时发生？

另存一份 guarded_tools.py：

```python
def format_sample(ticket_id):
    return f"本轮人工抽查：{ticket_id}"


def main():
    print(format_sample("T-104"))


if __name__ == "__main__":
    main()
```

__name__ 是 Python 为当前模块设置的名字。在本例中，直接运行 guarded_tools.py 时，它的值是 "__main__"；通过 import guarded_tools 导入时，则是 "guarded_tools"。

因此，直接运行会调用 main() 显示演示；导入时条件为假，跳过这一行调用。函数仍然定义好了，导入者可以随后使用 guarded_tools.format_sample(...)。

if __name__ == "__main__": 常叫入口保护（main guard）。**它只控制缩进在条件里面的启动动作，不会让整个模块在导入时停止执行。** 如果在条件外另写一条顶层 print，导入时仍会显示它。

这里可以判断一个改动：把最后两行换成没有条件的 main()，导入时会不会出现演示输出？

会。Python 为加载模块而执行到顶层 main()，就立即进入函数。问题不是 import 会自动调用每个函数，而是文件顶层明确写了这次调用；入口保护原本正是把它限制在直接运行的路线。

让命令、标准库和自己的模块协作

下面把已经分别看清的关系接起来。在保存了最初那份 `ticket_tools.py` 的目录，新建 `app.py`：

PYTHON
```
import random
import sys
import ticket_tools


def main():
    ticket_ids = sys.argv[1:]
    if len(ticket_ids) == 0:
        sys.exit("请在命令后提供至少一个工单编号。")

    sample_id = random.choice(ticket_ids)
    message = ticket_tools.format_sample(sample_id)
    print(message)


if __name__ == "__main__":
    main()
```

运行：

POWERSHELL
```
python .\app.py T-104 T-105 T-106
```

如果本次抽到 `T-105`，完整路线就是：

```
命令提供三项文字
→ sys.argv[1:] 得到三条编号的列表
→ main 检查列表不为空
→ random.choice 返回 "T-105"
→ ticket_tools.format_sample 接到这个值，返回说明
→ main 中的 message 接到说明
→ print 显示：本轮人工抽查：T-105
```

三个部分没有抢同一项工作：命令入口取得输入并检查数量，随机函数负责选择，自己的模块负责说明格式。改变显示文字时，不必同时重写随机选择。

这份小程序只完成按输入名单抽查，不验证工单是否存在，也没有合并前面的平均值计算。编号按给出的文字使用；同一个编号重复出现就占多个列表位置，并不会自动去重。把编号和时长成对解析、清洗格式、综合验证，属于后续整合问题，不需要先把那套解析器写完才能理解 import。

上机时可以核对有参数、无参数和只给一条工单三种情况。只给 `T-104` 就必须得到它；给多条时，检查结果属于这次名单，不要求几次运行一定出现不同编号。导入 `app` 时也不应自动抽查，因为入口调用已经受保护。

HTTPX 为什么还需要安装，而 random 不需要？

前面的模块要么随 Python 提供，要么是当前项目自己的文件。HTTPX 是另外分发的第三方库，用于向服务发请求、取得回应；Python 不保证已经安装它。

安装和导入处于两个不同时间：安装工具把库及依赖放进某个 Python 环境可使用的位置；之后程序执行 `import httpx`，才在当前运行中加载它。`import` **不会代替安装，另一个 Python 安装成功，也不证明当前 Python 能找到它。**

package 在安装语境里指一份可分发的软件；导入语境里也用这个词表示组织模块的包。这里 HTTPX 的安装名和导入名恰好都是 `httpx`，但其他库未必如此，应分别看官方说明，不能把两个名称永远当成一个。

第一次理解这条关系，不必先运行服务器。先看一个已经安装 HTTPX、且本地服务正在回应时的调用：

```python
import httpx

response = httpx.get(
    "http://127.0.0.1:8765/a06_endpoint.txt",
    trust_env=False,
)
print(response.status_code)
print(response.text.strip())
```

`127.0.0.1` 指本机，后面的路径指定这次要取的内容；`8765` 是本例约定的服务端口。本例的 `trust_env=False` 让这次本地调用不采用环境中的代理配置，不让电脑原有代理干扰观察。完整 HTTP 协议与代理含义留到 B 册。

假设服务为这条路径提供文字 `A06 local reply`，成功时会显示：

```
200
A06 local reply
```

这里的 `response` 不直接是这段文字，而是 HTTPX 返回的一个 Response 对象，里面保存了这次回应的信息。`response.status_code` 读取状态码，`response.text` 读取文字；它们是取属性，不是再发请求。请求在前面的 `get(...)` 调用处已经发生，最后的 `strip()` 只清理文本两端空白。

> 当前 Python 能 import httpx
> → get 向本机服务发请求
> → 服务交回回应
> → get 返回 Response，response 接住它
> → 当前代码读取信息并显示

“导入成功”只说明库可以加载，不能证明服务在线。服务没有启动时，调用可能发出连接异常，`response` 的这次赋值不会完成；服务返回404等 HTTP 回应时，则仍可能得到 Response，不能仅凭“有对象返回”就认定内容正确。这里用 `200` 和预期文字核对成功；状态、正文、JSON 及失败分类会在 B 册展开。

需要亲自运行这次本地调用时

以下只是上机准备，前面的调用关系不依赖先记住这些命令。不要在已有项目里试着覆盖环境；另建一个自己的空练习目录，在里面完成本节。

先创建专属于该目录的 `.venv`：

```powershell
python -m venv .venv
```

`-m venv` 表示让这个 Python 运行名叫 `venv` 的模块，创建一个隔离环境。创建成功后，环境中的 Python 位于相对路径 `.\.venv\Scripts\python.exe`。下面每条命令都明确使用它，不要求激活环境：

```powershell
& .\.venv\Scripts\python.exe -m pip install httpx
& .\.venv\Scripts\python.exe -m pip show httpx
```

PowerShell 的 `&` 在这里启动后面的可执行文件；`-m pip` 则让指定 Python 运行自己的安装工具。第一行需要可用的软件源或本地包来源；如果创建或安装失败，先处理该错误，不继续假定环境可用。不要为了让 import 成功就向不确定的其他 Python 反复安装。

第二行会显示版本（Version）和位置（Location）。核对使用的解释器，也可以运行：

```powershell
& .\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
```

`-c` 让该 Python 执行引号里的短代码；`sys.executable` 给出实际解释器路径。`pip show` 的位置应与所用环境相符。怎样切换编辑器解释器、比较两个环境，留到 A09。

把前面的 HTTPX 代码保存为 `local_client.py`。再在同一练习目录下新建一个名为 `serve` 的空子目录，只在其中新建 `a06_endpoint.txt`，以纯文本写入一行：

> A06 local reply

不要在 `serve` 放密码、私人文件或整个项目。第一个 PowerShell 窗口从练习目录运行：

```powershell
& .\.venv\Scripts\python.exe -m http.server 8765 --bind 127.0.0.1 --directory
  .\serve
```

这条命令用 Python 自带的小服务器，只向本机提供 `serve` 中的内容。它会占用这个前台窗口。必须看到服务已启动且仍在运行，才继续；如果端口已占用、命令报错后回到提示符，就停止本次步骤，不把原有服务当成本次服务，也不要结束来历不明的进程。

在第二个 PowerShell 窗口进入同一个练习目录，再运行：

```powershell
& .\.venv\Scripts\python.exe .\local_client.py
```

核对两行输出确实为 `200` 和 `A06 local reply`。完成后，回到自己启动服务器的第一个窗口，按 Ctrl+C 结束它。这只是学习用本地服务，不用它交付真实业务；前台程序、端口归属与停止后的证据在 A10 再系统学习。

网上的示例与本机写法不一样，先相信哪一个？

先看正在运行的解释器中安装了什么版本，再查看对应库的官方说明。比如 `pip show httpx` 会给出实际版本；去 HTTPX 的安装说明和 QuickStart 核对 `get` 的调用方式、参数及 Response 的取值方式，然后用这一个本地调用检查实际返回。

如果示例要求某个本机不支持的参数，应查该版本说明或变更记录，不要仅凭报错就升级整个项目。当前网页可能面向较新的版本，安装成功也不是所有示例都适配的证据。

需要记住的不是某一天的版本号，而是出现差异时从“哪个 Python、哪个已安装版本、哪份官方用法、什么实际结果”找原因。完整安装选项和参数拼写可以查；能解释安装与运行的关系，比记住某次成功命令更重要。

现在读一个程序，可以多看清什么？

看到调用时，可以同时追到来源和去向：这个名字来自标准库、项目文件还是第三方包；调用接到什么值，返回值交给哪一行；导入失败、参数不足和调用失败又发生在不同位置。

`import` 的常用写法、按模块名取函数、`sys.argv[1:]` 和入口保护，会在后面的项目中反复出现，应在使用中逐渐熟悉。这里的文件名和工单编号不要求背；也不需要把整个标准库记住。忘记函数名称时可以查，但要能先说清自己需要“选一项”“算平均值”“取得输入”还是“统一显示”。

下一章会问：一次运行得到预期结果之后，怎样用可重复的测试知道后来的修改没有破坏它？这时，让模块可以被导入却不擅自启动交互，就会成为实际需要。

进一步查阅：CS50P Week 4 讲义提供课程主线；Python 的模块教程、sys、random与statistics说明相应行为。上机准备可查 venv和本地 HTTP 服务器；第三方用法查 HTTPX 安装说明、QuickStart与环境配置说明。这些入口会更新，实际操作时应与本机版本一起核对。

A07 | 怎样知道这次修改真的修好了？

程序运行后显示了一个数字，不代表这个数字就是对的；今天手工试对一次，也不能保证下次修改没有破坏它。

这一章用一个时间换算函数，把“应该得到什么”变成能够重复执行的检查。先看一个为什么会漏掉的错误，再让测试暴露差异，沿值找到原因，最后用原来的检查核对修复。重点是**让需求、实际结果和修改证据连在一起**，不是先学一套测试工具命令。

> **第一次先观察，不先装工具：**先比较函数收到的输入、实际返回值和按需求推导出的预期值。只有当这三者的关系清楚后，才需要 pytest 帮你重复执行；安装命令和报告选项属于查用部分。

九点算对了，九点半就一定算对了吗？

预约程序需要把小时和分钟换成从当天零点开始经过的分钟数，后续才能比较两个时刻。比如：

> 9 点 00 分 → 9 × 60 + 0 → 540 分钟
> 9 点 30 分 → 9 × 60 + 30 → 570 分钟

这里先只处理两个整数：小时在 0–23 之间，分钟在 0–59 之间。读取键盘文字、日期和时区不属于这个函数当前的工作。

下面是故意有错的完整小程序：

PYTHON
```
def to_minutes(hours, minutes):
    return hours * 60


print(to_minutes(9, 0))
print(to_minutes(9, 30))
```

实际输出：

```
540
540
```

第一行对，第二行错。函数忘记加上 `minutes`，但只试整点时，遗漏的部分恰好为零，错误没有显现。

因此，预期的 570 要从时间换算规则推出，不能抄程序当前显示的 540。测试首先需要一个独立于当前实现的判断依据；如果连应该得到什么都不清楚，工具也不能替我们决定。

怎样让 Python 自己检查这个差异？

可以先取得实际结果，再写出它应该满足的条件：

```python
def to_minutes(hours, minutes):
    return hours * 60


actual = to_minutes(9, 30)
assert actual == 570
print("这条检查通过")
```

assert 是 Python 自带的断言语句。在这种普通运行方式下，它检查后面的条件；条件为真就继续，为假就发出 AssertionError。

to_minutes(9, 30) 返回 540
→ actual 接到 540
→ actual == 570 得到 False
→ assert 发出 AssertionError
→ 没有捕获它，程序停止
→ 后面的“这条检查通过”没有显示

assert 没有修复函数，也没有把 actual 变成 570。它让“不符合预期”成为明确失败，而不只是一个可能被人看漏的数字。

后面常见的 assert to_minutes(9, 30) == 570 只是把调用放进比较式，判断过程相同。要能看懂并写出这条“实际值与预期值比较”的关系，不必记住本例的数字。

不要用断言代替正式输入校验。Python 的优化运行选项可能移除断言；来自用户的数据该不该接受，仍应由普通条件判断和明确错误处理负责。本章也不使用优化选项运行检查。

> **第一次学到这里先停。**你现在应能说清三件事：570 从需求哪里来；函数实际交回了什么；assert 怎样把差异变成失败。还不能解释这三件事时，先回到这段值流，不要继续增加 pytest 命令。

检查多起来，谁负责一个个调用？

把每次检查都手工跑一遍，很容易遗漏。pytest 是测试运行工具：找到符合约定的测试，调用它们，再汇报哪些通过、哪些失败。断言负责提出要求，pytest 负责组织这次检查。

在自己的一个新空练习目录中，以 UTF-8 纯文本保存下面两个文件，不覆盖已有项目。`time_slots.py` 暂时仍保留错误：

PYTHON
```
def to_minutes(hours, minutes):
    return hours * 60
```

同目录的 `test_time_slots.py` 写成：

PYTHON
```
from time_slots import to_minutes


def test_whole_hour():
    assert to_minutes(9, 0) == 540


def test_half_hour():
    assert to_minutes(9, 30) == 570
```

第一行把另一个文件里的函数名字带进来。每个 `test_` 函数站在调用者的位置：给出输入，再核对函数返回值。

这两个文件职责不同。`time_slots.py` 提供换算能力，业务程序可以使用它；测试文件保存对这种能力的要求。这里的模块只有函数定义，没有顶层输入或运行调用，所以导入不会突然询问用户，也不需要为了测试额外写一个 `main()`。以后文件里若有交互入口，再用 `if __name__ == "__main__":` 限制启动动作。

需要实际运行时，从刚才的新目录创建隔离环境，再用其中同一个 Python 安装和运行 pytest：

POWERSHELL
```
python -m venv .venv
& .\.venv\Scripts\python.exe -m pip install pytest
& .\.venv\Scripts\python.exe -m pytest test_time_slots.py -q
```

`.venv` 是这个练习自己的环境；PowerShell 的 `&` 启动后面的 Python，`-m pip` 和 `-m pytest` 分别让它运行安装工具和测试工具，`-q` 减少报告细节。不需要激活环境。创建或安装失败时先处理该步，不继续假定测试已经运行；完整环境排错留到 A09。

在这里没有额外配置的情况下，pytest 按 `test_*.py` 或 `*_test.py` 等默认文件规则寻找测试，再收集其中 `test_` 开头的函数。本次命令明确指定了一个测试文件。pytest 调用两个函数，结果应为一条通过、一条失败。

普通执行 `python test_time_slots.py` 并不等价：这个文件只导入并定义函数，没有写调用；它安静结束，也不能证明两条检查已经执行。

报告指向 assert，是测试文件写错了吗？

失败报告里与当前问题有关的部分可摘成：

```
test_half_hour
>       assert to_minutes(9, 30) == 570
E       assert 540 == 570

1 failed, 1 passed
```

不同版本可能显示更多路径、行号和说明，关键差异仍是实际 `540`、预期 `570`。pytest 会增强测试断言的报告，普通 Python 的断言不一定显示同样多的比较细节。

这里的 `assert` 是发现差异的位置。被测函数已经正常返回 `540`，没有在函数内部抛异常；随后比较才失败。因此这份断言失败的回溯不必包含已经返回的函数内部步骤，不能指望它自动指出遗漏了哪次加法。

沿测试调用打开 `to_minutes`，才能继续追原因：小时 9 和分钟 30 都交进来了，`return hours * 60` 却只用了小时。

先核对预期是否来自需求，再追实际值在哪里偏离；不能为了让测试变绿，就把 570 改成当前错误的 540。 需求确实改变或原测试算错时，预期当然也需要修订，但应说明依据。

不猜原因，怎样看到计算前的值？

这个函数很短，直接读公式就能找到遗漏。遇到长一些的程序，如果不确定错误来自传值还是计算，可以先保留一个最小的重复入口：

POWERSHELL
```
& .\.venv\Scripts\python.exe -c "from time_slots import to_minutes;
  print(to_minutes(9, 30))"
```

它直接调用同一函数，仍得到 `540`。原本的菜单、键盘交互或整份预约名单都不参与这一小段换算，因此可以先不带它们。缩小问题的条件是错误还在；如果缩完不再失败，被拿掉的部分就可能与原因有关，不能直接丢弃。

还需要看中间值时，可以在这个练习副本中暂时把 `time_slots.py` 换为：

PYTHON
```
def to_minutes(hours, minutes):
    print("计算前：", hours, minutes)
    return hours * 60
```

再次运行上面的最小命令，会看到：

```
计算前： 9 30
540
```

现在有实际证据表明参数是 9 和 30，不需要先改命令或输入。遗漏发生在计算行。临时打印的作用是定位，不是永久测试；定位后移除它，避免给调用者增加无关输出。

值太多时，怎样暂停下来逐步看？

不想插入很多打印，也可以使用调试器。在编辑器中给将要执行的计算行设置断点，运行到那里会暂停；暂停在该行之前，当前行尚未执行，可以先观察 `hours` 和 `minutes`。单步后再看结果或下一位置。

需要不用编辑器按钮也能试时，Python 自带的 pdb 可以做这件事。另存独立的 `debug_slot.py`：

PYTHON
```
def to_minutes(hours, minutes):
    breakpoint()
    result = hours * 60
    return result


print(to_minutes(9, 30))
```

`breakpoint()` 是本次临时调试入口，不是业务计算的一部分。在未自定义调试钩子的普通 Python 环境中，运行它会进入 pdb：

POWERSHELL
```
& .\.venv\Scripts\python.exe .\debug_slot.py
```

看到 `(Pdb)` 提示符后，先看 `->` 指向哪一行。不同 Python 版本的首次显示可能不同：若仍指着 `breakpoint()`，先输入一次 `n`；等箭头指向 `result = hours * 60` 时，表示计算行还没有执行，再开始下面的观察。不要只按固定次数输入单步命令。

下面这些内容输入给调试器，不是 PowerShell：

```
p hours, minutes
n
p result
c
```

第一行显示 `(9, 30)`；`p` 用于查看表达式。`n` 执行当前行并停到同一函数的下一行：这里把 `540` 赋给 `result`，即将执行 `return`。所以再输入 `p result` 会得到 `540`；`c` 继续运行，直到下一个断点或结束，本例最后显示 `540`。

`n` 通常不会逐行进入当前行调用的另一个函数；需要进入被调用函数时才用 `s`。这类命令细节可以随用随查，重要的是知道当前停在哪、那一行是否已经执行、要观察哪个值。观察表达式也可能有副作用；这里仅查看变量，不在暂停时调用会写文件或改变业务状态的函数。

这一小段是条件性查用：直接读失败差异已经能定位时，不必每次都启动调试器。

修改以后，为什么还要重跑原来的测试？

在 `time_slots.py` 中，删除临时打印，改正公式，完整内容为：

```PYTHON
def to_minutes(hours, minutes):
    return hours * 60 + minutes
```

最小调用现在会得到 `570`。然后再次执行同一条 `python -m pytest` 命令：

```POWERSHELL
& .\.venv\Scripts\python.exe -m pytest test_time_slots.py -q
```

两条测试都应通过。这个过程不仅确认半点得到了修复，也重新检查整点这条原来正确的行为没有被弄坏。以后再次修改实现，保留下来的测试仍能重复核对，这就是回归检查的实际用途。

可以暂时想一个错误“修复”：直接让函数 `return 570`。半点测试会通过，整点测试会失败，因为整点仍应为 `540`。若只保留刚刚失败的半点案例，就会漏掉这种新错误。

因此，不需要为了凑数检查很多相似值；要挑能揭露不同遗漏的情况。对于全天换算，零点 `(0, 0)` 和当天最后一分钟 `(23, 59)` 也是有意义的范围边界，预期分别是 0 和 1439。九点本身只是普通时刻，不因为出现在示例里就成为函数边界。

函数显示对了，测试为什么还会失败？

下面这个独立对照说明 `return` 与 `print` 的区别：

```
PYTHON
def to_minutes(hours, minutes):
    print(hours * 60 + minutes)


actual = to_minutes(9, 30)
print(actual)
```

它显示：

```
570
None
```

函数内部把 `570` 显示到了输出通道，但走到函数末尾，没有显式返回业务结果，因此调用者得到 `None`。如果随后断言 `actual == 570`，仍会失败。

换算函数返回分钟数，调用者才可以显示、排序或比较它；测试也能直接拿这个值作比较。不是打印无法测试，而是需要另外捕获输出。只有显示行为本身是要检查的对象时，才需要做那件事；不能用“屏幕看见 570”代替“调用者收到整数 570”。

若程序直接报异常，回溯又应该怎么看？

前面是算完才发现不对。现在换一种情况：仍使用修正公式后的 `time_slots.py`，另存 `wrong_call.py`：

```
PYTHON
from time_slots import to_minutes


print(to_minutes(9, "30"))
```

这次第二个参数是文字 `"30"`，不是整数。尚未加入后面范围检查的这个函数会在加法处失败。省略个人路径后的回溯形状是：

```
Traceback (most recent call last):
  File "wrong_call.py", line 4, in <module>
    print(to_minutes(9, "30"))
  File "time_slots.py", line 2, in to_minutes
    return hours * 60 + minutes
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

最后一行说明错误种类：加法两边出现整数和字符串。往上看，出错操作在函数的计算行；再回到调用者，传入的分钟就是带引号的 `"30"`。这里不应把换算公式改成拼接文字，而应检查调用者为什么没有提供约定的整数。

真实程序里的文字可能来自 `input()` 或命令行；这些入口产生文字，需要在合适的输入处理位置转换后再调用。本例只把调用改为 `to_minutes(9, 30)`，就能与函数当前约定一致。

回溯（traceback）是异常传出时的调用路径，不是此前执行过的全部历史，也不会保证自动显示所有变量值。先用异常类别确定哪种操作失败，再结合项目代码、调用者和必要的值观察找原因，不机械地“只改最后一行”。

不合法的时间本来就该失败，怎样测试？

此前为了聚焦公式，只给函数合法范围内的整数。现在把拒绝越界也写进约定：小时必须为 0–23，分钟必须为 0–59；整数不在范围内时发出 `ValueError`。这里仍约定调用者提供整数，不承担任意对象的完整类型校验。

把 `time_slots.py` 更新为：

PYTHON
```
def to_minutes(hours, minutes):
    if hours < 0 or hours > 23:
        raise ValueError("小时必须在 0 到 23 之间")
    if minutes < 0 or minutes > 59:
        raise ValueError("分钟必须在 0 到 59 之间")

    return hours * 60 + minutes
```

`raise` 在这里主动发出指定异常：条件发现越界，函数不再继续换算，不把错误时间当成正常结果返回。它与 `except` 的方向不同：`raise` 发出失败，`except` 在适当位置处理已经发生的失败。

如果只是普通调用 `to_minutes(9, 60)`，这次出现 `ValueError` 是符合约定的行为。测试不能要求所有输入都成功返回，应该明确检查“这次确实被拒绝”。

在测试里，pytest 提供 `raises` 表达这个要求。将 `test_time_slots.py` 更新为完整文件：

PYTHON
```
import pytest

from time_slots import to_minutes


def test_whole_hour():
    assert to_minutes(9, 0) == 540
```

PYTHON · 续

```
def test_half_hour():
    assert to_minutes(9, 30) == 570


def test_start_of_day():
    assert to_minutes(0, 0) == 0


def test_end_of_day():
    assert to_minutes(23, 59) == 1439


def test_invalid_minutes():
    with pytest.raises(ValueError):
        to_minutes(9, 60)


def test_invalid_hours():
    with pytest.raises(ValueError):
        to_minutes(24, 0)
```

`with pytest.raises(ValueError):` 的意思是：在这个缩进块执行期间，预期出现 `ValueError`。`with` 在这里划定一次受管理的检查范围，不是普通条件判断；不需要先学会编写这种管理工具。

- 出现 `ValueError`，pytest 接住它，符合本次预期；
- 没有异常，说明预期的拒绝没有发生，测试失败；
- 出现不匹配的异常，测试仍失败，不会把任何报错都当作通过。

它也接受指定异常的子类，当前不需要展开异常继承；重点是它检查特定失败，而不是“忽略所有错误”。把被测调用放在这个块里面，才是在检查该调用；不要把无关工作一并塞进去。

再次运行同一测试文件，六条应通过。这些名字是为了看清各自检查哪种行为，不是要求以后每个输入都建立一个新函数。一个测试函数可以有几条相关断言，但其中一条失败后，该函数的后续语句通常不会继续；分开的测试在默认运行下仍会分别执行。不使用提前停止选项时，一项失败不等于整套检查立即结束。

这六条没有覆盖所有错误输入，例如负数、错误类型或更多文本形状。**通过只说明本次实际执行的案例符合写下的预期，不等于函数从此绝不会出错。** 也不等于整个预约程序的显示、保存或真实环境都已经验证。

以后出了问题，留下什么信息才有用？

测试适合重复核对已知行为，调试适合观察某次执行；程序交给别人使用后，不一定有人守着断点，就需要日志帮助找回那次运行。

例如一次时间转换失败，可以记录成下面这种形状。它是日志内容示意，不表示前面的函数已经自动写了日志：

> time=2026-08-01T09:12:00+08:00 run_id=demo-17 stage=convert_time error=ValueError

时间帮助找到发生时刻，`run_id` 关联同一次运行，阶段说明失败发生在哪里，错误类别提供后续排查线索。不要为了“信息完整”把密码、访问令牌或整份客户正文写进去。安全的标识与必要诊断信息通常比无差别打印所有输入更有用；日志的系统组织在 C 册再展开。

编辑器没有红线、格式排得整齐，也不能替代这些行为证据。格式工具整理写法，静态检查和类型提示帮助发现一部分可疑代码；它们各有作用，却没有替本例运行“9 点半应该是 570”这个要求。具体编辑器操作留到 A09，真的需要定位性能瓶颈时才进入性能分析。

这一章应逐渐用顺的是：先写清预期，读懂实际差异，选择最少但足够的观察，再用同一检查核对修改。常用的 `assert`、`test_` 命名和运行测试的方法会不断复用；pytest 的完整选项、调试器命令表以及报告长什么样可以查。

现在，换算函数负责交回分钟数，测试负责核对承诺，调试帮助找原因；它们没有替换彼此。下一章会把注意力转向另一条关系：内存中的值怎样保存到文件，并在程序退出后仍能重新取回。

进一步查阅：CS50P Week 5提供测试的课程主线；Python 的断言语句、错误与异常和pdb 调试器说明相应行为；pytest 的测试发现、断言与预期异常及运行方式用于核对实际操作。Missing Semester 的调试主题提供调试与日志的补充背景。操作文档会更新，运行时应与实际版本一起核对。

A08 | 程序结束后，数据怎样留下来？

报名程序显示了一份名单。关闭程序，第二天再打开，还能取回这些姓名吗？如果它只是把姓名放在变量里并打印，不能。终端可能保留旧输出，但新的一次运行没有接到上一次的变量。

这一章先让两个独立运行的程序通过文件交接名单。随后名单变成有多个字段的记录，再看 CSV 和 JSON 怎样帮助还原结构。最后用一张小图片看同一条关系：**内存中的值不会自动留下；需要明确写出，下一次再读取和解释。**

屏幕上有名单，为什么还不算保存？

先看完整的小程序：

PYTHON
```
first = input("第一位姓名：")
second = input("第二位姓名：")
names = first + "\n" + second + "\n"
print(names, end="")
```

依次输入“小林”和“小周”，`names` 得到字符串 `"小林\n小周\n"`。`\n` 是换行符，因此屏幕显示：

```
小林
小周
```

`print` 默认在输出末尾再加一个换行。这里 `names` 已经带换行，所以用 `end=""` 不再额外加一次。

名单现在是这次运行内存中的字符串。重新启动程序，会重新执行第一行；它并不知道上次的 `names` 是什么。需要另找一个能在运行结束后保留内容的位置，这里选数据文件。

先写出，再由另一份程序取回

以下程序存为 save_names.py。先在自己的新空练习目录中阅读和运行；目录里还没有 names.txt，不要拿真实名单试覆盖操作。

```python
first = input("第一位姓名：")
second = input("第二位姓名：")
names = first + "\n" + second + "\n"

with open("names.txt", "x", encoding="utf-8") as file:
    file.write(names)

print("名单已保存")
```

前三行仍然准备字符串。接下来，open 根据路径打开文件，返回这次操作文件的入口，交给 file。它与 names 不同：一个是文件对象，一个是准备写进去的文字。

"x" 表示只允许新建：目标不存在才打开；已经存在就拒绝。encoding="utf-8" 指定怎样把文字编码成文件中的字节，读回时也按 UTF-8 解释。

file.write(names) 把文字交给文件写入过程。它不自动补换行，所以姓名各占一行，靠的是字符串里的 \n。

with 管理文件的使用范围。正常走出缩进块时，文件关闭，写入缓冲中的内容被交给底层系统，然后才执行“名单已保存”。块内发生异常时，with 也会执行文件退出清理；这不等于自动撤销已经发生的写入。关闭成功也不是“即使随后断电，硬件上一定不会丢失”的普遍保证。

现在保存另一份完整程序 read_names.py：

```python
with open("names.txt", "r", encoding="utf-8") as file:
    saved_names = file.read()

print(saved_names, end="")
```

"r" 表示读取已有文件。read() 读取文字并返回一个字符串，由这次运行的 saved_names 接住。输出仍是小林、小周。

> 第一次运行：输入 → names 字符串 → 写入 names.txt → 运行结束
> 第二次运行：读取 names.txt → 新的 saved_names 字符串 → 输出

两次运行交接的是文件内容，不是变量。关闭文件以后，已经读到内存的 saved_names 仍可使用；with 的缩进不是“所有变量在这里消失”的边界。

在同一练习目录的 PowerShell 中执行：

```powershell
python .\save_names.py
python .\read_names.py
```

这里 `names.txt` 是相对路径，以运行时的当前工作目录为出发点。两份脚本即使放在一起，从不同目录启动，也可能读写不同位置的数据文件。若读不到，先用 `Get-Location` 确认出发点，不要先改打印代码。

文件不一定只能用 `with` 打开，也可以手动调用 `close()`；但那样必须自行保证正常和异常路线都清理。这里需要的只是“使用后关闭”，用 `with` 更直接。应逐渐用顺 `with open(...) as file` 这条基本写法，同时能解释路径、模式和读写动作各管哪件事；不必背下一整份名单程序。

第二次保存，究竟是保护、替换，还是追加？

原样再次执行 `save_names.py`，会在 `open` 遇到 `FileExistsError`。还没进入块内写入，原名单不会被这次打开覆盖。

如果明确想用新名单替换旧名单，可以把模式改成 `"w"`。下面的 `names` 是已经准备好的新字符串：

```python
with open("names.txt", "w", encoding="utf-8") as file:
    file.write(names)
```

`w` 在成功打开已有文件时就会清空旧内容，不会等到 `write`。 后续代码出错、没有写入，不会让旧名单自动回来。文件原本不存在时，`w` 会新建。

如果只是再补一人，不应重新替换整份名单。下面是可以单独运行的追加程序：

```python
with open("names.txt", "a", encoding="utf-8") as file:
    file.write("小陈\n")
```

`a` 把新写入放在文件末尾；目标不存在时也会新建。原来的两行都以换行结束，所以小陈落在第三行。如果原文件末尾没有换行，追加不会替我们猜“应该另起一行”。

这几种模式决定的是对旧文件的态度：

本次打算	模式	已有文件会怎样
取回内容	`r`	读取；不存在时失败
只建立新文件	`x`	已存在时拒绝
明确替换全部内容	`w`	成功打开时清空
在末尾增加内容	`a`	保留原内容，向末尾写

保存后仅执行 `names = "小吴\n"`，文件会跟着变化吗？不会。赋值改变内存中名字所指向的值；只有再次写入才会改变文件。这是本章最值得分清的两份状态。

一行一行读，得到的是什么？

读取整个小文件可以用 `read()`。想逐行处理，可以直接遍历打开的文件：

PYTHON
```
with open("names.txt", "r", encoding="utf-8") as file:
    for line in file:
        name = line.rstrip("\n")
        print("已登记：", name)
```

文件依次提供字符串。第一轮的 `line` 通常是 `"小林\n"`，不是一个姓名对象；`rstrip("\n")` 去掉末尾的换行符，得到要展示的名字。没有指定编码的默认行为可能随环境变化，这里明确使用 UTF-8。

这里不用无参数的 `strip()`，是因为它会连同两端其他空白一起去掉。若业务允许姓名两端空格忽略，当然可以主动选择 `strip()`；文件读取本身不替业务决定哪些字符该保留。

文件最后一行可能没有换行，也可能存在空行。循环都只是交付它读到的文字；要不要忽略空姓名，仍是后续条件判断。文件能打开，不代表每行都符合报名规则。

一个人变成多个字段，逗号为什么不够了？

现在换一个自包含的小任务：把工单的编号、标题和状态保存在文件里。`tickets.csv` 的完整内容如下，以 UTF-8 保存：

CSV
```
ticket_id,title,status
T-103,"Login fails, after reset",open
T-101,Billing question,closed
T-102,Password reset,open
```

本例每行一条记录，第一行说明列名。CSV 是一种表格文字格式。看第一条记录：标题本身包含逗号，所以它用双引号包住，表示这是同一个字段。

如果只调用字符串的 `split(",")`，它会在每一个逗号处拆开：

```python
line = 'T-103,"Login fails, after reset",open'
print(line.split(","))
```

得到：

```
['T-103', '"Login fails', ' after reset"', 'open']
```

本应三个字段，现在变成四份。`split` 没有坏掉，它做的就是“遇到指定分隔符就拆”；它不懂 CSV 的引号规则。这里该换的是格式解释工具，不是删掉标题里的合法逗号。

Python 标准库的 `csv` 模块已经处理这类格式规则。下面完整程序先按位置读取：

```python
import csv

with open("tickets.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

前两次输出是：

```
['ticket_id', 'title', 'status']
['T-103', 'Login fails, after reset', 'open']
```

`reader` 每次交付一个列表，列表里每个元素是一个字段字符串。它把列名那一行也当普通一行；它没有承诺第一行一定是表头。

`newline=""` 让 CSV 模块按自己的规则处理换行，避免文本层先作不合适的换行转换。读写 CSV 时沿用这个参数即可；关键是理解“打开文件交付文字，CSV 再把文字解释成字段”这两步不同。

知道字段名字以后，不必记它在第几列

同一文件可以改用 DictReader：

```python
import csv

with open("tickets.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["ticket_id"], row["title"], row["status"])
```

这里没有显式提供列名，因此 DictReader 用第一行作为字段名，后面每次交付一个字典。第一条数据相当于：

```python
row = {
    "ticket_id": "T-103",
    "title": "Login fails, after reset",
    "status": "open",
}
print(row["title"])
```

row["title"] 用键找到对应的值，因此得到完整标题，而不是含引号的残片。列交换位置但名字不变时，按名字取值通常比固定写 row[1] 更清楚。

不过，能解析不等于内容合格。假如表头把 title 写成 subject，上述 row["title"] 会发生 KeyError：当前字典没有这个键。如果表头正确，但某行比表头少一列，DictReader 默认会给缺少的值填入 None；它不会自动拒绝这条记录。None 是“这里没有提供值”，不同于空字符串 ""。

本例格式要求三列名字和顺序固定，而且每条记录都提供三列。需要接收他人提供的文件时，可以在使用字段前检查这份约定：

PYTHON

```
import csv

expected = ["ticket_id", "title", "status"]

with open("tickets.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.DictReader(file)
    if reader.fieldnames != expected:
        raise ValueError("表头应为 ticket_id,title,status")

    for row in reader:
        if None in row:
            raise ValueError("这一行比表头多了字段")
        for field in expected:
            if row[field] is None:
                raise ValueError("这一行缺少字段：" + field)
        print(row["ticket_id"], row["title"])
```

`reader.fieldnames` 是它读到的字段名列表。字典上的 `in` 检查键：默认设置下，多出来的字段被收在键 `None` 下面，所以 `None in row` 检查的是“有没有额外列”，不是检查值。`is None` 则判断当前值是否就是那个缺失标记。

这段只检查列的结构。`title` 是空字符串、`status` 是 `blocked`，仍可能通过；要不要接受它们，需要另一层业务规则。A13 会用一份完整程序区分“文件读不进来”和“某条记录应当拒绝”，这里不提前堆齐全部校验。

文件顺序不合适，怎样整理后再写出？

希望把尚未关闭的工单按编号列出，得到一个供别人继续读取的新 CSV。先看完整程序，再追踪其中的新动作：

PYTHON

```
import csv


def ticket_id(row):
    return row["ticket_id"]


with open("tickets.csv", "r", encoding="utf-8", newline="") as file:
    tickets = list(csv.DictReader(file))

open_tickets = []
for row in tickets:
    if row["status"] == "open":
        open_tickets.append(row)

ordered = sorted(open_tickets, key=ticket_id)

fields = ["ticket_id", "title", "status"]
with open("open_tickets.csv", "x", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerows(ordered)

print("已写出", len(ordered), "条未关闭工单")
```

这份程序使用前面展示的完整、有效输入；它不是所有坏文件的通用清洗器。开始写目标文件以前，已经完成读取、筛选和排序。第一次运行输出“已写出 2 条未关闭工单”；已有同名输出时，`x` 拒绝覆盖。

`list(...)` 把 reader 逐条提供的字典收集成列表，所以文件关闭后仍能处理 `tickets`。它适合这里的小文件；并不表示所有大文件都应该一次读进内存。

`append(row)` 把当前字典放进 `open_tickets`。原输入依次是 T-103、T-101、T-102，筛选之后留下 T-103、T-102。筛选只改变谁进入新列表，没有给文件重新排序。

排序时，`key=ticket_id` 把函数交给 `sorted`，由排序过程为每条记录取得比较用的值：

```
第一条字典 → ticket_id(row) → "T-103"
第二条字典 → ticket_id(row) → "T-102"
按这些字符串比较 → T-102 在前，T-103 在后
```

这里不写 `ticket_id()`：加空括号会立即调用一个缺少参数的函数，而不是把函数交过去。`sorted` 返回新列表 `ordered`，没有改掉 `open_tickets` 的顺序，更没有改写原来的 CSV。

本例编号长度相同，字符串次序恰好符合要求；`"T-10"` 和 `"T-2"` 按字符串比较时，前者反而在前。如果需求是按数字部分排序，就应先明确取出并转换那部分。工具只按给定的比较值排序，不理解“编号应该怎样排”的业务含义。

当比较函数只有这一句，也常写成：

```python
ordered = sorted(open_tickets, key=lambda row: row["ticket_id"])
```

`lambda row: row["ticket_id"]` 创建一个接收 `row`、返回编号的短函数，与上面的 `def ticket_id` 在这里作用相同。先用有名字的函数看清谁调用谁，再认得这种常见缩写；不需要为了短而把复杂判断塞成一行。

写出时，`DictWriter` 按 `fields` 指定的次序取值。`writeheader()` 写列名，`writerows(ordered)` 写多条记录。带逗号的标题会按 CSV 规则加引号，不需要自己拼接。目标内容为：

```csv
ticket_id,title,status
T-102,Password reset,open
T-103,"Login fails, after reset",open
```

如果手里的数据本来就是按位置排列的列表，可以使用 `csv.writer`。下面是独立例子，写到另一个新文件：

```python
import csv

with open("positions.csv", "x", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["ticket_id", "title"])
    writer.writerow(["T-103", "Login fails, after reset"])
```

`writerow` 写一行，`writerows` 写一组行。选 reader/writer 还是 DictReader/DictWriter，主要看当前程序按位置还是按字段名字表达数据更清楚；两者都比手工拼逗号更了解 CSV。

如果报告不是一张平表呢？

现在要保存一份报告：既有工单列表，又有统计数量。JSON 可以表达这种“一个对象里面装列表，列表里面又装对象”的结构。

以下是独立完整的 save_report.py，不依赖前面程序里的变量：

```python
import json

report = {
    "accepted_count": 2,
    "tickets": [
        {"ticket_id": "T-102", "title": "重置密码"},
        {"ticket_id": "T-103", "title": "重置后无法登录"},
    ],
}

with open("report.json", "x", encoding="utf-8") as file:
    json.dump(report, file, ensure_ascii=False, indent=2)

print("报告已写出")
```

report 是 Python 字典。json.dump 把它表示为 JSON 文字并写入文件；不是把当前变量的内存本身保存进去。ensure_ascii=False 让中文以可直接阅读的字符出现，indent=2 让文件便于人阅读。这两个参数不负责检查工单是否有效。

读回程序 read_report.py：

```python
import json

with open("report.json", "r", encoding="utf-8") as file:
    report = json.load(file)

print(report["accepted_count"])
print(report["tickets"][0]["title"])
```

输出 2 和“重置密码”。逐步看第二行取值：

```
report["tickets"] → 工单列表
列表的 [0] → 第一条工单字典
字典的 ["title"] → "重置密码"
```

load 读取文件里的 JSON，重建这次运行中的 Python 值。JSON 的对象在这里变成 dict，数组变成 list，数字 2 变成整数。它不只是像 file.read() 那样交付一整段原始文字。

CSV 适合这里那张同样字段、逐行记录的表；JSON 适合这份含统计和嵌套列表的报告。并不是后者更高级，所以所有文件都应换成 JSON。

不过，以下三件事仍不同：

文件情况	最早发生什么	还不能认定什么
report.json 不存在	open 发生 FileNotFoundError	还没有读取 JSON
内容是 {"tickets": }	json.load 发生 JSONDecodeError	还没有得到完整值
内容是合法的 []	json.load 成功，得到列表	它并不是程序要求的报告字典

若下游必须接到报告字典，先检查类型，再使用里面的字段。下面片段放在 json.load 之后、使用统计前：

```python
if not isinstance(report, dict):
    raise ValueError("报告应当是一个对象")
if "tickets" not in report:
    raise ValueError("报告缺少 tickets")
if not isinstance(report["tickets"], list):
    raise ValueError("tickets 应当是列表")
```

isinstance(值，类型) 在这里回答“这个值是不是该种类型”。这些判断只查了外层字典和列表，不代表每条工单的字段都查过，也没有证明统计数量与列表一致。**读得懂格式，只是进入数据处理的条件，不是业务正确的证明。**

图片也有读入、转换和写出，只是解释工具不同

文字文件以编码把字节解释成文字；图片需要按 PNG、JPEG 等图片格式解码，才能取得尺寸、像素等信息。不能把 PNG 当 UTF-8 文本读，也不能仅把 .png 改成 .jpg 就声称完成了格式转换。

下面用 Pillow 创建一张小色块，再另一次打开它。只在新的练习目录操作；save 会写目标，同名旧图片可能被覆盖。Pillow 是第三方包，安装部分放在本节末尾，先看完整变化：

```python
from PIL import Image

badge = Image.new("RGB", (200, 100), "teal")
badge.save("badge.png")
```

Image.new 在内存中建立宽 200、高 100 的 RGB 彩色图片；save 才把它编码并写成 PNG。导入名是 PIL，安装包名是 Pillow，两者不是必须同名。

再运行另一份程序：

```python
from PIL import Image

with Image.open("badge.png") as original:
    gray = original.convert("L")
    small = gray.resize((100, 50))
    small.save("badge_small.png")

print(small.size)
```

`Image.open` 识别图片并准备读取，实际像素可能按需才载入；`convert("L")` 产生灰度图片，`resize((100, 50))` 再产生指定尺寸的新图片。最后 `save` 写到另一个文件，输出的尺寸是 `(100, 50)`。

原文件没有被这些转换自动改掉。这里选择的新尺寸保持了宽高比；任意指定不同比例的宽高可能使图像变形，`resize` 不会替我们决定是否应该裁剪。

上机时可以为这个独立例子建立环境：

```powershell
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install Pillow
.\.venv\Scripts\python.exe .\make_badge.py
.\.venv\Scripts\python.exe .\resize_badge.py
```

把前后两段 Python 分别保存为 `make_badge.py` 和 `resize_badge.py`。`.venv` 应在新练习目录建立，不覆盖已有环境；安装需要网络。版本相关格式和图像选项用到再查，先能指出“读入的是哪一份，转换得到哪个新值，最后写到哪里”。

读写失败时，先找到哪一步还没有完成

不要把所有文件错误都写成“内容不对”：路径找不到、编码不匹配、格式解析失败、记录违反规则，发生的位置不同。

例如读取名单时，`FileNotFoundError` 指向路径；拿错误编码读字节可能出现 `UnicodeDecodeError`；打开目录或无权访问的目标，也可能在打开阶段失败。应先核对对应事实，不能用一个包住全程序的 `except Exception: pass` 把证据抹掉。

写文件还多一个关键问题：失败以前，旧文件有没有已经改变？`w` 已经清空目标，之后出错仍可能留下空文件或半文件。先把内存里的数据检查完，再打开输出，能减少提前破坏；但不能消除写入中断。需要保护一份重要的旧报告时，A13 会先写临时文件、成功后再替换，并准确限定这种做法的保证。

文件把程序的结果留给下一次运行。现在的完整关系是：外部字节经路径和格式解释成为 Python 值，程序筛选、验证或转换，再明确写出供下游读取。文件保存了什么、下一步愿意接收什么，必须两边说得一致；这比记住一组读写函数名字更重要。

继续练习与查用

本章的三个小任务——名单、工单表、报告——已经分别展示持久保存、字段解释和嵌套结构。上机不必反复重抄同类代码：可以把 `tickets.csv` 中 T-101 的状态改为 `open`，先预测新的未关闭工单表有几条、顺序是什么，再把前面的筛选排序程序的输出名改为一个不存在的新名字，重新运行它。新 CSV 应得到 T-101、T-102、T-103 三条；后面的 `save_report.py` 使用独立写好的数据，不会因为修改这个 CSV 自动改变。若只修改内存变量却未重新写出，旧文件也不会改变。

需要额外迁移时，CS50P Week 6 的 Scourgify 可用于 CSV 字段整理：输入姓名与学院，输出拆分后的姓名列；重点是字段映射和引号处理，不是复用本章工单字段。Shirt 可用于图片尺寸和叠加：输入照片，输出适合指定服装图层的图片，所需裁剪与合成规则应再按题目说明查用。它们是独立任务，不是理解本章的前置，也不要求在此完成全部官方题。

官方相对顺序与查用入口：CS50P Week 6、Week 6 题目。具体 API 行为可查 Python open、CSV、JSON 与 Pillow 教程。这些是持续更新的官方入口，实际软件行为以运行时版本为准。

A09 | 怎样确认运行条件没有找错?

代码改了，输出却没变；包安装成功，程序却说找不到。先别怀疑自己是否忘了某条 Python 语法：编辑器、终端和安装工具，可能正在处理不同的文件或环境。

这一章先让程序说出“我是谁、从哪里开始、由哪个 Python 运行”。再用这份证据解释安装为何只对一个环境生效，配置为何选中了另一份值。最后补上查找代码和看编辑器提示的方法。重点是**先让结果归属于明确的运行条件，再判断代码本身**。

这段输出到底属于谁?

在自己的新练习目录中，把下面完整程序以 UTF-8 保存为 `identity.py`：

PYTHON
```
import sys
from pathlib import Path

print("SCRIPT =", Path(__file__).resolve())
print("CWD =", Path.cwd())
print("PYTHON =", sys.executable)
```

在该目录的 PowerShell 中运行：

POWERSHELL
```
python .\identity.py
```

程序只报告三件事，不安装包，也不修改文件。比如在示例目录 `D:\FDE-Lab\A09` 运行，可能看到：

```
SCRIPT = D:\FDE-Lab\A09\identity.py
CWD = D:\FDE-Lab\A09
PYTHON = ……\Python313\python.exe
```

前两行是当前脚本与工作目录，第三行是实际启动的解释器完整路径，省略号只表示每台电脑安装位置不同，实际输出不会替你省略。

`__file__` 在这个脚本里提供当前模块文件的位置。`Path(...)` 把路径文字交给路径对象处理，`.resolve()` 得到消解相对部分等因素后的完整路径。`Path.cwd()` 直接询问当前工作目录；`sys.executable` 由正在运行的 Python 报告自己的可执行文件位置。它们是三份不同证据，不应该总期待相同。

从上一级目录运行 `python .\A09\identity.py`，SCRIPT 仍指向 A09 中的文件，但 CWD 是上一级目录。运行脚本不会自动让工作目录变成脚本所在目录。这也解释了：脚本同目录明明有 `names.txt`，用相对路径读取时却可能找不到。

如果编辑器打开的是另一份 `identity.py`，刚才的输出当然不会跟着它的修改变化。先对照编辑器显示的完整文件路径与 SCRIPT，确认它们是同一份，再在这份文件里加一行独有输出重新运行。保存、运行和观察要接到同一目标上。

命令中的 python，与真正启动的解释器是一回事吗？

`python` 是命令名，不是电脑上所有 Python 的统称。PowerShell 需要先解析这个名字，再启动它：

```powershell
Get-Command python -All | Select-Object CommandType, Name, Source
python -c "import sys; print(sys.executable)"
```

第一行列出 PowerShell 发现的同名命令，`-All` 有助于看见同名项；第二行让实际启动的 Python 报告自己。`-c` 表示直接执行后面的短 Python 文字，不读取 `.py` 脚本。这两条都不改变解释器选择。

PowerShell 不一定一开始就只查 PATH。它还可能先匹配 alias（别名）、function（函数）或 cmdlet（PowerShell 命令），外部程序的查找才涉及 PATH 中的目录及先后关系。知道这一点，是为了不把 `Get-Command` 列出的名字误当成已经运行的 Python 进程。这里不要求改 PATH；若名字有歧义，可以先使用已经确认的解释器路径。

命令输出多了一条也不等于故障。重要的是项目本来希望用哪个解释器，这次真正启动的是哪个。不要凭“最新安装”或窗口标题猜。

包装在 A 环境，为什么 B 还是找不到？

创建两个环境，让差异变得可见。虚拟环境提供各自的 Python 入口和包安装位置；本例不激活任何一个，命令明确指向要使用的入口。

只在刚才自己的新练习目录中执行，确认还没有这两个环境目录：

```powershell
python -m venv .venv-a
python -m venv .venv-b
& .\.venv-a\Scripts\python.exe -m pip install httpx
```

每条成功后才继续。前两条创建环境，第三条把 HTTPX 装给 A 的 Python；它没有给 B 安装。`&` 是 PowerShell 的调用运算符，用来运行后面的可执行文件；这里直接写明路径，所以不依赖激活或提示符前是否有环境名。

把下面完整程序保存为 `import_probe.py`：

```python
import sys

print("PYTHON =", sys.executable)

import httpx

print("HTTPX =", httpx.__file__)
```

分别运行：

```powershell
& .\.venv-a\Scripts\python.exe .\import_probe.py
& .\.venv-b\Scripts\python.exe .\import_probe.py
```

A 显示自己的解释器，以及导入的 HTTPX 文件位置。B 先报告 B 的解释器，然后在 `import httpx` 出现 `ModuleNotFoundError`，没有走到最后一行。

> A 的 python.exe → pip 把包安装到 A 可使用的位置
> B 的 python.exe → 用 B 的导入查找路径找 httpx → 没找到 → 导入失败

错误不是“HTTP 服务拒绝了请求”：这份程序根本没有发请求。也不是 A 的安装失效；安装成功只属于那个具体环境。

如果项目应该使用 A，就让运行指向 A，再核对输出。如果项目应该使用 B，则按项目依赖说明安装到 B。不要同时修改两个环境、PATH、编辑器和源代码，否则即使好了，也不知道哪项改变产生了作用。

本例用普通 `venv` 新建两个环境、没有额外改变导入路径，因此 B 不会使用 A 的包。更复杂环境也可能通过配置加入其他查找位置，所以实际导入的 `httpx.__file__` 比“目录名看着像 A”更有解释力。若在项目里误建了同名 `httpx.py`，也可能挡在真正的第三方包前面；先看导入位置，再决定如何修正文件名。

编辑器选了 A，终端会自动跟着变吗？

VS Code 的 Python 功能可以选择解释器，用于运行、调试和语言功能；集成终端则有自己的当前会话状态。扩展可以为新终端自动激活环境，调试设置也可能单独指定 Python。已经打开的终端是否跟着变化，不能只靠状态栏推断。

在装有 Python 扩展的 VS Code 中，打开当前练习目录和 Python 文件，用命令面板中的 `Python: Select Interpreter` 选择 A；若没有自动发现它，按当前界面提供的输入路径入口指定 `.venv-a\Scripts\python.exe`。然后分别用编辑器运行和终端明确路径运行 `identity.py` 或 `import_probe.py`，比较两次 PYTHON。

如果编辑器提示 `httpx` 不存在，而终端 A 已经成功导入，就先检查编辑器所选解释器和当前工作区，不要为了消除提示把包到处装一遍。反过来，编辑器没有红线，也不能证明某个终端正在用同样的环境。

这里不需要记住按钮坐标。要逐渐熟悉的是：从运行证据找回实际解释器，再让编辑器和命令明确指向项目要用的那一个。具体界面随扩展更新，查官方当前说明即可。

配置都叫 mode，谁的值会生效？

文件和解释器已经对齐，输出仍可能来自另一份配置。配置是程序运行时读取的设置，不等于代码本身。例如程序要决定当前模式是 `safe` 还是 `training`。

先看独立的小例子。把 `config.json` 和 `show_mode.py` 放在同一练习目录。配置文件为：

```JSON
{"mode": "training"}
```

完整程序为：

```PYTHON
import json
from pathlib import Path

config_path = Path(__file__).resolve().with_name("config.json")
with config_path.open("r", encoding="utf-8") as file:
    config = json.load(file)

print("MODE =", config["mode"])
print("SOURCE =", config_path)
```

`with_name("config.json")` 用同一个目录下的新文件名形成路径。这里明确让配置跟着脚本，而不是跟着启动目录；即使从上一级启动，仍读脚本旁的配置。`.open(...)` 则打开这个路径对应的文件，和 A08 的 `open(路径, ...)` 承担相同动作。

文件里的 `"training"` 经 JSON 解析成为字典值，然后输出 `MODE = training`。它之所以生效，是因为代码确实打开这份文件并取了这个字段，不是因为文件碰巧叫 config。

现在增加一个真实要求：某一次运行需要临时指定模式，不想编辑配置文件。可以让父 PowerShell 给新启动的 Python 传入一个环境变量。环境变量是一组进程能读取的名字和值；这里约定变量名为 `FDE_LAB_MODE`。

应用的优先顺序由代码决定：有这个环境变量就用它；没有时读脚本旁的配置；配置文件不存在才用默认值 `safe`。任何已提供的空值或不认识的模式都应拒绝，不偷偷退回默认值。

把 show_mode.py 完整替换为下面这一版：

```python
import json
import os
from pathlib import Path


def choose_mode():
    config_path = Path(__file__).resolve().with_name("config.json")

    if "FDE_LAB_MODE" in os.environ:
        mode = os.environ["FDE_LAB_MODE"]
        source = "environment FDE_LAB_MODE"
    else:
        try:
            with config_path.open("r", encoding="utf-8") as file:
                config = json.load(file)
        except FileNotFoundError:
            mode = "safe"
            source = "code default"
        else:
            if not isinstance(config, dict) or "mode" not in config:
                raise ValueError("config.json must contain mode")
            mode = config["mode"]
            source = str(config_path)

    if not isinstance(mode, str) or mode not in ("safe", "training"):
        raise ValueError("mode must be safe or training")
    return mode, source


mode, source = choose_mode()
print("MODE =", mode)
print("SOURCE =", source)
```

os.environ 按名字访问当前进程的环境值；in 在这里检查该名字是否存在。mode, source 分别接住函数返回的两个结果。允许值写在括号中，mode not in (...) 表示它不属于这两个值中的任何一个。

这里恢复了 A05 的异常关系：先尝试读取，只有 FileNotFoundError 进入默认值分支；读取成功才到 else 使用配置。文件已经存在却缺 mode 时，程序主动报错，不能宣称“读了配置文件所以来源正确”，实际却拿着默认值。配置文字损坏或无权读取时，错误也继续暴露；与文件根本没提供是不同情况。环境变量存在时，分支不去读配置文件，所以本次成功也不证明被跳过的配置文件有效。

可以先在纸上跟两条路线：未提供环境变量，文件给 `training`，结果来源是文件；若环境变量给 `safe`，优先选环境，文件里的 `training` 没有被改，只是本次没选它。

上机时临时设置前先记住原来的状态，结束后恢复，不破坏已有设置：

POWERSHELL
```
$hadLabMode = Test-Path -LiteralPath Env:FDE_LAB_MODE
$previousLabMode = $env:FDE_LAB_MODE
try {
    $env:FDE_LAB_MODE = 'safe'
    python .\show_mode.py
    if ($LASTEXITCODE -ne 0) { throw 'Mode program failed.' }
} finally {
    if ($hadLabMode) {
        $env:FDE_LAB_MODE = $previousLabMode
    } else {
        Remove-Item -LiteralPath Env:FDE_LAB_MODE -ErrorAction SilentlyContinue
    }
}
python .\show_mode.py
```

`try/finally` 是 PowerShell 这次实验的收尾写法：无论尝试部分怎样结束，都按原来是否存在恢复变量。第一次显示来源 `environment FDE_LAB_MODE`；最后一次使用原来的环境值（若本来存在），否则才使用文件或默认值。最后一次若恢复到了原来不合法的值，仍会拒绝，不能固定期待它总是 training。

这段完整恢复写法可以查着用。重要的关系是：本次 Python 在启动时获得父进程环境的一份副本，不会因为后来另一个终端改了同名变量，就自动读到新值。PowerShell 版本对空环境值的支持有差别，勿把“设为空字符串”当作通用的删除动作；上述代码按存在与原值恢复。

编辑器的 User Settings 和 Workspace Settings 是另一套配置，后者针对工作区，具体优先级还受语言、文件夹等作用域影响。它们可以改变编辑器功能，却不会自动成为应用的 `FDE_LAB_MODE`。只有启动配置或程序代码明确连接的值，才参与该次运行。

文件越来越多，怎样找到真正要看的位置？

下面单独准备一个小查找目录，不扫描环境安装目录。在自己的练习目录下，用编辑器建立 `search_demo` 及两个文件：

```
search_demo/
 ├── settings.py
 └── app.py
```

settings.py 完整内容：

```python
def choose_mode():
    return "training"
```

app.py 完整内容：

```python
from settings import choose_mode

print(choose_mode())
```

先按文件名找出所有 Python 文件，再看完整位置：

```powershell
Get-ChildItem -LiteralPath .\search_demo -Recurse -File -Filter *.py |
    Select-Object Name, FullName
```

.py 是文件名匹配： 代表任意一串字符，所以它匹配这些 .py 文件，不是要求存在一个名字真的叫星号的文件。-Recurse 进入子目录，-File 只取文件，-Filter 限制名字。

| 是管道，它把左边的结果交给右边。这里左边交付的是文件对象，包含 Name、FullName 等属性；Select-Object 选择展示哪些属性。不是把屏幕画出来的表格再拆成文字列。

若问题是“choose_mode 在哪里定义、哪里使用”，则要搜内容：

```powershell
Get-ChildItem -LiteralPath .\search_demo -Recurse -File -Filter *.py |
    Select-String -SimpleMatch -Pattern 'choose_mode' -Encoding UTF8
```

Select-String 根据这些文件路径读内容，显示文件、行号和匹配行。-SimpleMatch 明确按普通文字查找，不在这里引入正则表达式。会找到 settings.py 的定义，以及 app.py 的导入和调用三处。

搜索命中不证明这些行实际执行过：注释和未调用的函数也能含同一串文字。搜索先帮助定位，打开上下文后，再通过运行或测试判断当前路线。

不记得某个参数时，不用把整套帮助背下来：

```powershell
Get-Help Get-ChildItem -Parameter Filter
Get-Command Select-String -Syntax
```

第一条查看特定参数帮助；本地详细帮助没安装时可能只有有限信息，可以转向官方对应页面。第二条查看命令语法。它们帮助查具体写法，不替你决定应该搜索文件名还是内容。

格式整齐、没有红线，是否就能证明程序对了？

以下完整函数用类型提示说明意图：

```python
def total_minutes(values: list[int]) -> int:
    return sum(values)


print(total_minutes([10, 20]))
```

`values: list[int]` 表示预期收到整数列表，`-> int` 表示预期返回整数。`sum` 把当前列表里的数加起来，所以这次输出 30。

把调用换成 `total_minutes([1.5, 2.5])`，普通 Python 运行仍会算出 `4.0`，不会仅因提示写了 int 就阻止它。若传 `['10', '20']`，则会在 `sum` 的实际相加操作处出现 `TypeError`；不是提示先替程序完成了输入校验。

formatter 主要整理空格和换行；diagnostics 是编辑器或检查工具给出的诊断，可能发现语法、未定义名字、导入或类型关系问题，能力受工具和配置影响。运行测试则检查实际输入对应的结果。排版、静态提示、运行证据互相补充，不能用其中一个冒充全部正确。

这里不必现在安装一整套格式和类型工具。等代码规模需要它们协助时，知道应期待哪种帮助、哪些结果仍要实际验证，就不会把“没有红线”当成需求已经实现。

给这个小实验留下一份能照着做的说明

环境问题解决以后，留下明确入口，避免下次只记得“在某个终端里能跑”。例如本章 HTTPX 实验的简短说明可以是：

> 打开：包含 identity.py 与 import_probe.py 的练习目录。
> 解释器：正常导入实验选择 .venv-a\Scripts\python.exe。
> 安装：用该解释器执行 -m pip install httpx，不安装到 B。
> 运行：用该解释器执行 .\import_probe.py，核对 PYTHON 和 HTTPX 两个路径。
> 验证：A 成功导入，B 在 import httpx 处失败；本实验不发送网络请求。

本章说明的是这次小实验的运行条件；正式程序还需要依赖版本、测试与重建说明，A12 再把这些连完整。

现在，面对“装过却找不到”，可以先比较安装与运行的解释器；面对“改了没生效”，先比较文件和配置来源；面对“提示与运行不一致”，再比较编辑器所选环境。每次先找最早不一致的位置，只修正那一处后重新取证，比反复重装更容易解释结果。

官方查用入口

路径与环境机制：Python pathlib、虚拟环境 venv、sys。命令解析与查找：PowerShell 命令优先级、Get-ChildItem、Select-String。编辑器入口：VS Code Python 环境与设置作用域。这些是持续更新入口，实际界面、命令和解释器行为须与所用版本对应。

A10 | 程序运行后，怎样观察和停止？

先不要把后台任务、管道和权限命令一起记住；先沿同一个实例看启动、输出、响应、停止和复查证据。

输入启动命令以后，终端没有新的提示符：程序是在等你输入，正在计算，还是准备一直等请求？屏幕出现一行错误文字，又能不能直接判断这次运行失败了？

这一章先让一个短程序读入一行文字、处理并结束，看结果怎样交回终端。随后运行一个本机文件服务，观察它为什么不主动结束，以及怎样确认自己启动的程序已经停止。重点是把**正在运行的实例、它交出的结果，以及结束的证据**连起来；后台运行方式等这条关系清楚以后再进入。

一次短程序怎样交回结果

程序文件是磁盘上的代码；操作系统启动它以后，产生一次正在运行的实例，叫进程（process）。同一个文件可以启动两次，成为两个进程。进程结束，文件还在；下次运行会得到新的内存和新的运行身份。

先用一个只处理一行编号的程序：去掉两端空白，把编号转成大写。有效结果应当单独交给后续程序；去掉过空白时另给人一个提示；没有编号时拒绝这次处理。

如要跟着操作，使用已经准备好的独立练习目录；如果目录还没有准备好，先只把下面程序保存为 UTF-8 的 `normalize_line.py`。目录创建、解释器选择和路径核对是一次性查用动作，第一次理解本章时不需要先消化它们。真正要观察的是：程序收到一行输入后，交出了什么结果。

```python
import sys


def main():
    raw = input()
    code = raw.strip()

    if not code:
        print("error: code is empty", file=sys.stderr)
        return 2

    if code != raw:
        print("note: surrounding spaces removed", file=sys.stderr)

    print(code.upper())
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

`input()` 读一行文字并去掉输入结尾的换行；`strip()` 再去掉文字两端的空白；空字符串在 `if not code` 中成立。这里没有写 `input("请输入：")`，因为提示文字也会进入标准输出，我们希望这条通道只包含处理结果。

运行：

```powershell
python .\normalize_line.py
```

它没有立即显示新提示符，正在 `input()` 等待。输入 `ticket-42`，按 Enter，看到：

```
TICKET-42
```

随后 PowerShell 提示符回来。立刻读取这次运行的退出码：

```powershell
$runExit = $LASTEXITCODE
$runExit
```

结果是 `0`。`$LASTEXITCODE` 保存最近一次原生程序的退出码；Python 是操作系统直接运行的原生程序，不是 PowerShell 内部命令。先存到自己的变量，是为了避免后面又运行其他原生程序，把这个证据替换掉。

这里有三条数据通道和一个结束结果：

> 键盘的一行 ticket-42
> → stdin（standard input，标准输入）
> → input 得到字符串 → strip → upper
> → stdout（standard output，标准输出）：TICKET-42
>
> 需要给人看的诊断
> → stderr（standard error，标准错误）
>
> main 返回 0 → sys.exit(0) → 进程结束 → PowerShell 得到退出码 0

`print(...)` 默认写 stdout；`print(..., file=sys.stderr)` 改用 stderr。两者通常都显示在同一个终端，所以只看屏幕位置，未必分得出通道。`return 0` 先把整数交给 Python 内部的调用位置；是外层 `sys.exit(main())` 把它变成整个进程的退出码。普通函数返回一个数字，不会自动结束整次运行。

有诊断文字，这次运行也可能成功

不必每次手工输入。PowerShell 的管道符 | 可以把左侧文字交给右侧 Python 的 stdin。在同一练习目录中运行下面这段；这些输出文件尚不存在：

POWERSHELL
```
"  ticket-42  " | python .\normalize_line.py > result.txt 2> diagnostic.txt
$trimExit = $LASTEXITCODE
Get-Content -LiteralPath .\result.txt
Get-Content -LiteralPath .\diagnostic.txt
"exit=$trimExit"
```

读回的结果依次是：

> TICKET-42
> note: surrounding spaces removed
> exit=0

`>` 将标准输出重定向到文件；`2>` 把标准错误交给另一个文件。重定向只是改了接收者，没有改程序的处理规则。本例 `result.txt` 只有编号，后续程序可以继续使用；诊断文字留在另一条通道里。`>` 会覆盖已有目标，因此练习使用刚创建的空目录，重跑时也要知道将替换哪份练习输出。

现在发送一行只有空白的文字：

```powershell
"   " | python .\normalize_line.py > empty-result.txt 2> empty-diagnostic.txt
$emptyExit = $LASTEXITCODE
Get-Content -LiteralPath .\empty-result.txt
Get-Content -LiteralPath .\empty-diagnostic.txt
"exit=$emptyExit"
```

`empty-result.txt` 没有业务结果。诊断文件包含 `error: code is empty`，退出码是 `2`。因为 `strip()` 得到空字符串，程序在 `return 2` 处离开 `main`，没有执行后面的 `print(code.upper())`。

这两个输入的差别能回答一个常见误判：**stderr 里有文字，不等于退出码非零。** 程序可以把提醒、进度或日志写进 stderr，并正常返回 `0`；也可能几乎没有输出就以非零退出码结束。这里的 `2` 是这个程序给“编号为空”约定的结果，不是所有程序对 `2` 都有同一种解释。判断某个程序的成功条件，要同时看它约定的输出和退出码。

> **第一次学到这里先停。** 你现在只需要能沿着一条短程序说清：输入从哪里进来，正常结果从哪里出来，诊断文字和退出码分别证明什么。后面的管道、环境变量、后台服务和权限，是把同一条关系搬到更复杂的运行场景；第一次阅读可以先跳过，等遇到对应问题再回来。

查用：没有现成练习目录时再准备一次

如果没有带学时准备好的目录，再用下面几行创建一个临时目录；它们只负责放置本章文件，不是本章要背的 PowerShell 语法。创建后回到上面的 `normalize_line.py` 和结果观察。

```powershell
$labName = "a10-lab-" + [guid]::NewGuid().ToString("N")
$lab = New-Item -ItemType Directory -Path (Join-Path
  ([IO.Path]::GetTempPath()) $labName)
Set-Location -LiteralPath $lab.FullName
```

PowerShell 5.1 与较新的 PowerShell 对原生 stderr 的显示包装、重定向文件编码可能不同；诊断文件有时还带 PowerShell 的错误记录信息。别把颜色、错误记录前缀或文件字节编码当成 Python 的退出码。本章先用 ASCII 编号看清通道；如果以后要通过管道传中文，再查发送方编码、Python 标准流编码，以及接收方解码是否一致。

管道里流动的，什么时候是文字，什么时候是对象

刚才的 `" ticket-42 " | ...` 经过 PowerShell 到原生程序的交界时，文字被编码后写进 Python 的 stdin。Python 得到一行字符串，并没有得到 PowerShell 的变量对象。

而下面两端都是 PowerShell 命令：

```powershell
Get-Item -LiteralPath .\normalize_line.py | Select-Object Name, Length
```

`Get-Item` 交出的文件对象带有 `Name`、`Length` 等属性；`Select-Object` 选出属性，最后才显示成表格。它没有先把屏幕上的一行表格文字拆回属性。

这会影响你怎样连接工具。`Get-Item ... | python ...` 不会把一个可访问 `.Length` 的文件对象直接交给 Python；到了原生进程边界，要明确约定传文字、文件路径还是 JSON。反过来，Python 打印 JSON 时，PowerShell 首先收到的是文字；需要 `ConvertFrom-Json` 才能按字段继续处理。

本章后面会用 `Get-NetTCPConnection | Select-Object ...` 查看监听信息。它传递带属性的连接对象，因此能选择 `OwningProcess`，不必先猜表格中 PID 在第几列。PowerShell 还有其他流、参数绑定及不同版本的原生字节管道规则，遇到真实需求时查用；现在应能解释当前交界两边各拿到了什么。

新进程从哪里得到环境变量

同一份程序用同一个解释器启动，仍可能因为配置不同而表现不同。环境变量是启动进程时交给它的一份字符串配置。通常，子进程取得父进程当时环境的一份副本；这份副本不会随着父进程后来的修改自动同步。

从这里开始，如果需要把“谁执行了程序”也固定成证据，再选定一个解释器路径。下面这一行只是为后续查用操作准备变量，不是 A10 首次需要背的命令：

```powershell
$pythonExe = (Get-Command python -CommandType Application | Select-Object
  -First 1).Source
```

如果它找不到 Python，先处理解释器入口；不要把入口问题误判成程序内部的输入或输出问题。

把下面程序保存为 `show_mode.py`：

```python
import os

print("received=" + os.getenv("A10_MODE", "unset"))
os.environ["A10_MODE"] = "child-only"
print("inside=" + os.environ["A10_MODE"])
```

`os.getenv` 读取本进程环境，变量不存在时返回这里给出的 `"unset"`；`os.environ[...] = ...` 修改本进程环境。下面先保存当前 PowerShell 的旧值，运行完再恢复：

```powershell
$oldMode = $env:A10_MODE
try {
    $env:A10_MODE = "training"
    & $pythonExe .\show_mode.py
    "parent=" + $env:A10_MODE

    $env:A10_MODE = "review"
    & $pythonExe .\show_mode.py
}
finally {
    if ($null -eq $oldMode) {
        Remove-Item Env:A10_MODE -ErrorAction SilentlyContinue
    }
    else {
        $env:A10_MODE = $oldMode
    }
}
```

输出为：

```
received=training
inside=child-only
parent=training
received=review
inside=child-only
```

第一份 Python 从父 PowerShell 继承 `training`，随后只改自己的副本，因此父进程仍是 `training`。父进程改为 `review` 后再启动第二份 Python，第二份才收到 `review`。已经启动的第一份不会被追溯修改；若它还在运行，需要它自己实现重新读配置的机制，或者重新启动它。

`try/finally` 在这里负责练习收尾：不管中间是否成功，都恢复当前会话的原值；原来没有变量，就恢复为不存在。这些命令没有写入 Windows 的永久用户或系统环境。另一个已经打开的终端也有自己的环境，不能因为它显示同一个目录，就假设它拿到了这里刚改的值。

服务为什么一直占着当前终端

刚才的程序处理完一行就结束。服务需要反复接收请求：这次回应以后，还要等下一次。因此提示符迟迟不回来，可以是它正在正常等待。

先在当前练习目录创建 `status-site` 文件夹，用编辑器在其中保存 `status.json`，完整内容是：

```JSON
{"status": "ok", "purpose": "a10-local-check"}
```

这个目录只放这份练习文件，不放个人资料、密钥、链接或目录联接。接下来使用的 `http.server` 是 Python 自带的临时静态文件服务。`--directory` 指定取文件的目录，但它不是安全沙箱：服务会跟随符号链接，可能读到目录外的文件，也不提供本例之外的生产服务保障。

在当前终端先检查练习端口：

```POWERSHELL
Get-NetTCPConnection -State Listen -ErrorAction Stop |
    Where-Object { $_.LocalPort -eq 8765 } |
    Select-Object LocalAddress, LocalPort, OwningProcess
```

正常情况下没有匹配行。若显示了已有 listener（正在监听连接的入口），先别启动；换一个未被使用的练习端口，并同步更改后面的启动命令和请求地址。不要结束未知 PID 来腾位置。按端口查询全部本地地址，也能看见监听在 `0.0.0.0` 等地址上的占用者。只有命令能正常执行且没有匹配行，才把它当作“检查时未发现占用”；命令不存在或权限错误都不等于端口空闲。

在这个终端，也就是终端一，运行：

```POWERSHELL
$serveDir = (Resolve-Path -LiteralPath .\status-site).Path
"serving directory: $serveDir"
& $pythonExe -u -m http.server 8765 --bind 127.0.0.1 --directory $serveDir
```

会看到类似输出：

```
Serving HTTP on 127.0.0.1 port 8765 (http://127.0.0.1:8765/) ...
```

先保留这个终端。`-m http.server` 运行标准库模块；`-u` 让 Python 的标准输出与标准错误不等缓冲积累，观察时更及时。服务启动信息出现以后，它等网络请求，不在等 `input()`，也还没有完成这次运行。

`127.0.0.1` 是本机 IPv4 回环地址，绑定它使这个入口只接受本机连接；`8765` 是在这个地址上区分入口的端口号。端口号不是 PID：多个程序可能先后使用同一个端口，同一进程也可以监听多个端口。`localhost` 则是需要解析的名称，可能解析到 IPv4 `127.0.0.1` 或 IPv6 `::1`；这里统一用地址字面量，避免客户端走 IPv6 而服务只监听 IPv4。

从第二个终端追到真正的回应

打开第二个 PowerShell 终端，终端一继续运行服务。第二个终端的当前目录可以不同，因为这里先查操作系统中的监听信息：

```powershell
$listener = @(Get-NetTCPConnection -State Listen -ErrorAction Stop |
    Where-Object { $_.LocalAddress -eq '127.0.0.1' -and $_.LocalPort -eq 8765
      })
if ($listener.Count -ne 1) { throw 'Expected exactly one listener; inspect
  startup and ownership.' }
$listener | Select-Object LocalAddress, LocalPort, OwningProcess
$serverPid = $listener.OwningProcess
Get-CimInstance Win32_Process -Filter "ProcessId = $serverPid" |
    Select-Object ProcessId, ParentProcessId, ExecutablePath, CommandLine
```

一份示意结果是 `LocalAddress=127.0.0.1`、`LocalPort=8765`、`OwningProcess=4312`；实际 PID 每次可能不同。`Get-CimInstance` 进一步显示这个进程的解释器位置、父进程编号和启动命令。核对 `CommandLine` 中的 `http.server`、端口，以及终端一刚打印的完整 `status-site` 路径，确认它属于这次练习。只看到进程名称 `python` 还不够。

预检与启动之间也可能有别的程序抢先占用端口。如果这里的命令行不属于本次练习，停止当前核对，回终端一看启动错误；不能把查到的占用者当作本次服务。核对通过后，保存自己这次进程的身份和启动时间：

```powershell
$serverProcess = Get-Process -Id $serverPid -ErrorAction Stop
$serverStart = $serverProcess.StartTime
$serverProcess | Select-Object Id, ProcessName, StartTime
```

现在发一次真实请求：

```powershell
$response = Invoke-WebRequest http://127.0.0.1:8765/status.json
  -UseBasicParsing -TimeoutSec 3
$payload = $response.Content | ConvertFrom-Json
$response.StatusCode
$payload.status
$payload.purpose
```

预期输出：

```
200
ok
a10-local-check
```

`Invoke-WebRequest` 是这次请求的客户端；服务读取 `status.json`，把文件内容放进 HTTP 回应。`StatusCode` 是这一次请求的结果，`Content` 是收到的正文。JSON 文字经过 `ConvertFrom-Json` 才变成可按 `.status` 取字段的对象。`-TimeoutSec 3` 为请求设置等待上限；这里是本机地址，没有额外的域名解析步骤。`-UseBasicParsing` 兼容 Windows PowerShell 5.1 的基础解析方式。

终端一也会新增类似 `"GET /status.json HTTP/1.1" 200 -` 的访问日志；日志中的时间和客户端信息会变化。它帮助服务端说明收到了什么，但客户端仍要检查自己收到的状态和内容。

现在三份证据各自有了用途：

证据	已经证明什么	还没有证明什么
本次 Python 的 PID 与启动命令	这个运行实例存在，身份与本次实验相符	它已监听预期端口
listener 的地址、端口和 OwningProcess	本次进程在预期入口等待连接	指定文件存在，回应内容正确
请求得到 200 和预期 JSON 字段	这一次请求取到了预期内容	以后永远可用，其他资源也正确

试着把请求路径改成 `/missing.json`，仍保留相同地址、端口和超时。文件不存在时服务返回 HTTP `404`；`Invoke-WebRequest` 默认把这类状态显示为错误。此时进程与 listener 可以都还在，失败发生在“请求的资源没有找到”。不要从 `404` 跳到“Python 没启动”，也不要用 HTTP `200` 代替进程退出码：服务回应 `200` 以后，仍在继续运行。

发出停止动作以后，还要看一次结果

回到终端一，按一次 Ctrl+C。这会把键盘中断交给前台程序；这个 `http.server` 会处理它并退出，PowerShell 提示符回来。它不会删除 `status.json`。别把通用的“所有程序 Ctrl+C 都返回某个固定数字”当作规则：不同程序可以自行处理或忽略中断，退出结果要看该程序。

回到保留 `$serverPid` 与 `$serverStart` 的终端二，用最多约五秒的复查看自己这次进程和 listener 是否都消失：

```powershell
$deadline = (Get-Date).AddSeconds(5)
do {
    $sameProcess = Get-Process -ErrorAction Stop |
        Where-Object { $_.Id -eq $serverPid -and $_.StartTime -eq $serverStart
          }
    $sameListener = Get-NetTCPConnection -State Listen -ErrorAction Stop |
        Where-Object { $_.LocalPort -eq 8765 -and $_.OwningProcess -eq
          $serverPid }
    if (-not $sameProcess -and -not $sameListener) { break }
    Start-Sleep -Milliseconds 200
} while ((Get-Date) -lt $deadline)

if ($sameProcess -or $sameListener) {
    throw "This run has not been confirmed stopped."
}
"This run is stopped."
```

`do ... while` 至少检查一次；`$_` 表示当前经管道传来的对象；`-eq` 比较相等。这里用启动时间防止把以后复用同一 PID 的进程误认成原实例。期限限制了重查次数，不是先睡五秒就宣称成功；每次操作系统查询本身也需要时间。如果查询报告权限等真实错误，先解决观测失败，不能用“没拿到对象”冒充“已停止”。

如果仍未停止，先确认终端一确实是运行本次服务的终端，再检查它显示的状态。在确认过身份、且 `$serverProcess` 仍指向本次进程时，可以在终端二使用 `Stop-Process -InputObject $serverProcess` 请求结束，再重新执行上面的复查。强制结束不能保证程序执行自己的清理代码，所以日常优先使用程序提供的停止方式；不要改成按名称结束全部 Python。

进程消失以后，它持有的 listener 应释放。端口随后可能被别的进程使用，那是另一份所有权证据；本次收尾只结束自己启动的实例。不要因为端口又出现未知占用者，就继续结束别人。

如果希望提示符先回来

两个终端已经可以完成刚才的观察。只有想让同一终端继续做其他事时，才需要把工作放到后台。先回到终端一，仍使用最初的练习目录和 `$pythonExe`。保存一份很短的 `wait_piece.py`：

```python
import os
import time

print(f"python_pid={os.getpid()}", flush=True)
print(f"parent_pid={os.getppid()}", flush=True)
print("working", flush=True)
time.sleep(8)
print("done", flush=True)
```

`os.getpid()` 是这个 Python 进程自己的 PID，`os.getppid()` 是它的父进程 PID。`time.sleep(8)` 只是让这项有限工作等八秒，便于观察；它没有监听网络，也不代表在做八秒计算。`flush=True` 让这些打印及时离开 Python 缓冲。

用 PowerShell 的 background job 运行它：

```powershell
$workerPath = (Resolve-Path -LiteralPath .\wait_piece.py).Path
$job = Start-Job -ScriptBlock {
    param($exe, $script)
    "job_shell_pid=$PID"
    & $exe -u $script
    "python_exit=$LASTEXITCODE"
} -ArgumentList $pythonExe, $workerPath
Get-Job -Id $job.Id
Receive-Job -Job $job -Keep
```

提示符立即回来。`Start-Job` 返回一项由当前 PowerShell 会话管理的工作记录，保存在 `$job`；`Get-Job` 查看状态，`Receive-Job` 取得已收到的输出，`-Keep` 保留输出供后面再读。刚启动时可能还没有输出；这表示还没收到，并不能直接判定 Python 没启动。

`$job.Id` 是 PowerShell 给 job 编的号码，不能当成操作系统 PID。job 的脚本块在另一个 PowerShell 进程里执行，其中 `$PID` 是那份 PowerShell 的 PID；它再启动 Python。Python 打印的 `python_pid` 才是执行 `wait_piece.py` 的 PID，`parent_pid` 应与 `job_shell_pid` 对应：

```
当前 PowerShell：管理 job.Id
 → job 的 PowerShell：job_shell_pid
 → Python：python_pid；parent_pid 指向 job 的 PowerShell
```

等这项短工作结束并收尾：

```powershell
try {
    $completed = Wait-Job -Job $job -Timeout 15
    if ($null -eq $completed) { throw "Worker did not finish in time." }
    Receive-Job -Job $job -Keep
}
finally {
    if ($job.State -eq "Running") { Stop-Job -Job $job }
    Remove-Job -Job $job
}
```

正常输出包含 `working`、`done` 和 `python_exit=0`，再加上三个实际 PID 数字。`Wait-Job` 等的是 job；`python_exit` 则在运行 Python 的那份 shell 里立即读取。父终端的 `$LASTEXITCODE` 不会因此自动成为 job 内 Python 的退出码。即使内部原生程序返回非零，只要脚本块正常执行完，job 也可能显示 `Completed`；要读它收回来的具体结果。

`Stop-Job` 请求停止工作，`Remove-Job` 移除管理记录，两个动作不同。本例正常到达 `done`，说明 Python 自己走到了末尾；若提前停 job，不能把“管理记录已删除”当作所有子进程和资源都已消失。对刚才的长期服务，应继续用实际 Python PID 与 listener 做收尾复查。当前会话结束会影响它管理的 job，不能把这种练习方式当作持续服务的保证。

独立启动以后，谁负责收尾

在 Windows 本地终端，`Start-Process` 可以独立启动进程；调用者不用一直等它，进程通常也能在启动它的 PowerShell 退出后继续。它因此需要自己的 PID、输出去向和停止责任。这个行为不能直接外推到远程会话、其他操作系统或各种终端宿主。

仍用同一份八秒后结束的程序。下面的引号是给可能含空格的脚本完整路径准备的；`-ArgumentList` 最终形成命令行，因此路径引号需要保留下来：

```powershell
$workerPath = (Resolve-Path -LiteralPath .\wait_piece.py).Path
$workerArgs = '-u "' + $workerPath + '"'
$process = Start-Process -FilePath $pythonExe -ArgumentList $workerArgs
  -WindowStyle Hidden -PassThru -RedirectStandardOutput .\worker-out.txt
  -RedirectStandardError .\worker-err.txt
$null = $process.Handle
$process.Id
```

`-PassThru` 返回这次启动的进程对象，`$process.Id` 就是本次 Python PID。`-WindowStyle Hidden` 不另弹出窗口；因此 stdout 与 stderr 分别写入练习日志文件，观察入口仍明确存在。它没有自动重启、健康检查或开机启动能力。

启动后立即读取 `.Handle`，是为了让进程对象保留后续查询退出码所需的操作系统句柄；`$null =` 只是不把句柄值打印出来。Windows PowerShell 5.1 中，这种带重定向的启动方式可能在等待结束后仍给出空的退出码。尽早保留句柄能处理本例遇到的兼容问题；如果进程过快结束，查询仍可能失败，不能把缺失的退出码当成 `0`。

及时完成这次练习的等待和收尾：

```powershell
try {
    if (-not $process.WaitForExit(15000)) {
        throw "Worker did not finish in time."
    }
    $process.Refresh()
    $workerExit = $process.ExitCode
    if ($null -eq $workerExit) { throw "Worker exit code is unavailable." }
    "python_exit=" + $workerExit
    Get-Content -LiteralPath .\worker-out.txt
    Get-Content -LiteralPath .\worker-err.txt
}
finally {
    if (-not $process.HasExited) {
        Stop-Process -InputObject $process
        if (-not $process.WaitForExit(5000)) {
            throw "Worker is still running."
        }
    }
}
```

`.WaitForExit(15000)` 最多等 15000 毫秒，也就是十五秒，返回是否已退出。正常结果是退出码 `0`，输出文件中有 `working` 和 `done`，stderr 为空。这里读取 `$process.ExitCode`，因为 `Start-Process` 没有把这份独立 Python 的结果放进当前 shell 的 `$LASTEXITCODE`。

`finally` 仍只处理本次返回的进程对象。若中途需要结束，使用已确认身份的这个对象，再检查退出；不要只保存一个数字很久以后盲目停止，因为 PID 会复用。本例用有限工作先看清生命周期，真正长期服务还需要明确谁启动、谁查看日志、谁在失败后恢复，这些责任不会因为“放到后台”自动产生。

遇到权限拒绝时，先定位是哪次操作

程序能启动，不表示它能访问任意文件。操作系统还会在读文件、写文件等具体操作上检查当前身份的权限。比如服务已监听，但读取某个文件被拒绝，最早失败点在文件访问；它与启动命令找不到解释器是不同问题。

在原练习目录可以只读查看：

```powershell
whoami
Get-Acl -LiteralPath .\status-site\status.json | Format-List Owner,
  AccessToString
```

`whoami` 显示当前身份。Windows 的 ACL（access control list，访问控制列表）记录哪些身份或组对目标有哪些允许或拒绝规则。`Owner` 是文件所有者，`AccessToString` 展示权限规则；它们是调查线索，不是“看见自己名字就证明所有操作可做”。实际访问还会受组身份、继承和拒绝规则等因素影响。

面对 `Access is denied` 或 Python 的 `PermissionError`，先记下实际身份、完整目标路径、尝试的是读还是写，以及错误发生在哪一步。不要先把终端升级成管理员，也不要为了练习去改系统目录或整棵项目树的权限。

Unix 常见的 owner/group/other 和读、写、执行权限位使用另一套表示方法，不能把 `chmod 755` 机械翻译成 Windows ACL。以后在 Linux 环境遇到权限问题，再查那里的证据与命令；共同需要回答的是“哪个身份，对哪个目标，进行哪项操作”。

学到这里，可以用一个实际判断检验这条关系：本次 Python PID 仍存在，请求却失败了，下一步先取得本次 PID 对应的 listener；有了正确 listener，再区分请求路径、状态和内容。若一开始就是 Python 没被启动，就回到启动命令。没有理由在这些不同断点上都先修改业务代码。

本章需要逐渐用顺的是：运行后知道谁在等待；取证时知道是在看输出、退出码、进程还是一次请求；结束时只处理自己的实例并复查。完整查询参数与等待代码可以查回，但应能解释每份证据支持到哪一步。

查用来源

Python 的示例使用 3.13 系列标准库接口；PowerShell 命令面向 Windows PowerShell 5.1 和 PowerShell 7。以下为对应的官方参考；命令参数及版本兼容问题应按实际运行版本查询。

- Python 3.13：标准流与进程退出、环境变量与进程身份、http.server 的目录、绑定与安全限制。
- PowerShell：管道、重定向、环境变量。
- PowerShell：job 生命周期、Start-Process、Get-NetTCPConnection、Get-Acl。

A11 | 怎样保存和找回一份确定的代码?

你把程序改好了，编辑器也已经保存。第二天又改了几处，结果不对了。怎样看清这次到底动了什么，并找回自己确认过的那份内容?

这一章只用一个输出问候语的小文件。先让一次修改成为可以找回的记录，再看同一文件为什么可能同时有几份不同内容。理解这些内容的来源和去向以后，恢复、分支和分享才有明确对象。

这次只改哪一个对象?

我们的完整程序叫 `greet.py`，最初只有一行:

```python
print("Hello")
```

它运行后显示 `Hello`。这次只把问候语改为 `Hello, learner`，然后看清"当前文件变了，原来的内容怎样仍能找回"。先在自己的目录里建立这份文件和初始记录，接下来的比较就能对应到电脑上的同一个对象。

先准备自己的练习文件和初始记录

用资源管理器在自己的练习位置新建一个空文件夹，例如 `A11-greeting`；不要选择已有项目、资料发布目录或现有 Git 仓库里的子目录。目标同名目录已经存在，就另取新名字，不合并或覆盖。进入后先确认它是空的，再用纯文本编辑器把上面这一行保存为 UTF-8 的 `greet.py`，确认不是 `greet.py.txt`。保持这份文件打开，接下来只改它。

打开 PowerShell，输入刚才这个新文件夹的完整路径，再核对当前位置和文件清单:

```powershell
Set-Location -LiteralPath (Read-Host '刚新建的 A11 练习文件夹完整路径')
Get-Location
Get-ChildItem -Force -Name
```

位置必须是自己刚建的目录，清单应只有 `greet.py`；`-Force` 也会显示隐藏项目。切换失败、看到其他文件或看到 `.git`，都先停下来查清，不在这里初始化。再运行 `python .\greet.py`，看到 `Hello`，就把编辑器里的文件与终端结果对上了。暂时没有工具时可以先观察后面的示意结果，但它们还不是本人运行的证据。

接下来只检查 Git 是否可用:

```powershell
git --version
```

应显示 `git version` 和版本号；本章需要 2.28 或更新版本。命令找不到或版本不满足时，先处理工具入口，不继续下面的命令。目录里没有 `.git`，还不能排除它属于上层仓库，因此再问 Git 当前归属：

```
POWERSHELL
git rev-parse --show-toplevel
```

这个新目录应得到含 `not a git repository` 的提示，表示尚不属于仓库；这是本步要确认的结果。如果返回了某个仓库路径，或出现其他错误，就停止并换到确实独立的新空目录。不能在别人的仓库范围里继续本练习。

确认以后，才让 Git 管理当前目录；`main` 是初始分支名：

```
POWERSHELL
git init -b main
```

应显示初始化的仓库就在当前练习目录。失败就停在这里。下一步给这个练习仓库设置提交署名，`--local` 限定只写本仓库，不改全局身份：

```
POWERSHELL
git config --local user.name 'FDE Learner'
git config --local user.email 'learner@example.invalid'
```

两条都成功后，选中刚核对的这一份文件，准备记入历史；不使用会选入其他文件的 `git add .`：

```
POWERSHELL
git add -- .\greet.py
```

`add` 成功后，再保存这份初始记录；`-m` 后面的文字是给人看的说明：

```
POWERSHELL
git commit -m 'initial greeting'
git status --short
```

提交应成功，随后短状态没有输出。这表示当前这份文件已经与初始记录一致。任何一步失败都先停下，不能把“命令输过了”当成记录已建立。下面只改一行，`add`、`commit` 与普通保存的区别会由实际差异逐步显现。章末保留了自动建立隔离目录的完整配方，初次手动准备已成功时不需要再执行它。

文件已经保存，为什么还要再保存一次？

保持刚才的练习目录和编辑器文件不变，现在把 `greet.py` 改成：

```PYTHON
print("Hello, learner")
```

保存后再运行 `python .\greet.py`，应显示 `Hello, learner`。这证明这份新文件的行为符合当前要求。但原来的 `print("Hello")` 已经被编辑器的保存替换了；普通文件没有因此自动建立一条“从哪一份改到哪一份”的历史。

Git 刚才已经记录了我们明确选择的文件快照。现在仍在同一 PowerShell 目录中比较：初始记录保存的是 `print("Hello")`，当前文件已经改为 `print("Hello, learner")`。

```POWERSHELL
git status --short
git diff -- .\greet.py
```

`status` 先报告哪些路径发生变化。现在会看到：

```
 M greet.py
```

`diff` 再展示具体内容差异，关键两行是：

```DIFF
-print("Hello")
+print("Hello, learner")
```

这里的 `-`、`+` 是比较结果里的标记：旧内容少了这一行，新内容多了这一行。它们不属于 Python 程序。`--` 表示后面的参数是路径，本次只比较 `greet.py`。

编辑器正在修改的这份磁盘文件，位于 **working tree（工作区）**。现在它变了，Git 的已提交历史还没有变。

先选中内容，再把它记入历史

为了记录这次问候语修改，先把这一文件当前的内容交给 Git 的待提交位置：

```POWERSHELL
git add -- .\greet.py
git diff --staged -- .\greet.py
```

这个位置叫 stage，也叫 index，中文常称暂存区。可以把它读成“下一次提交准备保存的内容”。`add` 把执行当时选中的文件内容放进去；`diff --staged` 比较暂存内容与当前提交，因此仍能看到 `Hello` 变为 `Hello, learner`。

确认差异正确以后，运行当前程序，再建立新提交：

```powershell
python .\greet.py
git commit -m "greet the learner"
git status --short
git log --oneline -2
```

程序应先显示 `Hello, learner`。`commit` 根据暂存区建立一份提交记录，`-m` 后面的文字说明本次改变的意图。最后 `status --short` 没有输出，表示这份受控练习里没有待提交或未跟踪的变化。`log` 能列出新的提交和初始提交。

日志每行前面会有一段提交 ID。它标识具体提交内容及相关信息，不是要求记忆的编号。下面用 C0、C1 代称真实 ID：

```
C0：greet.py 是 print("Hello")
 ↓
C1：greet.py 是 print("Hello, learner")
```

一次提交保存的是项目受管理文件的一份快照，而不只是一段 diff。没有变化的文件也属于这份快照；`diff` 是把两份内容拿来比较后得到的结果。

现在可以解释三种“保存”：编辑器保存改变工作区，`git add` 选择待提交内容，`git commit` 把暂存内容记入本地历史。它们依次交接，却不会互相自动代办。

add 以后继续编辑，下一次提交会得到哪一份？

接着从 C1 开始。先把文件改为 `print("Hello, team")` 并执行 `git add`，然后又在编辑器中改为 `print("Hello, everyone")`，保存，但这次没有再 add。

此刻同一个路径对应三份内容：

位置	greet.py 中的文字	它由什么动作得到
当前提交 C1	`Hello, learner`	上一次 commit
暂存区	`Hello, team`	本次 add
工作区	`Hello, everyone`	add 之后的编辑器保存

因此，下面两个命令会得到不同差异：

```powershell
git diff -- .\greet.py
git diff --staged -- .\greet.py
git status --short
```

普通 `diff` 看见 `team → everyone`，因为它比较暂存区与工作区。带 `--staged` 的比较看见 `learner → team`，因为它比较当前提交与暂存区。短状态会是：

```
MM greet.py
```

左列 `M` 表示暂存区相对当前提交已修改，右列 `M` 表示工作区相对暂存区又有修改。这两个位置是相邻的两次比较，并不是两个文件。

这时直接执行普通 `git commit`，新提交会保存 `Hello, team`。编辑器里的 `Hello, everyone` 仍留在工作区，成为相对新提交的未暂存修改。若真正想保存 `everyone`，应重新比较，再对这个文件 add，检查 staged diff 后提交。

这个区别也影响测试。`python .\greet.py` 此刻运行的是工作区里的 `everyone`，测试通常同样读取工作区文件。测试通过不能自动证明暂存区里的 `team` 也正确。提交前，应看清测试运行的文件是否和准备提交的内容一致；普通 diff 为空能帮助确认受跟踪文件没有这类差异，新文件仍需看 status。

例如下面的状态表示，普通 commit 只会记录 `greet.py` 的暂存变化：

```
M  greet.py
 M notes.txt
?? draft.txt
```

`notes.txt` 只在工作区改变；`draft.txt` 的 `??` 表示尚未被 Git 跟踪。文件保存到了磁盘，不会因此自动进入下一次提交。

只是不想提交，和想丢掉修改，是两个动作

回到刚才的三份内容：当前提交为 `learner`，暂存区为 `team`，工作区为 `everyone`。假设现在暂时不提交，但要保留最后写的 `everyone`。

```powershell
git restore --staged -- .\greet.py
git status --short
git diff -- .\greet.py
git diff --staged -- .\greet.py
```

`--staged` 指定改变暂存区。没有另给来源时，Git 从当前提交取回 `learner` 放进暂存区，工作区中的 `everyone` 没动。状态变成右列的 `M`；普通 diff 显示 `learner → everyone`，staged diff 为空。

如果接下来确认连 `everyone` 也不要了，先看刚才的 diff，确定它确实只含要放弃的内容，再执行：

```powershell
git restore -- .\greet.py
git status --short
```

这次没有 `--staged`，目标是工作区，默认来源是暂存区。于是工作区得到 `learner`，三份内容重新一致。被放弃的未提交文字不会因为执行 restore 而生成一个备份提交。

顺序很关键。若在最初 `MM` 的状态直接执行 `git restore -- .\greet.py`，工作区会得到暂存区的 `team`，不会跨过它直接回到 C1 的 `learner`。想恢复前先说清“改变哪个位置，从哪里取内容”，就不必把 restore 当成含糊的撤销键。

实际操作中应先取得 `status` 与对应 diff，只给已经核对的文件路径。不要把本来只涉及一个文件的目标扩大成整棵工作区。

如果错误已经提交，怎样保留纠正的经过？

假设在干净的 C1 之后，把问候语改成了 `Hello, team` 并提交为 C2。后来发现该改动不需要，但还希望历史保留“改过什么、后来怎样纠正”。

先用日志找到目标，再看它的实际差异：

```powershell
git log --oneline -3
git show HEAD -- .\greet.py
```

`HEAD` 在这里指当前检出的提交，正好是 C2。它不是固定指代“错误提交”；如果后来又提交了内容，应重新从日志确定真实目标。

已经核对 C2，且工作区与暂存区都干净时，可以反向应用它的改动：

```powershell
git revert --no-edit HEAD
git log --oneline -3
python .\greet.py
```

程序重新显示 `Hello, learner`，历史却多了一项：

```
C1：learner
 ↓
C2：team
 ↓
C3：撤销 C2 的改变，内容回到 learner
```

`revert` 新建反向提交；`--no-edit` 使用 Git 生成的说明，避免练习突然进入文字编辑器。它不会删除 C2。若后面的修改与反向改变冲突，Git 也可能停下来要求解决内容，不能期待所有旧提交都能无冲突撤销。

恢复一个未提交文件，和撤销一项已提交改变，处理的是不同事实。前者先比较、再对明确路径 restore；后者先确定提交及其影响，再用 revert 保留纠正经过。

两个人从同一份内容继续改，历史怎样分开？

现在有一份共同起点 C。希望试改问候语，又能保留原来的开发方向，可以建立名为 `fix` 的分支：

```powershell
git switch -c fix
```

分支 branch 是指向某个提交的名字。刚建立时，原来的 `main` 与新的 `fix` 指向同一个 C；没有因此复制出两套永久独立的文件。此时当前选择的是 `fix`，在这里提交新改变，移动的是 `fix`。

假设 `fix` 把同一行改为 `Hello, team` 并提交，回到 `main` 后又把它改为 `Hello, everyone` 并提交，关系就成为：

```
     Cfix：team     ← fix
    /
C ----+
    \
     Cmain：everyone ← main，当前选中
```

切换用 `git switch main`，Git 会检出那个分支对应的文件。这里的例子每次切换前都已提交且状态干净；有未提交改动时应先查看，不能把切换分支当成自动清空或保存修改的动作。

在 `main` 上执行 `git merge fix`，意思是把 `fix` 自共同起点以来的改动整合进当前分支。若修改互不冲突，Git 可能自动合并；本例两边改了同一行，Git 无法决定该问候 team 还是 everyone，会留下 conflict（冲突）：

```
<<<<<<< HEAD
print("Hello, everyone")
=======
print("Hello, team")
>>>>>>> fix
```

这些标记帮助找到两边内容，不能作为最终 Python 代码保留。当前需求如果确定为问候所有人并加感叹号，就把这一块编辑成唯一的最终结果：

```python
print("Hello, everyone!")
```

运行得到预期文字后，`git add -- .\greet.py` 表示已选择解决后的内容，再 commit 完成合并。真正需要判断的是两边改动的用途；不能只因为一边叫“自己的”就总选它。

本地提交以后，另一台机器为什么还没有变化？

目前所有历史都在本地 repository（仓库）中。练习目录的 `.git` 保存 Git 管理历史和状态需要的信息。远端 remote 是另一处仓库，并不会自动镜像编辑器里的内容。

动作	这次交接什么	对当前工作区的影响
`fetch`	取得远端提交等对象，更新本地记录的远端分支位置	不自动把远端变化合进当前分支
`pull`	先 fetch，再把取得的变化整合进当前分支	可能改变文件，也可能因冲突停止；整合方式受选项和配置影响
`push`	请求远端接收本地提交并更新指定分支等引用	未提交的工作区文字不会因此上传

因此，只想先查看远端进展时，fetch 对应“取得信息”；需要整合时再明确 pull 的策略。GitHub 是可以托管 Git 仓库的服务，Git 的本地比较和提交不要求有 GitHub 账号。

本章练习不配置远端。以后进入真实项目，需要先看项目的分支与评审规则，再决定向哪个分支发送哪些提交。

准备分享时，哪些文件不该进入这份历史？

Python 项目可能同时有源码、虚拟环境、缓存、生成报告和真实输入。它们都在磁盘上，不代表都适合交给别人。

例如 `.gitignore` 可以写：

```
.venv/
__pycache__/
output/
.env
```

它告诉 Git：对尚未跟踪的匹配路径，普通 add 不主动加入；已经被跟踪的文件仍受跟踪。`.venv` 是本机安装状态，缓存和 output 通常能重新生成；`.env` 常用于放本机配置。这个文件名本身没有保密能力。

分享前，除了看 staged diff，还要用 `git ls-files` 查看受跟踪文件清单。真实密码、token、API key、客户数据和只适用于本机的路径，都要按实际内容判断；重命名成 `config.example` 不会把里面的真实秘密变为占位值。

若秘密已经提交，补写 `.gitignore` 不能让它从历史消失。应停止继续分享并撤销或轮换秘密；是否清理历史，要结合它已经到达的远端和协作者范围处理。这类协调不属于本章临时文件的普通 restore。

需要逐渐用顺的是“先比较、再选择、再确认快照”，以及恢复前辨认来源和目标。`status`、两份 `diff` 和精确路径是稳定的查回入口；完整分支、合并和远端参数可以在真实任务中按文档查，不需要先背一张命令表。

需要补做时，从当前状态接回

章首已经建立了本次练习目录和初始记录；跟着前文做过的部分不必重新初始化或重复提交。若回到终端后忘了当前位置，先重新看 `Get-Location` 和 `git rev-parse --show-toplevel`，应仍是自己记录的练习目录。再用 `git status --short` 和对应 diff 确认停在哪一份内容，不靠文件名猜进度。

如果还只完成了章首准备，就从 `Hello` 改为 `Hello, learner` 那一段继续。保存、add、运行和 commit 各步应依次看见右列 `M`、左列 `M` 、输出 `Hello, learner`，最后状态为空。已经完成这一轮，就直接保留它作为后面比较和恢复的起点。

尚未练过三份内容的区别时，从干净的 learner 记录开始：改为 team 后 add，再改为 everyone 只保存，核对 `MM` 和两个 diff。按前文先 `restore --staged`、再看 diff、最后精确 restore，才会回到 learner。每次恢复前仍要确认当前差异只包含这次愿意放弃的文字。

提交后的纠正和分支冲突按前文对应情境补做即可。已经存在 `fix` 分支、已经完成合并或还有未解释修改时，不要重新照跑建立分支的动作；先查看当前状态与 `git log --oneline --graph --all`，再回到尚未完成的那段。练习的证据是自己看见的内容变化，不是把同一组命令再运行一遍。

这些修改只用于本次新建的练习文件。练习完成后保留实际目录的绝对路径和需要的证据；这里没有自动删除目录的命令，也不需要进入任何已有个人仓库练习恢复。

在独立目录中完成这次保存与恢复

手动准备和本章练习已经顺利完成，就跳过这一节。下面是需要重新取得一份干净例子时的完整准备配方，会另外建立全新的 TEMP 子目录；它不会接着修改之前的练习。临时目录只用于这次隔离实验，需长期保留的作品应放在自己的稳定实验位置。

命令适用于 Windows PowerShell 5.1 与 PowerShell 7，保留版本和目标路径检查。创建、提交或恢复失败时，停在该命令，核对输出，不跳过失败继续后面的修改。先检查 Git 和 Python：

POWERSHELL

```
$gitVersionText = git --version
if ($LASTEXITCODE -ne 0 -or
    $gitVersionText -notmatch '^git version (?<number>\d+(?:\.\d+){1,3})') {
    throw 'Cannot determine Git version.'
}
if ([version]$Matches.number -lt [version]'2.28') {
    throw 'This exercise requires Git 2.28 or newer.'
}
$gitVersionText
python --version
```

以下命令只创建自己的练习目录和文件；路径若已存在就停止，不覆盖它。`$a11Lab` 是这个新目录的完整路径，配方中的后续操作都在这里进行。

```powershell
$a11Temp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
$a11Prefix = $a11Temp.TrimEnd('\') + '\'
$a11Lab = [IO.Path]::GetFullPath(
    (Join-Path $a11Temp ('fde-a11-' + [guid]::NewGuid().ToString('N')))
)
if (-not $a11Lab.StartsWith($a11Prefix, [StringComparison]::OrdinalIgnoreCase)
  -or
    (Test-Path -LiteralPath $a11Lab)) {
    throw 'Expected a new directory inside TEMP.'
}
New-Item -ItemType Directory -Path $a11Lab -ErrorAction Stop | Out-Null
Set-Location -LiteralPath $a11Lab
Set-Content -LiteralPath .\greet.py -Value 'print("Hello")' -Encoding ASCII
git init -b main
if ($LASTEXITCODE -ne 0) { throw 'Git init failed.' }
git config --local user.name 'FDE Learner'
git config --local user.email 'learner@example.invalid'
git add -- .\greet.py
git commit -m 'initial greeting'
if ($LASTEXITCODE -ne 0) { throw 'Initial commit failed.' }
git rev-parse --show-toplevel
git status --short
$a11Lab
```

ASCII 写入这份纯英文文件，仓库署名只写本地配置。核对根目录是打印出的 `$a11Lab`、短状态为空，再用编辑器打开该目录的 `greet.py`，从章首“文件已经保存，为什么还要再保存一次？”的小修改接入；不要把这份新目录的状态与先前的练习混用。

Git 现在能回答“保存的是哪份内容”。下一章还要补一个实际条件：这份内容离开当前已经装好的 Python 环境以后，怎样依据留下的说明重新运行。

官方来源

本章的文件和问候语是原创例子。Git 关系对应 Missing Semester 2026：版本控制。需要查回具体行为时，可以从当前问题进入官方文档：

- 选择和比较内容：add、diff、status。
- 恢复与合并：restore、revert、branch、merge。
- 远端交接与排除规则：fetch、pull、push、gitignore。

命令文档是持续更新的入口，实际运行前可用 `git --version` 和 `git help <命令>` 核对本机适用写法。

A12 | 离开当前环境还能重新运行吗？

本章只证明一件事：另一个人只拿到确定版本、依赖记录和输入，就能在没有旧 .venv、旧报告和口头说明的目录里重新得到同一结果。

先看见这三个对象，再决定要不要进入重建：输入文件是程序处理的材料，程序文件决定处理规则，报告是这次运行留下的结果。它们先在当前实验副本中对上，后面才有意义讨论另一台电脑怎样按相同版本和运行条件重建。

例如，当前副本可能留下这样的报告片段（这里只是帮助识别字段的示意，不是本机已经运行得到的证据）：

```JSON
{
  "summary": {"accepted_records": 2, "rejected_records": 0},
  "issues": []
}
```

这段内容只说明某一次运行写出了一个报告；它还没有告诉我们使用了哪份源码、哪个 Python、哪份输入，也没有证明陌生环境能重建。先把“我看到了什么”与“我因此能断定什么”分开，后面的每一步才不会把一份输出夸大成完整交付证据。

“能不能运行”至少可能在问四件不同的事：文件是否存在、是否来自指定版本、依赖能否在新环境建立、真实入口是否得到约定结果。文件夹里有 .py、测试通过或退出码为 0，各自只能支持其中一部分，不能互相冒充。

先做一个选择：

这次要证明什么	先看什么
只是确认这份输入现在能得到报告	输入、程序、报告和退出码
要交给别人，让对方在没有旧环境时重建	再加确定版本、依赖记录、新环境、测试和真实入口

如果只是前一项，不必先建立 Git worktree；如果是后一项，后面的重建路线才是必要证据。两种目标都合理，但不能用较轻的证据冒充较重的承诺。

第一次学习时可以只选择 **A（当前正确性）** 或 **B（陌生环境重建）**。如果还没有交付对象，也不知道未来是否要交给别人，先选 A；等当前输入、报告和退出码都能解释后，再决定是否需要 B。这个选择不是考试，而是让接下来的动作与要作出的承诺相匹配。

所以本章不是先把 .venv、pytest、Git、worktree 和标签全部跑一遍，而是先决定当前交付承诺需要哪份证据。只想证明一次输入得到报告，先看真实报告；要证明陌生环境可以重建，才把精确内容、依赖记录、新环境、测试和真实入口连起来。

你把源码保存成了一份 Git 提交，程序在当前电脑上也运行成功。把这些文件交出去以后，对方还需要哪个 Python、哪些依赖、什么输入，以及哪条启动命令？这些条件如果只存在于旧电脑和作者记忆里，源码快照还不足以让程序重新工作。

这一章使用随书的离线工单整理器，先说明一份目录应留下什么，再从一个精确提交建立新的工作树和 Python 环境，重新运行测试与真实入口。这是在同一台电脑上排除旧项目环境的影响；真正搬到另一台电脑，还要在那里按同样方法取得证据。

先让陌生人知道，这些文件要做成什么

工单整理器读取一份 CSV 或 JSON，验证每条记录，把有效工单、拒绝原因和统计写成 JSON 报告。它只处理本地文件，没有账号、网络服务或必需环境变量。主程序使用 Python 标准库，测试另用 pytest。

这一章不要求先读懂它的全部业务源码。这里把它当作有明确输入和输出的程序；下一章会逐函数解释内部处理。需要交给重建者的文件已经随书提供：

```
ticket-organizer/
├── ticket_organizer.py      程序入口与业务函数
├── test_ticket_organizer.py  自动检查
├── data/
│   ├── tickets.json          两条有效记录
│   └── tickets.csv           三条有效、四条拒绝记录
├── requirements.txt          测试需要安装的依赖
├── .gitignore                本机环境与生成结果的排除规则
├── VERSION                   当前版本文字
├── RELEASE_NOTES.md          能力、运行条件、限制与恢复方法
└── README.md                 操作入口及预期结果
```

有效 JSON 中的第一条记录如下：

```json
{
  "id": "T-201",
  "subject": "VPN cannot connect",
  "priority": 1,
  "status": "open",
  "owner": "Wei"
}
```

文件顶层是一个列表，另有一条 `T-202`：优先级为 3，状态为 pending，owner 是空文字。程序接受两条记录，把空 owner 统一为 `unassigned`，输出统计应是有效 2、拒绝 0。

假设已经进入自己的实验副本根目录，真实入口是：

```powershell
python .\ticket_organizer.py .\data\tickets.json .\output\report.json
$LASTEXITCODE
```

第一个路径参数是输入，第二个是要生成的报告。成功应同时留下三项事实：屏幕显示有效 2、拒绝 0；退出码为 0；指定路径出现可解析的新报告，其中计数也是 2、0。只看见文件名或一句正常文字，还没有核对整条交接。

这里的路径都相对于实验副本根目录。`01_配套练习` 中的随书项目只用来读取和复制；实际运行要先完成章后的独立副本准备，不能在随书目录里生成环境、报告或 Git 状态。

先停在这里做一个最小判断：查看刚生成的 `output/report.json` 后，打开实验副本中的**输入文件** `data/tickets.json`，把其中一条记录的 `priority` 改成 `9` 并保存；不要修改报告。先预测“有效数、拒绝数和退出码”会怎样，再运行一次并查看新报告。观察完把输入恢复，重新运行，确认回到有效 2、拒绝 0、退出码 0，再继续后面的正常重建路线。你观察到的是**当前程序对当前输入的正确性证据**；它还没有证明别人能在没有旧环境的目录里重建。后面的新 `.venv` 和 Git 路线只为第二个承诺服务。

> **第一次学到这里先停。** 这一段只要求你区分两种承诺：一是“这份输入现在能得到正确报告”，二是“陌生人没有我的旧环境也能重建”。前者看输入、报告和退出码；后者才需要继续绑定提交、依赖、新环境和真实入口。不要为了完成第二种承诺，提前背下后面的整套命令。

新环境根据什么知道该安装什么？

这个版本声明 Python 3.10 或更高版本。`requirements.txt` 中记录：

```
pytest==9.0.3  # Test dependency only; runtime code uses the standard library.
```

`pytest` 是包名，`==9.0.3` 指定这条记录要求的版本。后面的注释说明它用于测试。该文件是安装工具的输入，文件存在本身不会把 pytest 安装到任何环境。

在一个没有旧 `.venv` 的目录中，下面三个动作有不同结果：

```powershell
python -m venv .venv
& .\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
& .\.venv\Scripts\python.exe -m pytest -q
```

第一行由当前选中的 Python 创建 `.venv`。这个目录保存项目环境中的解释器入口和安装位置。第二行明确使用新环境的 `python.exe` 启动 pip，按 requirements 安装测试依赖。第三行仍用同一解释器启动 pytest，才取得测试结果。

这里不必激活环境。PowerShell 的 `&` 表示调用后面的可执行文件；完整写出 `.venv\Scripts\python.exe`，便明确指定了安装和测试的对象。激活主要是调整当前 shell 的命令查找环境，让简写 `python` 更容易指向它，不是环境开始存在的条件。

`.venv` 是一份已安装状态，requirements 是重新建立这种状态的记录，两者不能互相代替。比如旧环境碰巧装有 HTTPX，后来某程序加入 `import httpx`，却没有更新依赖记录：旧环境还能运行；新环境按记录安装后，导入最先出现 `ModuleNotFoundError`。这个反例不要求工单整理器加入 HTTPX，只用来暴露“旧机器替遗漏的说明补了条件”。

应根据实际功能补全依赖记录，再用新环境验证。直接搬运旧 `.venv` 会继续带着旧路径和安装状态，无法证明记录完整。依赖安装还需要可用的软件源或本地包缓存；本程序运行可以离线，不表示首次安装测试工具必然不需要包来源。

怎样确定测试的是刚才保存的那份内容?

前面先在源目录完成测试和有效 JSON，再用 `git rev-parse HEAD` 取得这次实际测试对应的提交；下面把这个值暂称为 `$testedCommit`。不要先假定一个神秘的版本编号：真正重要的是测试时的内容身份。

```powershell
git status --porcelain
git rev-parse HEAD
```

第一条用适合程序读取的格式报告变化；在本次受控副本中应没有输出。第二条取得当前提交的完整 ID。只有测试和入口完成后，受跟踪内容仍与提交一致，才把这个 ID 记录为本次测试对应的提交。

为什么不能只记“测试通过时文件夹叫 ticket-organizer”？因为目录中的源码还可以继续编辑。即使名称不变，刚才通过的内容和现在要交出去的内容也可能不同。A11 中 working tree 与暂存区的区别，在这里进一步要求运行证据绑定具体内容。

Git 的干净状态也有范围：被忽略的 `.venv` 和生成报告不会出现在普通 status 里。空状态帮助证明受管理内容一致，不能证明整个磁盘没有其他文件。接下来还要取得一份确实不带旧安装状态的新工作树。

让 Git 从指定提交取出文件

源目录的绝对路径暂记为 `$sourceRepoPath`，刚记录的完整提交 ID 暂记为 `$testedCommit`，一个尚不存在的新目录路径叫 `$rebuildPath`。它们都是 PowerShell 变量，章后的配方会实际取得这些值。

```powershell
git -C $sourceRepoPath worktree add --detach $rebuildPath $testedCommit
git -C $rebuildPath rev-parse HEAD
git -C $rebuildPath status --porcelain
```

`-C` 让 Git 在指定目录执行操作。`worktree add` 为同一仓库建立另一份可工作的文件目录；最后给出的 `$testedCommit` 决定取哪份内容。`--detach` 表示这次直接检出指定提交，当前没有附着在某个分支上，适合固定内容的运行验证。

新目录的 HEAD 应与 `$testedCommit` 完全相同，初始状态应为空。没有被提交的旧 `.venv`、报告和工作区新修改不会因此被复制进去。新目录与源目录共享 Git 历史管理信息，却有各自的工作区；它不是一份可以任意搬走的独立仓库备份。

然后在新目录创建自己的 `.venv`，按同一份依赖记录安装，用这一个新解释器运行测试与入口：

```
$testedCommit 所指提交里的源码、依赖记录、tests 和样例
→ Git 在新目录取出这些文件
→ 当前选中的系统 Python 创建新 .venv
→ 新 python.exe 按记录安装 pytest
→ 新 python.exe 运行 tests
→ 新 python.exe 处理有效 JSON
→ 新报告中有效 2、拒绝 0，进程退出码 0
```

这里既验证内容来源，也验证运行结果。两个 HEAD 相同只回答“检出的提交相同”；tests 和真实入口才回答“这份内容在新环境中按预期工作”。

这条路线仍使用同一台电脑的操作系统、已选择的 Python 安装和可用包来源。它排除了旧项目 `.venv` 与未提交源码的依赖，没有证明每一种 Python 版本或另一台操作系统都已实测。真正交给另一台机器时，应记录那里的 Python 版本，再重新运行相同检查。

版本名怎样连接回这些证据？

工单项目的 VERSION 内容是 `1.0.0`，RELEASE_NOTES 说明本版支持什么、需要什么、有哪些限制。它们和本次被测试的源码一起保存在 `$testedCommit` 所指的提交中。

当这份提交在新环境的检查也通过以后，可以让 `v1.0.0` 这个 tag（标签）指向它。标签提供容易交流的版本名；真正把结果固定下来的仍是它所指的提交。

```
v1.0.0
  ↓
$testedCommit：包含源码、测试、依赖、说明、VERSION 与数据
  ↓
针对这份提交取得的新环境测试、入口和报告结果
```

此后只在源工作区编辑文件，不会自动把改动加入这份提交，也不会让 `v1.0.0` 跟着变。若修改了 VERSION 或说明，应形成新的明确提交，再对该内容取得证据；不能先测试旧内容、后加新文件，却继续沿用旧结论。

当前程序只有一个脚本入口，使用者能够运行 Python，交付完整受控源码与说明已经足以让人运行。若以后需要统一安装命令、多个模块、命令行入口或版本分发，再进入 Python packaging（打包）。打包改变内容的分发和安装方式，仍需要正确的依赖、入口和验证；它不会补出遗漏的运行条件。

说明要留下哪些能执行的事实？

一个陌生使用者可以从本项目 README 与 RELEASE_NOTES 找到：需要的 Python 条件、如何建立环境和安装测试依赖、运行哪个文件、参数怎么给、样例应得到什么，以及失败后从哪里找回。

对当前程序，还要留下几个会影响后续动作的区别：

程序退出码	下一位使用者应怎样理解
0	完整报告已写出，全部记录有效
1	命令参数数量不对，检查入口写法
2	完整报告已写出，但存在拒绝记录，查看 issues
3	输入、格式或路径合同有问题，既有目标报告未被替换
4	权限或文件操作失败，不把临时半文件当完成报告

这也是为什么本章选择有效 JSON 来核对退出码 0。混合 CSV 合同本来就返回 2，不能因非零就说重建失败；应同时检查 3 条接受、4 条拒绝和完整报告。

当前没有必需环境变量或 secret，应在说明里明确写出这个空条件。以后若真的需要配置，说明保留字段名、用途与无效占位值，真实秘密由受控渠道另行提供，不能进入源码、日志或 Git。只有进程实际读取了指定来源，配置才参与这次运行。

限制同样影响使用：这个版本只处理本地单文件，同一时间只允许一个进程写同一输出路径。输出可从原始输入重新生成，源码失误可依据 Git diff 精确恢复；写出策略不承诺断电、强制终止或并发写入时的普遍恢复。这些内容应随文件存在，不靠作者口头补充。

查用：从随书内容建立独立的源仓库（首次可跳过）

首次学习只需记住短路线：复制到独立目录 → 建立 Git 提交 → 取得提交 ID → 新工作树建新 `.venv` → 用新解释器跑测试和真实入口。下面的完整 PowerShell 配方只是把这些动作和安全检查自动化；不需要逐行背诵。

接下来的完整配方适用于 Windows PowerShell 5.1 与 PowerShell 7，需要 Git 2.28+、Python 3.10+，以及 requirements 中测试依赖的可用来源。每个失败检查都让路线在当前事实不成立时停止；无需记住这些保护代码的完整写法。

先输入包含 `01_配套练习` 的资料根目录。命令检查随书清单和九个必需文件，只把这些文件复制到新 TEMP 子目录。清单用于识别配套来源，读取 release_id 不是重新计算每个文件的完整性校验。随书目录不会被用作 Git 仓库或安装目标。

POWERSHELL

```
$releaseRoot = [IO.Path]::GetFullPath((Read-Host 'Folder containing
  01_配套练习'))
$bundleRoot = Join-Path $releaseRoot '01_配套练习'
$manifestPath = Join-Path $bundleRoot 'companion-manifest.json'
if (-not (Test-Path -LiteralPath $manifestPath -PathType Leaf)) {
    throw 'Companion manifest is missing.'
}
$manifest = Get-Content -LiteralPath $manifestPath -Raw -Encoding UTF8 |
  ConvertFrom-Json
$releaseId = [string]$manifest.release_id
if ([string]::IsNullOrWhiteSpace($releaseId)) { throw 'Missing release_id.' }
$sourcePath = Join-Path $bundleRoot 'examples\a_program_tools\ticket-organizer'
$requiredFiles = @(
    'ticket_organizer.py', 'test_ticket_organizer.py', 'requirements.txt',
    '.gitignore', 'VERSION', 'README.md', 'RELEASE_NOTES.md',
    'data\tickets.csv', 'data\tickets.json'
)
foreach ($relativePath in $requiredFiles) {
    if (-not (Test-Path -LiteralPath (Join-Path $sourcePath $relativePath)
      -PathType Leaf)) {
        throw "Missing companion file: $relativePath"
    }
}
$a12Temp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
$a12Prefix = $a12Temp.TrimEnd('\') + '\'
$sourceRepoPath = [IO.Path]::GetFullPath(
    (Join-Path $a12Temp ('fde-a12-source-' + [guid]::NewGuid().ToString('N')))
)
```

POWERSHELL · 续

```
if (-not $sourceRepoPath.StartsWith($a12Prefix,
  [StringComparison]::OrdinalIgnoreCase) -or
    (Test-Path -LiteralPath $sourceRepoPath)) {
    throw 'Expected a new source directory inside TEMP.'
}
New-Item -ItemType Directory -Path $sourceRepoPath -ErrorAction Stop | Out-Null
New-Item -ItemType Directory -Path (Join-Path $sourceRepoPath 'data')
  -ErrorAction Stop | Out-Null
foreach ($relativePath in $requiredFiles) {
    Copy-Item -LiteralPath (Join-Path $sourcePath $relativePath) `
        -Destination (Join-Path $sourceRepoPath $relativePath) -ErrorAction
          Stop
}
Set-Location -LiteralPath $sourceRepoPath
$gitVersionText = git --version
if ($LASTEXITCODE -ne 0 -or
    $gitVersionText -notmatch '^git version (?<number>\d+(?:\.\d+){1,3})') {
    throw 'Cannot determine Git version.'
}
if ([version]$Matches.number -lt [version]'2.28') { throw 'Git 2.28+
  required.' }
git init -b main
if ($LASTEXITCODE -ne 0) { throw 'Git init failed.' }
git config --local user.name 'FDE Learner'
git config --local user.email 'learner@example.invalid'
git add -- .
git diff --staged --stat
git commit -m 'baseline ticket organizer'
if ($LASTEXITCODE -ne 0) { throw 'Baseline commit failed.' }
git rev-parse --show-toplevel
$releaseId
$sourceRepoPath
```

这里的 `git add .` 只用于刚按明确清单建立的副本，初始化身份只影响本仓库。之后的安装和报告位于被排除的 `.venv` 与 output 内。pytest 自己产生的缓存也不应成为业务源码。

接着取得这份源内容的运行结果，最后检查变化并记录提交。`-p no:cacheprovider` 关闭 pytest 的磁盘缓存，不改变当前测试断言；使用它可以让本次源状态更容易核对。

```powershell
python --version
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw 'Source environment creation failed.' }
$sourcePython = Join-Path $sourceRepoPath '.venv\Scripts\python.exe'
& $sourcePython -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw 'Source dependency installation failed.' }
& $sourcePython -m pytest -q -p no:cacheprovider
if ($LASTEXITCODE -ne 0) { throw 'Source tests failed.' }
& $sourcePython .\ticket_organizer.py .\data\tickets.json
  .\output\source-report.json
$sourceExit = $LASTEXITCODE
if ($sourceExit -ne 0) { throw "Source entry failed: $sourceExit" }
$sourceReport = Get-Content -LiteralPath .\output\source-report.json -Raw
  -Encoding UTF8 | ConvertFrom-Json
if ($sourceReport.summary.accepted_records -ne 2 -or
    $sourceReport.summary.rejected_records -ne 0) { throw 'Unexpected source
      report.' }
$sourceChanges = @(git status --porcelain)
if ($LASTEXITCODE -ne 0 -or $sourceChanges.Count -ne 0) {
    throw 'Source changed; do not label HEAD as the tested content.'
}
$testedCommit = (git rev-parse HEAD).Trim()
if ($LASTEXITCODE -ne 0 -or -not $testedCommit) { throw 'Cannot read tested
  commit.' }
$testedCommit
```

若出现未解释的变化，应先看具体路径与 diff。不能临时 add 或 restore 把状态抹平后，声称刚才测过的就是这份提交。

在新工作树中重新建立运行结果

下面继续使用刚取得的变量。新的路径必须不存在，由 Git 直接从 `$testedCommit` 建立。没有复制旧工作区或旧 `.venv`。如果中途关掉过 PowerShell，先用本节末的续接配方恢复已有源副本的身份，再回到这里；不要重新复制或初始化仓库。

```powershell
$rebuildPath = [IO.Path]::GetFullPath(
    (Join-Path $a12Temp ('fde-a12-rebuild-' + [guid]::NewGuid().ToString('N')))
)
if (-not $rebuildPath.StartsWith($a12Prefix,
  [StringComparison]::OrdinalIgnoreCase) -or
    (Test-Path -LiteralPath $rebuildPath)) { throw 'Expected a new rebuild
      directory inside TEMP.' }
```

POWERSHELL · 续

```
git -C $sourceRepoPath worktree add --detach $rebuildPath $testedCommit
if ($LASTEXITCODE -ne 0) { throw 'Worktree creation failed.' }
$rebuiltCommit = (git -C $rebuildPath rev-parse HEAD).Trim()
if ($LASTEXITCODE -ne 0 -or $rebuiltCommit -ne $testedCommit) { throw 'Commit
  mismatch.' }
$initialChanges = @(git -C $rebuildPath status --porcelain)
if ($LASTEXITCODE -ne 0 -or $initialChanges.Count -ne 0) { throw 'Initial
  worktree is not clean.' }
Set-Location -LiteralPath $rebuildPath
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw 'Rebuild environment creation failed.' }
$rebuildPython = Join-Path $rebuildPath '.venv\Scripts\python.exe'
& $rebuildPython -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw 'Rebuild dependency installation failed.' }
& $rebuildPython -m pytest -q -p no:cacheprovider
if ($LASTEXITCODE -ne 0) { throw 'Rebuild tests failed.' }
& $rebuildPython .\ticket_organizer.py .\data\tickets.json
  .\output\clean-report.json
$rebuildExit = $LASTEXITCODE
if ($rebuildExit -ne 0) { throw "Rebuild entry failed: $rebuildExit" }
$rebuiltReport = Get-Content -LiteralPath .\output\clean-report.json -Raw
  -Encoding UTF8 | ConvertFrom-Json
if ($rebuiltReport.summary.accepted_records -ne 2 -or
    $rebuiltReport.summary.rejected_records -ne 0) { throw 'Unexpected rebuilt
      report.' }
$finalChanges = @(git -C $rebuildPath status --porcelain)
if ($LASTEXITCODE -ne 0 -or $finalChanges.Count -ne 0) { throw 'Final worktree
  is not clean.' }
"tested commit: $testedCommit"
"rebuilt commit: $rebuiltCommit"
"entry exit: $rebuildExit"
"source: $sourceRepoPath"
"rebuild: $rebuildPath"
```

现在应同时看见：两个提交 ID 一致；新工作树初始和最终无未解释变化；该副本全部测试通过；有效 JSON 的新报告统计为 2、0，退出码为 0。当前收集到多少测试，以这份测试文件的实际结果为准，不能抄用别的版本数字。

若还要实际标记本次版本，只在这个临时源仓库中执行；同名标签已存在时停止，不强制改写：

```powershell
$existingTag = @(git -C $sourceRepoPath tag --list v1.0.0)
if ($LASTEXITCODE -ne 0 -or $existingTag.Count -ne 0) { throw 'Tag already
  exists or cannot be checked.' }
git -C $sourceRepoPath tag -a v1.0.0 $testedCommit -m 'offline ticket
  organizer 1.0.0'
if ($LASTEXITCODE -ne 0) { throw 'Tag creation failed.' }
git -C $sourceRepoPath rev-parse 'v1.0.0^{commit}'
git -C $sourceRepoPath show --stat v1.0.0
```

`^{commit}` 让 Git 取得标签最终指向的提交，结果应仍是 `$testedCommit`。不需要背这段写法；知道应核对“版本名实际指向哪份内容”才是使用它的理由。

请保留两个明确的临时路径和运行结果。需要清理时，先离开工作树，用 `git -C $sourceRepoPath worktree list` 核对 Git 登记的目录及需要保留的内容，再按确定的范围处理。这里不自动删除目录，也不把生成结果的成功当成删除其他路径的依据。

如果本次一直在原窗口完成，就跳过下面这段。它只供**关掉 PowerShell 后，从已有源副本接着做重建**时查用：目录仍在，窗口里的变量却不会自动回来。输入的是自己此前保存的源目录完整路径和完整 tested commit，不是随书目录，也不是新建目录。运行记录可以是自己保留的终端结果或笔记，但要能对上这份提交的测试通过、有效 JSON 入口退出 0、报告接受 2 / 拒绝 0，以及测试后的干净状态；不能只凭目录名或当前 HEAD 声称以前测过。

这段配方只用于本章在 TEMP 中建立的现存源仓库，输入路径时应对照此前保存的源目录，不要拿另一份仓库或重建工作树代替。先核对位置和状态，再询问记录；没有保存完整提交或对应运行结果时，留空或不回答 `YES`，程序会在**这份已有副本**重新运行测试和入口，成功后才取得本次 tested commit。原来的 `.venv` 若也已丢失，应停下，只在该源副本补建环境并按 requirements 安装依赖，再回到这里；不重做复制、`git init` 或基线提交。这里的参数是查用细节，不是新增的记忆任务。

```powershell
$testedCommit = $null
$a12Temp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
$a12Prefix = $a12Temp.TrimEnd('\') + '\'
$sourceRepoPath = [IO.Path]::GetFullPath((Read-Host 'Previously saved source
  directory')).TrimEnd('\')
if (-not $sourceRepoPath.StartsWith($a12Prefix,
  [StringComparison]::OrdinalIgnoreCase) -or
     -not (Test-Path -LiteralPath $sourceRepoPath -PathType Container)) {
     throw 'The existing source directory must be inside TEMP; do not create or
       initialize it here.'
}
$sourceRoot = git -C $sourceRepoPath rev-parse --show-toplevel
```

POWERSHELL · 续

```
if ($LASTEXITCODE -ne 0 -or -not $sourceRoot) { throw 'Cannot identify the
  existing source repository.' }
if ([IO.Path]::GetFullPath($sourceRoot).TrimEnd('\') -ne $sourceRepoPath) {
    throw 'Use the saved repository root, not a subdirectory.'
}
$sourceChanges = @(git -C $sourceRepoPath status --porcelain)
if ($LASTEXITCODE -ne 0 -or $sourceChanges.Count -ne 0) { throw 'Inspect
  unexplained source changes before resuming.' }
$resumeHead = git -C $sourceRepoPath rev-parse HEAD
if ($LASTEXITCODE -ne 0 -or -not $resumeHead) { throw 'Cannot read source
  HEAD.' }
$resumeHead = $resumeHead.Trim()
$recordedCommit = (Read-Host 'Saved full tested commit, or leave empty if not
  recorded').Trim()
if ($recordedCommit -and $recordedCommit -ne $resumeHead) { throw 'Saved
  commit does not match source HEAD; stop and inspect.' }
$hasRunRecord = Read-Host 'Do your saved passing results belong to this exact
  commit? Type YES, otherwise leave empty'
$sourcePython = Join-Path $sourceRepoPath '.venv\Scripts\python.exe'
Set-Location -LiteralPath $sourceRepoPath -ErrorAction Stop
if ($recordedCommit -and $hasRunRecord -ceq 'YES') {
    $testedCommit = $recordedCommit
} else {
    if (-not (Test-Path -LiteralPath $sourcePython -PathType Leaf)) { throw
      'Prepare only the source environment, then retry this resume block.' }
    & $sourcePython -m pytest -q -p no:cacheprovider
    if ($LASTEXITCODE -ne 0) { throw 'Source tests failed; no tested commit
      recorded.' }
    & $sourcePython .\ticket_organizer.py .\data\tickets.json
      .\output\source-report.json
    if ($LASTEXITCODE -ne 0) { throw 'Source entry failed; no tested commit
      recorded.' }
    $sourceReport = Get-Content -LiteralPath .\output\source-report.json -Raw
      -Encoding UTF8 -ErrorAction Stop | ConvertFrom-Json -ErrorAction Stop
    if ($sourceReport.summary.accepted_records -ne 2 -or
        $sourceReport.summary.rejected_records -ne 0) { throw 'Unexpected
          source report.' }
    $verifiedHead = git -C $sourceRepoPath rev-parse HEAD
    if ($LASTEXITCODE -ne 0 -or $verifiedHead -ne $resumeHead) { throw 'Source
      HEAD changed during verification.' }
    $sourceChanges = @(git -C $sourceRepoPath status --porcelain)
    if ($LASTEXITCODE -ne 0 -or $sourceChanges.Count -ne 0) { throw 'Source
      changed during verification.' }
    $testedCommit = $resumeHead
    'Passing results were obtained again in this existing source copy.'
}
```

POWERSHELL · 续

```
"source: $sourceRepoPath"
"tested commit: $testedCommit"
```

这段恢复的是源副本与证据的对应关系，还没有创建新工作树；成功后，回到本节的 `git worktree add --detach` 配方继续。若新工作树已经建立甚至已经通过验证，不必因换窗口再造一份：先用上面的源路径查看 `git worktree list`，找到自己保留的重建目录与结果，核对后从尚未完成的动作接续。源目录已经不存在、提交不符或有未解释修改时，先保留提示查清，不拿当前 HEAD 替换旧记录，也不删除或覆盖旧副本。

这次已经把一份提交、新环境、入口和结果接到一起。下一章再打开同一工单程序的内部，沿一条具体记录观察字段校验、循环、文件写出与退出结果怎样形成。

官方来源

本章使用随书原创工单项目，环境与分发关系参考 Missing Semester 2026：Packaging and Shipping Code。具体操作可查 Python venv、pip requirements 格式、Git worktree与 Git tag。这些操作文档是持续更新入口；实际使用时应核对本机版本、项目依赖记录和运行结果，不能用某次旧环境通过代替当前检查。

A13 | 把输入整理成一份可靠报告

先读一条有效记录从输入到报告的完整变化；后面的格式、测试和恢复内容是在这条主线需要时补证据。

你收到一份工单文件。里面有的记录完整，有的优先级写成了 `urgent`，还有重复编号。现在需要一份能交给别人继续处理的报告：有效工单按优先级排列，坏记录留下原因，数量能够核对；运行失败时，也能知道失败发生在哪里。

这一章用随书的离线工单整理器，把前面分开学过的函数、条件、循环、文件和测试接成一个能运行的小程序。先看一条工单最后变成什么，再顺着代码找出谁完成了每一步。

先看 T-201 最后到了哪里

输入文件 `data\tickets.json` 的完整内容是：

```JSON
[
  {
    "id": "T-201",
    "subject": "VPN cannot connect",
    "priority": 1,
    "status": "open",
    "owner": "Wei"
  },
  {
    "id": "T-202",
    "subject": "Replace keyboard",
    "priority": 3,
    "status": "pending",
    "owner": ""
  }
]
```

这里的工单只是本地样例记录。`T-201` 是编号，`subject` 是问题标题，`priority` 中的 `1` 表示最高优先级、`3` 表示最低，`status` 是处理状态，`owner` 是负责人。

整理后，报告中的 T-201 是：

```JSON
{
  "id": "T-201",
  "subject": "VPN cannot connect",
  "priority": 1,
  "priority_label": "high",
  "status": "open",
  "owner": "Wei"
}
```

它先从 JSON 文字变成 Python 字典，程序检查必需字段、优先级和状态，确认有效后补上 `priority_label: "high"`，把它放进有效工单列表。另一条 T-202 的负责人是空文字，程序把它补为 `unassigned`，表示尚未分配。优先级 `1` 排在 `3` 前面，因此报告里的顺序是 T-201、T-202。

完整报告还保存这些信息：

```JSON
{
  "input_records": 2,
  "accepted_records": 2,
  "rejected_records": 0,
  "by_status": {"closed": 0, "open": 1, "pending": 1},
  "by_priority": {"1": 1, "2": 0, "3": 1}
}
```

上面这一块是报告的 `summary` 部分。报告另有 `tickets` 保存两张有效工单，`issues` 保存拒绝原因；这次没有拒绝，所以它是空列表 `[]`。`source` 保存输入文件名 `tickets.json`。程序把整份报告写入指定的 JSON 文件，显示“有效 2，拒绝 0”，然后以退出码 `0` 结束。

这条路线已经包含了本章的核心：**文件读得出来、记录符合规则、报告写得完整，是三个分别要成立的事实。** 以下先沿 T-201 的成功路线展开，再看坏记录和写文件失败分别从哪里改道。

函数怎样接过上一段的结果

程序按下面的关系合作：

```
输入文件
  ↓ load_rows：选解析器，读成原始 rows
原始记录列表
  ↓ organize_rows：逐条交给 validate_row
  │              ├── 有效：ticket
  │              └── 无效：issue
  ↓ 检查重复、排序、计数
完整 report 字典
  ↓ write_report：写成目标 JSON 文件
磁盘报告
```

run 负责依次调用这些函数，并把结果交回命令行。

`load_rows` 不决定 `"urgent"` 能不能当优先级；`validate_row` 不决定文件该存在哪个目录。它们通过参数和返回值交接，所以可以单独验证某一段，也能从报告异常追回最早改变值的位置。

第一次读到这里先停。 先只确认 T-201 的输入怎样交给 `load_rows`、怎样经过 `validate_row`、最后怎样成为报告；现在不需要读完下面的整份源码。完整文件保留在随书配套中，下面的连续代码属于查用层：需要修改或追踪某一段时，再回到对应函数。

查用：完整 `ticket_organizer.py` 源码

下面的连续代码合起来就是完整的 `ticket_organizer.py`，没有用省略号藏起分支。它是修改、逐段追踪或运行后回查时使用的查用材料，不是 A13 首次主线的连续阅读任务。第一次先停在上面的输入→函数交接→报告关系；需要定位某个结果时，再只打开对应函数，不必从头读完整文件。

程序开头如下：

```python
"""Organize offline support tickets from CSV or JSON into a JSON report."""

from __future__ import annotations

import csv
import json
import sys
from contextlib import suppress
from pathlib import Path


REQUIRED_FIELDS = ("id", "subject", "priority", "status")
ALLOWED_STATUSES = {"open", "pending", "closed"}
PRIORITY_LABELS = {1: "high", 2: "normal", 3: "low"}
```

`csv`、`json` 负责文件格式，`sys` 连接命令行，`Path` 用来表示文件路径；清理失败时用到的 `suppress` 在写出部分再解释。它们都来自 Python 标准库。

三个大写名字保存共同规则。`REQUIRED_FIELDS` 是一个 tuple，按顺序列出必需字段；`PRIORITY_LABELS` 是 dict，用整数找到文字标签。`ALLOWED_STATUSES` 的花括号中没有 `key: value`，这是 set（集合）：这里只需用 `status in ALLOWED_STATUSES` 询问某个状态是不是允许值。

文件内容先成为原始记录

下面三个函数覆盖两种格式。先顺着 JSON 分支读：文件打开后，`json.load(file)` 把文字解析成 Python 值；函数确认它是一组字典，再返回列表。

这里的 `path: Path`、`-> list[dict[str, object]]` 是类型提示（type hints）：参数预期是路径对象，结果预期是列表，列表里每项是字典，字典的键是文字，值暂时可能有多种类型。提示本身不验证输入；真正的运行检查仍要写在函数体内。开头的 `from __future__ import annotations` 推迟这些注解的求值，也不替代检查。

`Path("data/tickets.json")` 根据文字建立路径对象；`.suffix` 取得 `.json`，`.open(...)` 打开对应文件。建立路径对象不等于文件已经存在，也不等于已经读过文件。

```python
def load_csv(path: Path) -> list[dict[str, object]]:
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        reader = csv.DictReader(file, strict=True)
        if reader.fieldnames is None:
            raise ValueError("CSV 缺少表头。")

        rows: list[dict[str, object]] = []
```

PYTHON · 续

```
        for row in reader:
            rows.append(dict(row))
        return rows


def load_json(path: Path) -> list[dict[str, object]]:
    with path.open("r", encoding="utf-8") as file:
        data = json.load(file)

    if not isinstance(data, list):
        raise ValueError("JSON 顶层必须是 ticket object 组成的 list。")

    rows: list[dict[str, object]] = []
    for record_number, item in enumerate(data, start=1):
        if not isinstance(item, dict):
            raise ValueError(f"JSON 第 {record_number} 条记录不是 object。")
        rows.append(item)
    return rows


def load_rows(path: Path) -> list[dict[str, object]]:
    suffix = path.suffix.lower()
    if suffix == ".csv":
        return load_csv(path)
    if suffix == ".json":
        return load_json(path)
    raise ValueError("输入文件只支持 .csv 或 .json。")
```

`isinstance(data, list)` 询问“这个值是不是 list”，得到 `True` 或 `False`。JSON 语法也允许一个单独的 object、数字或文字，所以解析成功以后还要检查容器形状。`not` 反转判断：顶层不是 list 就主动 `raise ValueError(...)`，整份输入在这里停止。

`enumerate(data, start=1)` 每次同时交出一个记录号和当前项。上面的输入第一次得到 `1` 与 T-201 的字典，第二次得到 `2` 与 T-202 的字典。`record_number, item` 分别接住这两个位置，因此如果输入是 `[{}, 7]`，错误可以明确写成“第 2 条记录不是 object”。这里的编号是记录顺序，不是文本文件行号；一条 JSON 记录可以跨多行。

T-201 通过这两层形状检查，`rows.append(item)` 把它加入列表，最后交回包含两条记录的 `rows`。此时 `priority` 是整数 `1`，但还没有证明它在允许范围内。

CSV 分支的交接目标相同，不过 CSV 的字段值通常是文字：文件中的优先级 `1` 会变成字符串 `"1"`。`DictReader` 按表头取得字段名称，并处理带引号的逗号；`newline=""` 把换行处理交给 CSV 模块，`utf-8-sig` 也接受开头带 UTF-8 BOM 的文件，`strict=True` 让未闭合引号等格式错误成为 `csv.Error`。这些参数以后可以查，当前要把解析器产生的值和经过验证的工单分开。

这里的“严格”只涉及解析器检查的 CSV 格式语法，不会自动验证表头名称不重复、每条记录的字段数与表头一致。本章的 CSV 输入要求这两项成立；当前程序没有单独核对它们。重复表头可能让前一列的值被覆盖，多余列会被收在 `None` 键下而被后续整理忽略，因此不能用“报告已写出”证明它们没有发生。A08 已展示字段数检查；接收不符合上述合同的外部文件时，还需要补上表头和记录结构检查，不能只增加 `strict=True` 就认为这一步完成。

假如 JSON 文字损坏，`json.load` 不能产生可用结果；假如 CSV 中某行写着 `urgent`，CSV 仍可以把它读成文字。前者是整份输入无法载入，后者留给下一段决定是否拒绝这一条记录。

一条记录怎样取得继续使用的资格

程序要求 `id`、`subject`、`status` 是文字，清理两端空白后，编号与标题不能为空，状态只能是 `open`、`pending`、`closed`。优先级接受整数，或能用 `int(...)` 转成整数的文字，转换后只允许 `1`、`2`、`3`。

负责人 `owner` 可以缺少、为 JSON 的 `null`，或是空文字，最后统一为 `unassigned`；提供其他值时必须是文字。例如数字 `0`、空列表 `[]` 也会被拒绝，不能仅凭“它看起来是空的”跳过类型检查。

先看一个很短的辅助函数。它把允许清理的基础值变成去掉两端空白的文字；遇到其他类型返回空文字：

PYTHON

```
def cleaned(value: object) -> str:
    if isinstance(value, bool) or not isinstance(value, (str, int)):
        return ""
    return str(value).strip()
```

`isinstance(value, (str, int))` 表示 str 或 int 满足其一即可。为什么另外拒绝 bool？因为在 Python 中，`isinstance(True, int)` 也为 `True`，`int(True)` 会得到 `1`；但本程序不能把 JSON 的 `true` 当成“高优先级”。所以要明确拒绝这个原始类型。这也是为什么不能只靠“`int` 转得出来”决定业务数据有效。

`cleaned` 本身不会记错误，也不是最终接收规则。比如 `cleaned(123)` 返回 `"123"`，但数字标题仍要由下面的 `validate_row` 拒绝。清理方便后面的判断和错误显示，不能把原来的类型错误洗掉。

validate_row 比较长，下面按连续三段读。首先检查字段和原始类型：

```python
def validate_row(
    row: dict[str, object], record_number: int
) -> tuple[dict[str, object] | None, dict[str, object] | None]:
    errors: list[str] = []
    missing_fields: list[str] = []
    for field in REQUIRED_FIELDS:
        if field not in row:
            missing_fields.append(field)
    if missing_fields:
        errors.append("缺少字段：" + "，".join(missing_fields))

    for field in ("id", "subject", "status"):
        if field in row and not isinstance(row[field], str):
            errors.append(f"{field} 必须是文字")

    owner_value = row.get("owner")
    if owner_value is not None and not isinstance(owner_value, str):
        errors.append("owner 必须是文字")

    priority_value = row.get("priority")
    priority_has_valid_type = isinstance(priority_value, (str, int)) and not
      isinstance(
        priority_value, bool
    )
    if "priority" in row and not priority_has_valid_type:
        errors.append("priority 必须是整数或整数字符串")
```

返回注解里有两个位置，分别预留给有效工单和拒绝说明；dict[...] | None 表示该位置可能是字典，也可能没有结果。稍后会看见它怎样真的返回这两个位置。

field in row 查的是字典里有没有这个键。row[field] 则要求这个键存在；这里先检查 field in row，and 左边不成立时不会继续访问右边，避免缺字段时先触发 KeyError。

row.get("owner") 也取字段，但键缺少时默认返回 None。JSON 的 null 解析后也是 None，本程序有意把两种情况都当作未提供负责人。必需字段则不能这样略过：开头的循环已经单独收集缺失名称。

if missing_fields: 使用列表的真值：[] 在条件里视为假，收集到一项之后视为真。它问的是“有没有缺失字段”，并没有把列表变量改成布尔值。"，".join(...) 再把多个缺失名称连成一条可读说明。T-201 的字段齐全、类型符合要求，所以当前 errors 仍是 []。

接着，函数清理具体值并检查内容。下面代码仍缩进在同一个函数中：

```python
    ticket_id = cleaned(row.get("id"))
    subject = cleaned(row.get("subject"))
    status = cleaned(row.get("status")).lower()
    owner = cleaned(owner_value) or "unassigned"

    if isinstance(row.get("id"), str) and not ticket_id:
        errors.append("id 不能为空")
    if isinstance(row.get("subject"), str) and not subject:
        errors.append("subject 不能为空")
    if isinstance(row.get("status"), str) and status not in ALLOWED_STATUSES:
        allowed = ", ".join(sorted(ALLOWED_STATUSES))
        errors.append(f"status 必须是：{allowed}")

    priority: int | None = None
    if priority_has_valid_type:
        try:
            priority = int(priority_value)
        except ValueError:
            errors.append("priority 必须是整数 1、2 或 3")
        else:
            if priority not in PRIORITY_LABELS:
                errors.append("priority 必须在 1 到 3 之间")
```

这里的 `or` 会返回一边的原值。`cleaned("Wei")` 是非空文字，因此 `"Wei" or "unassigned"` 得到 `"Wei"`；T-202 的 `cleaned("")` 是空文字，因此 `"" or "unassigned"` 得到 `"unassigned"`。它没有把负责人变成 `True` 或 `False`。同样，`not subject` 在清理后的标题为空时成立。

T-201 的 `status` 是 `"open"`，清理并转成小写后仍然属于允许集合。`priority_value` 是整数 `1`，`int(1)` 得到 `1`，它也是 `PRIORITY_LABELS` 的键，所以不会追加错误。CSV 的 `"1"` 在这个位置也会成为整数 `1`；下游终于可以按同一种类型处理优先级。

这段同时保留两条不同的拒绝路线：

```
"urgent" → int(...) 引发 ValueError → except 记录“必须是整数”
"9"      → int(...) 得到 9          → else 中发现超范围
```

第一条转换没有完成，`priority` 保持先前的 `None`；第二条已经有整数，只是不符合本程序规则。`try` 圈住的是转换操作，后面的范围条件处理正常产生的值。如果一条记录同时有多个错误，函数继续收集它们，便于一次报告出来。

最后决定交回哪个结果：

```python
    if errors:
        issue = {
            "record": record_number,
            "id": ticket_id or None,
            "errors": errors,
        }
        return None, issue

    ticket = {
        "id": ticket_id,
        "subject": subject,
        "priority": priority,
        "priority_label": PRIORITY_LABELS[priority],
        "status": status,
        "owner": owner,
    }
    return ticket, None
```

`if errors:` 表示已经收集到至少一个错误，就建立 `issue`，保留记录号、能显示的 ID 和完整错误列表；`ticket_id or None` 让空 ID 以 `None` 表示，写成 JSON 后是 `null`。这里的 `return` 结束本次函数调用，下面的有效工单构造不会再执行。

T-201 没有错误，函数建立新字典，利用 `PRIORITY_LABELS[1]` 得到 `"high"`，于是产生章首展示的最终工单，再返回 `ticket, None`。它没有直接修改原始 `row`，新工单拥有清理后的值和补充标签。

调用者这样接收：`ticket, issue = validate_row(row, record_number)`。右边先返回一个包含两个位置的 tuple，再拆包给左边两个名字。有效时第一个位置有 ticket、第二个为 `None`；无效时顺序相反。调用者因此可以检查 `issue is not None` 决定下一步，而不是猜测函数有没有打印错误。

多条记录怎样变成一份报告

单条验证尚不知道之前接受过哪些编号，也不知道最终的统计数量。这些跨记录的状态由 `organize_rows` 保存。它的完整实现如下：

```python
def organize_rows(rows: list[dict[str, object]]) -> dict[str, object]:
    accepted: list[dict[str, object]] = []
    issues: list[dict[str, object]] = []
    seen_ids: set[str] = set()
```

PYTHON · 续

```
    for record_number, row in enumerate(rows, start=1):
        ticket, issue = validate_row(row, record_number)
        if issue is not None:
            issues.append(issue)
            continue

        ticket_id = str(ticket["id"])
        if ticket_id in seen_ids:
            issues.append(
                {
                    "record": record_number,
                    "id": ticket_id,
                    "errors": ["id 与前面的有效记录重复"],
                }
            )
            continue

        seen_ids.add(ticket_id)
        accepted.append(ticket)

    accepted.sort(key=lambda ticket: (int(ticket["priority"]),
      str(ticket["id"])))
    status_counts = {"closed": 0, "open": 0, "pending": 0}
    priority_counts = {"1": 0, "2": 0, "3": 0}

    for ticket in accepted:
        status_counts[str(ticket["status"])] += 1
        priority_counts[str(ticket["priority"])] += 1

    summary = {
        "input_records": len(rows),
        "accepted_records": len(accepted),
        "rejected_records": len(issues),
        "by_status": status_counts,
        "by_priority": priority_counts,
    }
    return {"summary": summary, "tickets": accepted, "issues": issues}
```

`accepted` 收集有效工单，`issues` 收集拒绝说明。`seen_ids = set()` 建立空集合，只保存已经接受的 ID；这里不能写 `{}`，那是空字典。`in` 询问 ID 是否已在集合中，`.add(...)` 把新 ID 记进去。

为什么不把它们都放进同一种容器？`accepted` 后面还要按优先级排序、输出完整记录，因此用列表；`seen_ids` 只需要回答“这个编号是否出现过”，不需要记录出现次数或排列次序，因此用集合。用列表配合成员判断也能记已用编号，集合不是唯一能实现去重的办法；这里选它，是因为它直接表达了只关心成员是否存在的用途。真正拒绝重复工单的仍是后面的判断和跳过动作，不是建立集合后程序就自动完成去重。

T-201 的一轮是：验证交回 `(ticket, None)`，略过错误分支；`"T-201"` 尚不在 `seen_ids`，所以同时执行 `seen_ids.add(...)` 和 `accepted.append(...)`。处理 T-202 时，两者已记着前一轮结果。它也通过以后，集合有两个 ID，`accepted` 有两张工单。

`continue` 的作用是结束本轮剩余动作，直接取下一条记录。若验证已经产生 issue，就不能继续读取 `ticket["id"]`，因为此时 ticket 是 `None`；若编号重复，也不能继续加入 accepted。两个 `continue` 都保护了这个“拒绝后不再接收”的顺序。

只有被接受的记录才占用编号。 例如先来一条空标题的 T-300，它在验证后就进入 `continue`，尚未修改 `seen_ids`；随后标题有效的 T-300 可以被接受。如果后来再来一条有效的 T-300，才会因前面已有有效记录而被拒绝。

记录全部处理完以后，`accepted.sort(...)` 直接调整列表顺序。`key=` 接收一个产生排序依据的函数；这里的 `lambda ticket: (...)` 是短小的无名函数，接收一张工单，返回 `(priority, id)` 这个 tuple。

这里后续只需要排好序的有效工单，不需要另留接收时的顺序，所以直接用列表的 `sort()`。如果还要保留原顺序，并另做一份排序结果，就用 A08 的 `sorted(accepted, key=...)`，由新名字接住返回的列表。两者都能排序，区别在是否改变原列表；`sort()` 的返回值是 `None`，不能用它替换 `accepted`。

tuple 按位置依次比较：先比较优先级，相同才比较 ID。因此 `(1, "T-1")` 排在 `(1, "T-3")` 前面，两者都排在 `(3, "T-2")` 前面。ID 在这里按文字比较，并没有提取末尾数字，所以相同优先级下 `T-10` 会排在 `T-2` 前面。这是当前代码的规则，修改规则时测试也要明确记录新的预期。

随后计数。处理 T-201 时，`status_counts["open"] += 1` 相当于先读出原值 `0`、加一，再将 `1` 放回同一个键；`priority_counts["1"]` 也从 `0` 变成 `1`。T-202 让 `pending` 和 `"3"` 各加一。计数只遍历 `accepted`，拒绝记录不会被算进业务分类。

最后 `len(rows)` 是输入条数，`len(accepted)` 是接受条数，`len(issues)` 是拒绝条数。一个输入记录至多产生一条 issue，但那条 issue 的 `errors` 可以有多个原因。于是输入数量满足“接受条数 + 拒绝条数”，不会因为一条记录有两个错误就多算一条输入。

坏记录怎样进入报告，而让下一条继续

配套的另一份输入 `data\tickets.csv` 完整内容如下：

```csv
id,subject,priority,status,owner
T-101,"Printer offline, second floor",1,open,Wei
T-102,Password reset,2,pending,
T-103,Monitor flickers,3,closed,Lin
T-104,Missing priority,urgent,open,Chen
T-105,,2,open,Chen
T-102,Duplicate id,2,open,Wei
T-106,Unknown status,1,blocked,Wei
```

第一条标题里的逗号是标题内容，外围引号让 CSV 解析器知道它属于同一个字段。`split(",")` 会把这条标题错拆成两列。

前三条通过以后，`accepted` 中已有 T-101、T-102、T-103。后四条的最早拒绝位置不同：

输入记录号	关键值	最早拒绝的位置	保存的原因
4	T-104，`priority="urgent"`	`int(...)` 无法转换	priority 必须是整数 1、2 或 3
5	T-105，`subject=""`	清理后的标题为空	subject 不能为空
6	第二个 T-102	ID 已在 `seen_ids`	id 与前面的有效记录重复
7	T-106，`status="blocked"`	状态不在允许集合	status 必须是：closed, open, pending

例如第 4 条最终成为这份拒绝说明：

```json
{
  "record": 4,
  "id": "T-104",
  "errors": ["priority 必须是整数 1、2 或 3"]
}
```

程序把它加入 `issues`，继续处理第 5 条，最后仍写出完整报告。统计是输入 `7`、接受 `3`、拒绝 `4`；有效状态 closed、open、pending 各 `1`，有效优先级 1、2、3 也各 `1`。报告的 `tickets` 按优先级排列为 T-101、T-102、T-103。

这次进程退出码为 2，表示“报告已完成，但有拒绝记录，需要查看 issues”。因此，看到非零退出码时要根据这个程序的约定解释，不能把所有非零结果都当成未捕获异常。

完整报告怎样才成为磁盘上的新目标

前面的 report 是内存里的字典，程序结束后不会自动留在磁盘。现在需要把它写成 JSON。若直接以 "w" 打开已有的好报告，旧内容会先被截断；假如只写到一半就失败，目标只剩半份。

所以程序先写同目录临时文件，写完并关闭后才替换正式目标。完整代码如下：

```python
def write_report(path: Path, report: dict[str, object]) -> None:
    if path.suffix.lower() != ".json":
        raise ValueError("输出文件必须使用 .json 扩展名。")

    path.parent.mkdir(parents=True, exist_ok=True)
    temporary_path = path.with_name(f".{path.name}.tmp")
    try:
        with temporary_path.open("w", encoding="utf-8") as file:
            json.dump(report, file, ensure_ascii=False, indent=2)
            file.write("\n")
        temporary_path.replace(path)
    except (OSError, UnicodeEncodeError):
        with suppress(OSError):
            temporary_path.unlink()
        raise
```

如果输出路径是 output/report.json，path.parent 是 output；.mkdir(parents=True, exist_ok=True) 建立需要的目录，已经存在的目录可以继续使用。path.name 是 report.json，.with_name(...) 在同一个父目录下得到 output/.report.json.tmp。

json.dump 把 report 写给打开的文件，ensure_ascii=False 允许直接写出中文字，indent=2 让层级容易阅读；再写一个换行。退出 with 后文件关闭，才执行 .replace(path)，用完成的临时文件替换目标。创建路径对象、改变内存字典和真正写文件分别发生在明确位置。

已有报告的变化可以这样追：

写入前：report.json 是旧报告
写入中：新内容进入 .report.json.tmp，report.json 仍是旧报告
写完并关闭：临时文件已含完整新 JSON
替换成功：report.json 才变成新报告

同目录安排使替换不需要跨文件系统搬运。这里需要理解操作顺序，不必把全部 Path 方法一次背完。

如果写临时文件或替换时发生 OSError，或者编码时发生 UnicodeEncodeError，程序进入 except。.unlink() 尝试删除这个明确的临时文件；with suppress(OSError) 只包这一步清理，意思是清理本身若又发生文件错误，不让第二个错误盖住最初错误。随后单独写的 raise 把当前正在处理的原异常继续抛给调用者，它既不表示成功，也没有重新开始写入。

旧报告的保护来自“先写临时文件，后替换”的顺序，异常处理负责报告和尽力清理。 清理如果也失败，临时文件可能留下。程序没有多进程锁，也没有断电恢复协议；同一个输出路径同时只运行一个写入进程。强杀、断电和并发写入不在这里的清理保证内。实际输入输出也应使用分开的普通文件，不用硬链接等别名把同一个文件伪装成两个目标。

最外层怎样告诉命令行这次发生了什么

下面是主程序最后一段：run 与直接执行入口。之前各函数都只定义了自己的工作；直接运行文件时，最底下的入口才把真实命令参数交给 run，由它按顺序调用。

PYTHON

```
def run(argv: list[str]) -> int:
    if len(argv) != 3:
        print("用法：python ticket_organizer.py INPUT OUTPUT.json",
              file=sys.stderr)
        return 1

    input_path = Path(argv[1])
    output_path = Path(argv[2])

    try:
        if input_path.resolve() == output_path.resolve():
            raise ValueError("输入和输出不能是同一个文件。")
        rows = load_rows(input_path)
        report = organize_rows(rows)
        report["source"] = input_path.name
    except FileNotFoundError:
        print(f"输入失败：找不到 {input_path}", file=sys.stderr)
        return 3
    except (csv.Error, json.JSONDecodeError, UnicodeDecodeError, ValueError)
      as error:
        print(f"输入或格式失败：{error}", file=sys.stderr)
        return 3
    except PermissionError as error:
        print(f"权限失败：{error}", file=sys.stderr)
        return 4
    except OSError as error:
        print(f"文件操作失败：{error}", file=sys.stderr)
        return 4
```

PYTHON · 续

```
    try:
        write_report(output_path, report)
    except PermissionError as error:
        print(f"权限失败：{error}", file=sys.stderr)
        return 4
    except UnicodeEncodeError as error:
        print(f"文件编码失败：{error}", file=sys.stderr)
        return 4
    except OSError as error:
        print(f"文件操作失败：{error}", file=sys.stderr)
        return 4
    except ValueError as error:
        print(f"输入或格式失败：{error}", file=sys.stderr)
        return 3

    summary = report["summary"]
    print(
        "报告已写入："
        f"{output_path}；有效 {summary['accepted_records']}，"
        f"拒绝 {summary['rejected_records']}。"
    )
    if summary["rejected_records"] > 0:
        return 2
    return 0


if __name__ == "__main__":
    sys.exit(run(sys.argv))
```

命令行运行 `python ticket_organizer.py data/tickets.json output/report.json` 时，`sys.argv` 有三个字符串：程序文件名、输入路径、输出路径。数量不对就显示用法，返回 `1`，避免后面读取不存在的位置。

第一个 `try` 处理输入和内存整理。`.resolve()` 把路径解析成绝对路径并解析符号链接；两个解析结果相同时，程序提前拒绝，避免用报告覆盖输入。这是当前采用的路径检查，不是所有可能文件别名的完整识别。

若载入坏 JSON，异常就在第一个 `try` 中发生，`run` 打印格式错误并返回 `3`。第二个 `try` 尚未开始，旧目标根本没有被打开。若载入成功，第二个 `try` 才调用 `write_report`。

为什么不用一个巨大 `try` 包住全部？`FileNotFoundError` 在输入侧意味着缺少输入，返回 `3`；如果它发生在写出侧，例如写入期间输出目录消失，它是 `OSError` 的一种，会得到文件操作失败的 `4`。异常的类型与发生位置共同决定外部含义。

退出码	本程序中的含义	能观察到的文件事实
`0`	整理完成，全部记录有效	完整新报告已写出
`1`	参数数量不正确	没有开始载入和写出
`2`	整理完成，有记录被拒绝	完整新报告包含 issues
`3`	输入、解析或格式约定不满足，包括输出非 `.json`	不替换旧目标
`4`	权限、编码或其他文件操作失败	失败阶段明确；写临时文件失败不把半份作为最终目标

只有写出成功后，程序才显示“报告已写入”。`run` 返回的整数经 `sys.exit(...)` 成为进程退出码；PowerShell 用 `$LASTEXITCODE` 取得它。显示文字给人看，退出码让外部命令判断，JSON 报告让人或下一段程序继续读取。

测试导入 `ticket_organizer` 时，底下的 `if __name__ == "__main__"` 不成立，因此不会擅自执行真实命令行。测试可以自己传一份 argv 给 `run`，得到可检查的返回值。

先用简单测试固定业务，再检查失败边界

先看配套测试中最直接的一条。它不给文件系统安排工作，只把一条原始记录交给 `validate_row`，再检查清理后的值：

PYTHON
```
from ticket_organizer import validate_row


def test_valid_row_is_cleaned_and_classified() -> None:
    ticket, issue = validate_row(
        {
            "id": " T-1 ",
            "subject": " Printer offline ",
            "priority": "1",
            "status": "OPEN",
            "owner": "",
        },
        1,
    )

    assert issue is None
    assert ticket is not None
    assert ticket["id"] == "T-1"
    assert ticket["priority_label"] == "high"
    assert ticket["owner"] == "unassigned"
```

这些预期来自业务约定：去两端空白，优先级 1 对应 high，空负责人用默认值。这个测试具体断言了 ID、标签和负责人；不能因为它输入了大写状态，就说它已经单独证明状态小写转换正确，断言范围要看代码。

如果要保护“CSV 的 `"1"` 和 JSON 的 `1` 最终相同”，应该比较 `organize_rows(load_rows(path))["tickets"]`。不能比较刚解析的原始 rows，因为 CSV 与 JSON 在那时允许有不同的值类型。配套测试正是在整理后比较两份 tickets。

文件失败需要另一种输入：让预期失败在可控制的位置发生，不必真把磁盘塞满。pytest 可以为测试提供几个现成对象，这些对象叫 fixture：`tmp_path` 给当前测试一个独立临时目录；`capsys` 收集正常输出和错误输出；`monkeypatch` 在本测试中临时替换某个被调用对象，结束后恢复。

下面是配套中的一个完整入口测试，连同它需要的导入和辅助函数，可以单独阅读。它先准备有效输入和旧报告，再让 `write_report` 有意抛出文件错误：

PYTHON

```
import json
from pathlib import Path

import pytest

import ticket_organizer
from ticket_organizer import run


SENTINEL_REPORT = '{"keep": true}\n'


def write_valid_input(path: Path) -> None:
    path.write_text(
        json.dumps(
            [
                {
                    "id": "T-1",
                    "subject": "Printer offline",
                    "priority": 1,
                    "status": "open",
                }
            ]
        ),
        encoding="utf-8",
    )


def test_run_write_oserror_returns_four_and_preserves_target(
    tmp_path: Path, capsys: pytest.CaptureFixture[str], monkeypatch:
      pytest.MonkeyPatch
) -> None:
    input_path = tmp_path / "tickets.json"
    output_path = tmp_path / "report.json"
    write_valid_input(input_path)
    output_path.write_text(SENTINEL_REPORT, encoding="utf-8")

    def fail_write_report(path: Path, report: dict[str, object]) -> None:
        raise OSError("disk full")

    monkeypatch.setattr(ticket_organizer, "write_report", fail_write_report)

    result = run(["ticket_organizer.py", str(input_path), str(output_path)])
    captured = capsys.readouterr()
```

PYTHON · 续

```
    assert result == 4
    assert output_path.read_text(encoding="utf-8") == SENTINEL_REPORT
    assert "文件操作失败" in captured.err
```

pytest 看见函数参数名后提供对应 fixture；这些参数不是我们忘记传入的值。`tmp_path / "tickets.json"` 用 Path 的 `/` 连接路径。辅助函数里的 `json.dumps(...)` 先产生 JSON 文字，再由 `write_text` 写文件；主程序里的 `json.dump(..., file)` 则直接写给打开的文件。

`monkeypatch.setattr(...)` 暂时让模块中的 `write_report` 指向 `fail_write_report`。于是 `run` 仍真的走过载入、验证和整理，但来到写出调用时立即收到 `OSError("disk full")`。`capsys.readouterr()` 取出已收集输出，`.err` 是错误输出。这三条断言共同证明：run 返回 4、旧内容保持、错误说明进入 stderr。

这个测试替换了整个 `write_report`，所以它单独不能证明真实写入函数会清理临时文件。配套还有直接调用真实 `write_report` 的测试，只把其内部的 `json.dump` 替换成会抛出 `OSError` 或 `UnicodeEncodeError` 的函数，并在预期异常后检查旧报告内容与临时文件是否存在。两组证据分别保护“错误怎样传到入口”和“真正写文件时怎样处理旧目标”。

测试通过只覆盖断言实际走过的路线。强杀、断电、并发，以及未安排的输入组合，不会因为测试全绿就自动得到保证。

在自己的实验目录里运行一次

随书项目在资料包根目录下的 `01_配套练习\examples\a_program_tools\ticket-organizer`。那里是供复制的发布副本。先把下面列出的九个文件复制到新实验目录，再在副本中运行、安装测试依赖或练习 Git。

```
ticket-organizer/
├── ticket_organizer.py
├── test_ticket_organizer.py
├── data/
│   ├── tickets.csv
│   └── tickets.json
├── requirements.txt
├── .gitignore
├── VERSION
├── RELEASE_NOTES.md
└── README.md
```

在 PowerShell 运行下面这段。输入资料包根目录后，它会核对所需文件，再在系统临时目录下建立一个带随机名称的独立实验目录；原配套目录保持作为复制来源。

```powershell
$a13ReleaseRoot = [IO.Path]::GetFullPath((Read-Host '资料包根目录'))
$a13Source = Join-Path $a13ReleaseRoot
  '01_配套练习\examples\a_program_tools\ticket-organizer'
$a13Files = @(
    'ticket_organizer.py', 'test_ticket_organizer.py', 'requirements.txt',
    '.gitignore', 'VERSION', 'RELEASE_NOTES.md', 'README.md',
    'data\tickets.csv', 'data\tickets.json'
)
foreach ($a13Relative in $a13Files) {
    if (-not (Test-Path -LiteralPath (Join-Path $a13Source $a13Relative)
      -PathType Leaf)) {
        throw "配套文件不完整：$a13Relative"
    }
}
$a13Temp = [IO.Path]::GetFullPath([IO.Path]::GetTempPath())
$a13Prefix = $a13Temp.TrimEnd('\') + '\'
$a13Lab = [IO.Path]::GetFullPath(
    (Join-Path $a13Temp ('fde-a13-' + [guid]::NewGuid().ToString('N')))
)
if (-not $a13Lab.StartsWith($a13Prefix, [StringComparison]::OrdinalIgnoreCase)
  -or
    (Test-Path -LiteralPath $a13Lab)) {
    throw '需要 TEMP 内尚不存在的新实验目录。'
}
New-Item -ItemType Directory -Path $a13Lab -ErrorAction Stop | Out-Null
New-Item -ItemType Directory -Path (Join-Path $a13Lab 'data') -ErrorAction
  Stop | Out-Null
foreach ($a13Relative in $a13Files) {
    Copy-Item -LiteralPath (Join-Path $a13Source $a13Relative) -Destination
      (Join-Path $a13Lab $a13Relative) -ErrorAction Stop
}
Set-Location -LiteralPath $a13Lab
$a13Lab
```

记下最后显示的实验目录，后续命令都在这里运行。主程序只需要 Python 标准库，可以先运行成功输入：

```powershell
python .\ticket_organizer.py .\data\tickets.json .\output\report-valid.json
$a13ValidExit = $LASTEXITCODE
$a13ValidExit
Get-Content -LiteralPath .\output\report-valid.json -Raw -Encoding UTF8
```

这次应显示“有效 2，拒绝 0”，保存的退出码为 `0`。打开报告，检查 T-201 的 `high`、T-202 的 `unassigned`，以及章首的统计。这里紧接程序保存 `$LASTEXITCODE`，避免后来另一条外部命令更新它。

再运行混合 CSV：

```powershell
python .\ticket_organizer.py .\data\tickets.csv .\output\report-mixed.json
$a13MixedExit = $LASTEXITCODE
$a13MixedExit
Get-Content -LiteralPath .\output\report-mixed.json -Raw -Encoding UTF8
```

应得到“有效 3，拒绝 4”和退出码 `2`。检查 issue 的记录号是 `4`、`5`、`6`、`7`，并把每条原因与输入对上。报告中的记录号不包括 CSV 表头。

需要运行自动测试时，在这份实验副本里创建环境并安装测试依赖：

```powershell
python -m venv .venv
if ($LASTEXITCODE -ne 0) { throw '无法建立测试环境' }
$a13Python = Join-Path $a13Lab '.venv\Scripts\python.exe'
& $a13Python -m pip install -r .\requirements.txt
if ($LASTEXITCODE -ne 0) { throw '测试依赖安装失败' }
& $a13Python -m pytest --collect-only -q
if ($LASTEXITCODE -ne 0) { throw '无法收集测试' }
& $a13Python -m pytest -q
if ($LASTEXITCODE -ne 0) { throw '测试未全部通过' }
```

正常运行工单整理器不需要网络；首次安装 pytest 需要可用软件源或事先准备的本地包。收集命令会列出当前文件里的测试，随后应全部通过。核对的是当前副本实际有哪些测试及其结果，不能拿另一份项目或旧截图的数字代替。

需要追溯这份配套来自哪里时，资料包内 `01_配套练习\companion-manifest.json` 保存其标识和文件信息；将它与实验目录位置一起保留。单纯复制成功不等于已经逐文件核对哈希，也不等于证明精确 commit 可以重建。

用一次小改动检查自己能否找回关系

先不改主程序，在实验副本的 `data\tickets.json` 中把 T-202 的优先级 `3` 改成 `1`，把负责人改成 `" Lin "`。运行前预测：它最后排在哪里，负责人变成什么，哪些计数要变？

完整推演是：优先级转换后为整数 `1`，标签成为 `high`；负责人清理两端空白后为 `Lin`。两条工单同优先级，要继续比较 ID，所以 T-201 仍在 T-202 前面。按优先级统计变为 `{"1": 2, "2": 0, "3": 0}`，按状态统计不变，仍接受 2、拒绝 0。运行后打开报告，对照这几项即可。如果负责人对而统计不对，就从 accepted 后的计数位置找；如果标签已错，先回到 validate 的转换和标签查询。

有 Git 实验基线时，修改前先用 `git status --short` 确认起始状态，用 `git diff -- data/tickets.json` 看这次实际改了什么。只有确认该文件的未提交变化都属于这次可丢弃的实验，才用 `git restore -- data/tickets.json` 精确恢复，然后重跑原始 JSON，检查重新得到章首报告。没有基线时，另建一份新实验副本即可，不要在真实项目里试恢复命令。

如果准备修改主程序，例如调整同优先级时的排列规则，应先写出新的业务预期，增加能区分旧规则和新规则的测试，再修改排序处；不能只改断言追随当前输出。原有测试随后继续运行，`git diff` 则帮助确认修改没有扩散到无关函数。A11 解释了工作区与提交的恢复关系；若要证明某个已测试 commit 能在新的隔离环境重建，使用 A12 的精确 commit 路线，并为修改后的版本重新取得证据。

本里程碑使用一个可以逐函数读完、修改和验证的小程序。判断完成情况，看的是能否追踪输入到报告、解释分工、修改规则并核对结果，不按代码行数判定。合理修改时可以增加或减少代码；不要为压缩行数把多个职责挤在一行，也不为凑规模加入无关功能。实验完成后保留你记下的目录；需要清理时先确认那个明确目录确实是自己的实验副本，本章没有自动删除目录的步骤。

现在，本地文件经过解析，得到原始记录；验证决定谁能继续，整理函数拥有接受列表、拒绝列表和已用编号，写出函数才改变磁盘报告。以后遇到“结果不对”，可以从错误输出所在的位置沿着这几个交接点往回找。B 册将增加跨进程的 HTTP 请求与响应；这里的 T-101、T-201 仍只是本例本地记录，当前程序没有数据库、监听端口或 HTTP 调用。

本章涉及的查用入口包括 Python 的内置类型与真值规则、CSV、JSON、Path、contextlib.suppress，以及 pytest 的临时替换工具。不需要先读完文档才能跟上本章；忘记某个具体参数时，带着当前输入、操作和预期结果去查。

知识关系总览 | 把程序与工具放回同一张图

前面的章节按学习顺序展开；这里换一个角度，把解决相近问题的知识放在一起，再看不同部分怎样合作。需要找回某个关系时，可以直接从相应标题读，不必重新运行整份工单程序。

先找位置：这些知识分别在管什么？

一个程序既有“怎样处理数据”的内部关系，也有“怎样运行、检查和重现”的外部条件。下面按各部分承担的工作定位，不把它们排成一条执行流水线。

看的是哪件事	放在一起理解的知识	与其他部分怎样连接
数据是什么，怎样取得和改变	值与类型、变量、字符串、列表、字典、集合；取值、计算、转换	操作接收值并产生结果；容器组织多份值，供查找、判断和逐项处理
下一步执行哪里	条件、循环、异常处理；`continue`、`break`、`return`	判断读取当前值；循环重复工作；状态变化影响下一轮；异常可能打断正常交接
工作怎样组织和交接	函数、参数、返回值、模块、导入	函数内部也有数据与控制；调用者交入值、接回结果，模块让代码能被别处使用
程序怎样接触外面的数据	输入输出、文本编码、CSV/JSON、文件读写	解析把文件内容变成程序里的值；写出把结果留存；格式正确与业务有效仍是两层判断
运行的是哪一份	路径、当前工作目录、解释器、依赖、配置、进程	这些条件影响实际执行；同一源码在错误目录或另一环境中，可能读取不同文件或无法导入依赖
怎样知道对不对，以后能否找回	观察、调试、测试、Git、依赖与运行说明、重建	观察提供事实，测试对照预期，Git 固定选中的内容；重新运行再检查内容与条件能否合作

这些不是互斥的盒子。例如，`return` 同时改变执行位置并交回值；文件读写既涉及数据，也可能产生异常。分类帮助找到入口，真正要理解的是它们之间的连接。

有了数据，为什么还需要条件、循环和函数？

先看一段完整的小程序：统计名单中有多少人已经确认参加。

PYTHON

```
people = [
    {"name": "Lin", "confirmed": True},
    {"name": "Mo", "confirmed": False},
]

count = 0
for person in people:
    if person["confirmed"]:
        count = count + 1

print(count)
```

列表把两个人的资料放在一起；每个人用字典把名字与确认状态放在对应字段下。`for` 每轮取出一个人，`person["confirmed"]` 从当前字典取出布尔值，`if` 再据此决定是否计数。

第一轮取到 Lin，条件是 `True`，`count` 从 0 变成 1；第二轮取到 Mo，条件是 `False`，不执行加一，`count` 仍是 1。列表没有下一项后，循环结束，最后显示 `1`。

这里有几种不同关系：

- 列表与字典是在组织数据，不会自己决定谁该被计数。
- `for` 负责逐项取出，`if` 负责当前这一项的选择；它们能够嵌套，因为不是在解决同一个问题。
- `count` 在循环前建立，才能把前一轮结果带到下一轮。如果每轮开头都重新设为 0，就不再是在累计整份名单。

这也帮助区分容易混淆的工具：

当前要做什么	相近写法的区别
从一组数据中取出内容	列表通常按位置取，字典按 key 取；`"confirmed" in person` 只检查有没有这个 key，不是取出它的值
看两份内容的关系	`==` 比较值是否相等；`in` 检查成员是否存在；方括号取出具体内容
决定执行次数与去向	`if` 在执行到这里时作选择；`for` 从可迭代对象逐项取得内容；`while` 在每轮开始前重新判断条件
留下整批处理的中间结果	列表可以保留待排序的完整记录；集合适合在这里只关心成员是否已出现的情况；字典适合按名字或 key 关联内容

类型不是只供背诵的名称。它影响什么操作有意义：`"2" + "3"` 得到文字 `"23"`，`2 + 3` 得到整数 `5`。输入的表示、要做的操作和后面期待的结果要接得上，才知道转换为什么需要出现。A02 细讲值与操作，A03–A04 细讲选择、重复及跨轮状态。

函数把哪些关系包在一起，又怎样交出来？

函数不是排在“数据、条件、循环”之后的另一道必经工序。它可以把其中一段工作组织成可调用的整体：需要什么值通过参数传入，计算出的结果通过返回值交回。

下面把小时换成分钟，再判断是否达到半小时：

```python
def to_minutes(hours):
    return hours * 60

minutes = to_minutes(0.5)
if minutes >= 30:
    print("At least half an hour")

assert to_minutes(0.5) == 30
```

调用 `to_minutes(0.5)` 时，`hours` 接住 `0.5`；乘法得到 `30.0`，`return` 把它交回调用位置。外面的 `minutes` 接住结果，随后 `if` 用这个值判断。最后的断言是另一次调用，用已知换算关系检查结果；`30.0 == 30` 为真，所以不报错。

```
调用者的值 → 参数接住 → 函数内部处理 → return 交回
                              ↓
                  调用位置继续：赋值、计算、判断或测试
```

函数负责完成一项工作，不代表它必须同时决定结果的一切用途。 换算函数只需要交回分钟数；是否达到半小时，由调用者按当前需要判断。另一处可以用同一返回值计算费用，测试也可以直接比较它，无需先从终端文字里猜答案。

因此，`return`、`print` 和写文件不能只归成“输出”：`return` 把值交给调用者，`print` 把文字送往标准输出，写文件把数据交给指定文件。如果函数只有 `print(hours * 60)` 而没有返回分钟数，调用者得到的会是 `None`，不是屏幕上显示的数字。返回值也可以是列表、字典或布尔值，并不只限于数字。

函数分工还与状态有关。只检查一个人的字段，可以每次从空的错误列表开始；判断整份名单是否有重复，却需要保留前面出现过的编号。把状态放在谁那里，会影响谁有足够信息作判断。这是从语法走向设计的一步：不是先规定“必须拆几个函数”，而是先弄清各项工作需要知道什么、应该交回什么。

模块继续组织可复用的代码，import 让当前程序能够使用它。入口保护 if __name__ == "__main__" 可以让直接启动时的工作不在导入时自动发生；这又使测试能够自己选择参数和调用对象。入口保护只管其下面的那段代码，不会阻止模块中其他顶层语句执行。A02、A06 与 A07 由参数、返回值、导入和测试这些实际交接连在一起。

为什么“处理失败”和“再做一次”要分开理解？

int("cat") 无法产生整数，会引发 ValueError。若这一行是 number = int("cat")，右边失败，赋值就没有完成；新执行中此前未绑定过 number，后面直接使用它就可能出现 NameError。若此前已有绑定，则不能误认为这次失败把旧值更新了。

相反，int("-3") 可以成功得到整数 -3。它是否适合报名人数，是普通业务判断的问题。**异常处理解决操作没能正常完成时由谁接手，业务条件判断解决已经得到的值是否符合当前要求。**

这些机制与重复、函数交接的关系，可以按“改变哪个执行范围”分清：

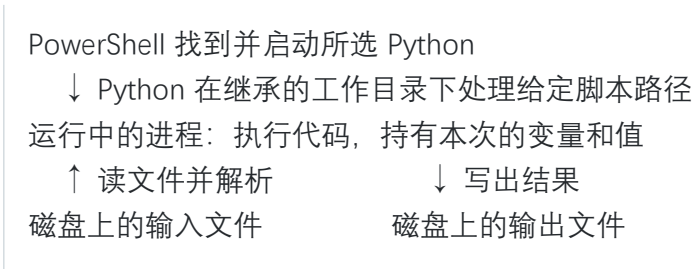
机制	实际改变什么
try/except	在被保护的执行中出现匹配异常时改走处理分支；不会自动返回输入处
try 的 else	正常执行完 try、没有异常且没有提前用 return、break、continue 离开时，承接成功路线
循环	负责下一轮的推进或判断；转换抛出异常时，可先接住异常再继续循环重问，业务条件不合格时也可直接进入下一轮
pass / continue	pass 本身不做事，随后照常执行；continue 跳过最近循环本轮余下部分，进入下一轮的推进或判断
break / return	break 结束最近的循环；return 结束当前函数调用，把结果交回调用者，即使它写在循环里面也是如此

try 的范围不必包住整份程序：只保护预计可能失败的操作，成功后依赖其结果的工作放到合适的后续位置，才能看清失败由哪里接住。也不能为了“程序不报错”就把错误全部吞掉；接住失败后，是继续、重问、返回拒绝说明还是停止，要由当前工作决定。

A05 把这些路线逐步展开；这里把它接回前面的图：失败会改变控制去向，未完成的操作影响可用的值，捕获位置又划出函数之间的失败交接。

内存、文件、环境和进程怎样连在一起？

程序里的一个字典，不等于磁盘上的一份 JSON 文件；磁盘上的脚本，也不等于正在运行的进程。把这些对象分开，很多“我明明改了，怎么没变化”的问题才有查找入口。



这是本册普通脚本启动与文件处理的关系，不表示每个程序都必须读写文件。相对路径以实际使用时的当前工作目录为起点；脚本目录与工作目录可能不同。A01 先确认运行目标，A09 再区分脚本位置、工作目录和实际解释器，A10 观察启动以后的那次运行。

同一份列表可以先只存在于内存中。要下次运行继续使用，就需要明确写出，并在下次明确读取。JSON 或 CSV 负责约定文件怎样表示内容，解析器负责把内容读成相应的值；解析成功仍不保证字段齐全、人数合理或编号不重复。文件格式、数据结构和业务规则彼此衔接，但不能互相代替。

运行条件也要有真实连接：

- 安装依赖把包放入所选环境；`import` 从当前解释器可用的位置加载代码，不负责代替安装。装在另一个 Python 的环境里，当前 Python 仍可能找不到它。
- 虚拟环境帮助隔离依赖，依赖文件记录如何重新安装；两者不会自动修复业务逻辑。
- 配置只有被代码读取并使用时才会影响行为。环境变量不会仅凭名字相同就自动参与某个函数的判断。
- 一份脚本可以启动多次，产生不同进程；进程各有自己的运行状态。看到 PID 只说明查到了某次运行，不证明它已经完成报告或成功回应请求。

结果的去向也影响观察方式：报告看文件内容，命令结束看退出状态，长期服务还要看监听和实际回应。退出码与 HTTP 状态码不是同一种证据。屏幕出现一句成功文字，也不能代替核对新文件是否真的写出。

正确运行一次，为什么还需要测试、版本和重建？

这些工具围绕同一份工作提供不同帮助，不是工单数据必须逐一经过的处理站。

想弄清的事	工具怎样参与	不能由此推定什么
实际走到了哪里、值是什么	输出、日志、调试器提供观察，帮助找最早差异	看见一次输出不等于所有输入都正确

想弄清的事	工具怎样参与	不能由此推定什么
行为是否符合要求	测试给定输入与预期，用断言比较实际结果；pytest 组织运行	测试不会自动修代码，未覆盖的行为不因此正确
哪些代码内容被保留	Git 比较变化、保存选中的快照，并支持从确定来源找回内容	提交成功不等于代码正确，也不等于安装环境被保存
换个干净目录和环境还能否运行	按确定版本、依赖和说明重新建立运行条件，再检查结果	同机重建成功不等于已验证所有电脑、系统和版本

测试粒度来自工作分工。一个只负责换算的函数，可以独立核对返回值；涉及文件路径、解析和写出的工作，还需要经过真实入口检查。这两类证据互补，不能拿局部函数通过代替整个文件处理成功。

Git 则把“当时测试的是哪份内容”这个问题接了进来：

> 编辑并保存 → 工作区文件
> git add → 选取当时的内容进入暂存区
> git commit → 根据暂存区建立提交

Python 通常运行工作区文件。如果 `add` 后又改了代码，刚测试的内容可能已经不同于即将提交的内容。普通 `git diff` 比较工作区与暂存区，`git diff --staged` 比较暂存区与当前提交；确认这些位置，才知道行为证据对应哪份代码。

A12 把确定提交、依赖说明与 README 接到新环境。源码规定怎样处理数据，依赖说明帮助准备需要的包，README 说明入口与预期。旧 `.venv` 不能代替重建说明；只恢复源码，也不会自动把已经生成的报告恢复成旧结果。保存了什么、运行了什么、产生了什么，始终需要分别确认。

回到工单：一次变化怎样穿过这些关系？

用一个具体变化检验上面的结构。A13 的工单整理器接收 CSV 或 JSON，逐条检查、去重和统计，再写出 JSON 报告；它没有数据库，也不监听端口。下面是完整的两条 JSON 输入：

```JSON
[
  {"id": "T-201", "subject": "VPN cannot connect", "priority": 1, "status":
    "open", "owner": "Wei"},
  {"id": "T-202", "subject": "Replace keyboard", "priority": 3, "status":
    "pending", "owner": ""}
]
```

编号是 `id`，标题是 `subject`；优先级允许 1、2、3，状态允许 `open`、`pending`、`closed`，空负责人补成 `unassigned`。两条原记录都合格，报告接受 2 条、拒绝 0 条；成功写出后进程退出码为 0。这里只沿这些已给出的字段观察重复编号，不需要记得其他旧输入。

若在自己的实验副本里，只把第二条 `id` 改为 `"T-201"`，各部分为什么会有不同反应？

知识关系落到哪里	这次实际发生什么
文件 → 内存里的值	`load_rows` 仍能解析 JSON；编号重复不会破坏 JSON 格式
单条函数的职责	`validate_row` 仍认为第二条字段有效，因为非空编号、标题、优先级与状态都合格；它没有前面记录的集合
跨轮状态 → 条件选择	`organize_rows` 的 `seen_ids` 已记住第一条被接受的 T-201，因此第二条命中重复判断
控制去向 → 后续数据	保存重复原因后执行 `continue`，第二条不进入 `accepted`；统计遍历接受列表，因此只计第一条
返回值 → 文件与退出证据	整理结果是输入 2、接受 1、拒绝 1；报告写出成功后，`run` 返回 2，由外层 `sys.exit` 交成进程退出码

第一条仍是 open、优先级 1，所以对应统计各为 1；pending 和优先级 3 都变成 0。拒绝原因指向第 2 条的重复编号。这里的非零退出码表示“报告已完成，但有拒绝记录”，不是程序崩溃。

`seen_ids` 放在整理函数的循环外，才记得前面接受了谁；每次调用整理函数时又重新建立，因而不延续上次运行的集合。若前一条同编号记录先因空标题被拒绝，它不会占用编号，后一条有效记录仍可被接受。这说明去重规则不仅与集合有关，还与“何时登记、登记谁”有关。

这种分工直接影响怎样测试：只调用第二条的 `validate_row`，不能证明去重正确；必须把两条一起交给整理函数，检查接受列表、拒绝原因与统计。文件入口的验证再核对指定输入、新报告及退出码；Git 的差异则说明究竟改了哪份输入，不能代替行为测试。

如果只是文件里保存了一份旧报告，不能据此认定这次也写出成功。A13 的写出函数先写完并关闭同目录临时文件，再替换目标；这避免一开始就截断旧报告，但不提供断电、强杀或并发写入的普遍恢复保证。坏 JSON 则在载入时就改走失败路线，不会继续生成新报告。

由此可以看到：数据结构影响怎样查找，状态位置影响能作什么判断，函数分工影响返回与测试边界，文件和进程又决定结果怎样留下证据。后面的 B 册把值交给另一个程序时，仍从实际交接出发；网络是新的交接条件，不会取消这些已有关系。

忘记细节时，从哪个问题找回

现在最不确定什么	先找哪段关系
改了却没有变化	A01 的运行目标；A09 的实际解释器和配置来源
数值不对或函数没交回结果	A02 的值和 return；A07 的预期、实际与最早差异
分支或循环走错	A03 的条件顺序；A04 的本轮状态和停止位置
一次失败后没有重新询问	A05 的异常改道与循环重试
import 失败或导入就开始运行	A06 的模块入口；A09 的安装与运行对象
文件读得出但结果不合格	A08 的格式与业务分层；A13 的字段及写出边界
看见进程却不能访问	A10 的身份、监听、请求与停止证据
不知道哪份改动会保存，或怎样找回	A11 的工作区、暂存区与提交
只在旧目录能运行	A12 的确定快照、依赖说明与新环境

复习时不必背这几张表的排列。能解释两个工具为什么不同、哪里可以组合，或沿着一次变化找到受影响的结果，才是在恢复关系。常用写法会在实际使用中逐渐熟悉，完整命令和低频参数仍可以查回原章。

后面的文件操作查用页只在整理实验材料时打开；它把此前的路径和状态关系用于复制、移动和清理，不是进入下一册前要背完的命令附录。

文件操作查用 | 先核对目标，再复制、移动或清理

整理实验目录时，最容易丢东西的不是不会写 Python，而是把命令用在了另一份文件上。这里补齐一个小动作：保留原始样例，复制一份用于检查，再移动副本；最后只清理明确不要的那一份。

这段适合在 A11 保存与恢复关系清楚以后查用。它不改变 Git 历史，不碰随书发布目录，也不要求先掌握一套 Shell 自动化。

引号是在给哪一个参数划边界?

假设文件名是 `sample [1].txt`。空格属于名字，不是分开两个参数的意思；方括号也属于名字，不是要匹配某组字符。使用带引号的完整文字，并让支持这个参数的文件命令按字面解释路径：

```powershell
Get-Item -LiteralPath '.\sample [1].txt'
```

`-LiteralPath` 的意思是不要把路径里的方括号等字符当作通配模式。引号解决文字怎样交给命令，LiteralPath 解决命令怎样理解这段路径；两者不是同一个保护。

单引号和双引号在 PowerShell 中还对变量采取不同态度：

```powershell
$label = 'sample'
'copy of $label'
"copy of $label"
```

结果分别是 `copy of $label` 和 `copy of sample`。单引号保留字面文字，双引号会展开变量。比如输出一份实际路径可以用 `"source=$sourcePath"`；希望文字中的 `$` 原样留下，则单引号更直接。先看当前究竟要传字面内容还是变量值，不用把所有特殊字符规则一次背下。

原件与副本怎样分开?

先新建自己的实验目录，后续只操作这里：

```powershell
$filesLab = Join-Path ([IO.Path]::GetTempPath()) ('fde-files-' +
  [guid]::NewGuid().ToString('N'))
if (Test-Path -LiteralPath $filesLab) { throw 'Expected a new directory.' }
New-Item -ItemType Directory -Path $filesLab -ErrorAction Stop | Out-Null
Set-Location -LiteralPath $filesLab
```

在这个目录用编辑器建立 `sample [1].txt`，内容写成 `original example`。只是一份新练习文字，不使用个人资料。然后取得原文件与新副本的明确位置：

```powershell
$sourcePath = (Get-Item -LiteralPath '.\sample [1].txt' -ErrorAction
  Stop).FullName
$copyPath = Join-Path $filesLab 'working-copy.txt'
if (Test-Path -LiteralPath $copyPath) { throw 'Copy target already exists.' }
Get-Item -LiteralPath $sourcePath | Select-Object FullName, Length
Copy-Item -LiteralPath $sourcePath -Destination $copyPath -WhatIf
```

`-WhatIf` 只显示命令打算做的动作，还没有建立副本。先看它报告的源和目标；明确正确后，再执行真正的复制：

```powershell
if (Test-Path -LiteralPath $copyPath) { throw 'Copy target already exists.' }
Copy-Item -LiteralPath $sourcePath -Destination $copyPath -ErrorAction Stop
Get-Content -LiteralPath $sourcePath
Get-Content -LiteralPath $copyPath
```

现在两份都应读到 `original example`。复制保留了源文件，新增了一个普通副本；之后改副本不会自动改原件。WhatIf 不等于“今后执行必然成功”，预览到真正执行之间目标和权限仍可能变化。本例在自己独立、没有其他写入者的目录操作；这些检查不是并发文件操作的锁。

移动以后，还剩哪一份？

给副本换一个归档名字。先预览并核对，再实际移动：

```powershell
$archivePath = Join-Path $filesLab 'checked-copy.txt'
if (Test-Path -LiteralPath $archivePath) { throw 'Move target already exists.'
  }
Move-Item -LiteralPath $copyPath -Destination $archivePath -WhatIf
```

此时 `working-copy.txt` 仍在，因为还没动。确认后执行：

```powershell
if (Test-Path -LiteralPath $archivePath) { throw 'Move target already exists.'
  }
Move-Item -LiteralPath $copyPath -Destination $archivePath -ErrorAction Stop
Test-Path -LiteralPath $copyPath
Get-Content -LiteralPath $archivePath
Get-Content -LiteralPath $sourcePath
```

旧副本路径返回 `False`，新路径可以读到内容，原件仍然可读。这里在同一目录给普通文件改名；移动和复制的关键区别，是成功移动后不再保留原来的副本路径。

若只想删除刚才的归档副本，先重新打开并核对它，不把目标扩大为整个目录：

```powershell
Get-Item -LiteralPath $archivePath | Select-Object FullName, Length
Get-Content -LiteralPath $archivePath
Remove-Item -LiteralPath $archivePath -WhatIf
```

真正决定不要这份练习副本以后，才去掉 WhatIf：

```powershell
Remove-Item -LiteralPath $archivePath -ErrorAction Stop
Test-Path -LiteralPath $archivePath
Get-Content -LiteralPath $sourcePath
```

应先得到 `False`，随后仍能读到原件。`Remove-Item` 通常不送进回收站，不能把它当作保证可撤销的删除。这个实验故意留下原始样例和目录，也不提供递归清空命令。

Git 只对已经记录的内容提供相应找回路线；一个从未提交、没有其他副本的文件，不能指望删除后一定由 Git 找回。操作前先分清原件、副本、当前目录和目标，远比执行完再寻找“撤销”可靠。

查用入口：PowerShell 引号规则、Copy-Item、Move-Item、Remove-Item。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

这些章节不是互不相干的语法课和工具课。它们共同解决一个问题：怎样让程序处理数据，并让人能够看清、验证和重新运行它？先从下面这些普通问题认识全貌，具体写法随后再学。

程序本身怎样工作?	数据：拿着什么值，怎样组织、取出和改变？ 执行：下一步做什么，何时选择、重复或因失败改道？ 分工：哪些工作放在一起，怎样传入和交回结果？ 输入输出：数据怎样进来，结果给谁、留在哪里？
怎样让这份程序可检查、可再次运行?	运行条件：哪份代码、哪个 Python、什么目录和依赖？ 观察与验证：实际发生了什么，是否符合预期？ 版本与重建：保存的是哪份内容，能否按说明重新运行？

这是观察同一个程序的几个角度，不是程序必须依次经过的步骤，也不是需要背下来的分类。数据是处理的对象，执行路线决定先后与次数，函数把一项工作组织起来并交接结果；输入输出则把程序里的值接到人、文件或其他程序。同一段代码往往同时涉及这些关系。

例如，处理一份报名名单时，名单先组织多个人的资料，循环逐人取出，条件判断决定这一人是否符合要求；这项检查可以放进函数，交回结果后再由调用者决定怎样记录。这里不是四种互相替代的办法，而是几个部分各管一件事、合起来完成工作。

外围工具也各有所用：测试把实际行为与预期比较，Git 保存明确选择的代码内容，环境与说明帮助下一次运行。它们并不代替程序处理名单。**程序怎样产生结果，与人怎样确认、保存和重现这份工作，是相互关联但不能互相代替的两件事。**

早期用报名、问候语、名单等较小例子看清局部；最后的工单整理器再把这些关系组合起来：读取一份文件，接受合格记录、留下拒绝原因，写出报告。各例需要的输入和背景会在用到时给出。

读完本册正文后，可以回到后部的“知识关系总览 | 把程序与工具放回同一张图”按知识关系重新整理：相近的工具怎样区分，不同部分怎样组合，忘记细节时从哪里找回。第一次先认清要探索什么，不必提前读完那份详细梳理。

这册怎样展开

阅读位置	逐渐能回答的问题
A01–A02	命令运行谁？输入怎样成为后续能用的值？

阅读位置	逐渐能回答的问题
A03–A05	为什么走这条路？下一轮从哪里来？失败以后谁负责继续？
A06–A08	别处的代码怎样参与？怎样验证修改？运行结束后数据怎样保留？
A09–A12	实际环境是什么？进程怎样观察和停止？保存的内容怎样被重新运行？
A13	一条工单怎样经过各段处理，成为完整报告？

可以按自然问题标题暂停，不必一口气读完整章。第一次先顺着给出的输入与结果追踪；到实际运行段再用电脑核对。章节里的完整参数和长操作配方可以查用，承担判断的关系则需要能解释：为什么用这个动作，改变了哪份值或文件，什么证据才支持成功。

“不用逐字背代码”不等于看懂一次就够。`input`、条件、循环、函数调用、读写、运行与比较这些常用动作，会在后面的真实任务中反复出现，应逐渐用顺；不必在第一次遇到时就默写整份程序。若讲解处已经能预测一处改变会带来什么，就不用为形式再加一组同类题。

书中的上机任务只在自己选择的新实验目录或明确复制的配套副本里运行。原始随书文件保留作读取和复制来源，不在里面建立环境、生成报告或练习恢复。短例可按正文保存为 UTF-8 文本；综合程序位于资料包根目录下的 `01_配套练习/examples/a_program_tools/ticket-organizer`，A12、A13 会给出具体复制与运行入口。

这一册之后，B 册才把本地值放进 HTTP 请求，交给另一个程序。当前先看清一次本地运行，后面遇到多个服务时才有可以复用的判断基础。